1 Algorithms (358)

Searching, Sorting, Hashing, Asymptotic worst case time and Space complexity, Algorithm design techniques: Greedy, Dynamic programming, and Divide-and-conquer, Graph search, Minimum spanning trees, Shortest paths.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	4	4	2	2	1	2	2	2	3	2	1
2 Marks Count	6	4	3	3	4	2	2	2	3	4	2
Total Marks	16	12	8	8	9	6	6	6	9	10	**6**

Welcome to the "Algorithms" chapter, a cornerstone of Computer Science and a high-scoring section in the GATE examination. This subject delves into the systematic procedures and computational methods used to solve problems, focusing on their efficiency and correctness. Mastering algorithms is crucial not only for theoretical understanding but also for practical application in software development and system design. In GATE CS, algorithms typically carry a significant weightage, often ranging from 10 to 15 marks, with questions spanning theoretical concepts, time and space complexity analysis, problem-solving, and application of various design techniques. Expect a mix of Multiple Choice Questions (MCQ), Multiple Select Questions (MSQ), and Numerical Answer Type (NAT) questions, requiring a deep understanding of underlying principles and the ability to analyze and compare different algorithmic approaches.

Topic-wise Key Concepts

Algorithm Design

Definition: Algorithm design is the process of creating a well-defined computational procedure to solve a specific problem. It involves understanding the problem, choosing appropriate data structures, and devising a step-by-step method that is correct, efficient, and terminates.

- **Properties/Identities:**
  - **Correctness:** An algorithm must produce the correct output for all valid inputs.
  - **Efficiency:** Measured by time and space complexity.
  - **Finiteness:** Must terminate after a finite number of steps.
  - **Definiteness:** Each step must be precisely defined.
  - **Input/Output:** Must take zero or more inputs and produce one or more outputs.
- **Pitfalls/Tricks:**
  - Overlooking edge cases or constraints that might break the algorithm's logic.
  - Designing an algorithm that is correct but too inefficient for practical use or given constraints.
- **Techniques/Shortcuts:**
  - Start with a brute-force solution, then optimize.
  - Break down complex problems into smaller, manageable subproblems.
  - Consider different data structures and their impact on performance.

Algorithm Design Techniques

Definition: These are general strategies or paradigms used to develop algorithms for a wide range of problems. Key techniques include Divide and Conquer, Greedy Algorithms, Dynamic Programming, Backtracking, and Branch and Bound.

- **Properties/Identities:**
  - **Divide and Conquer:** Divide problem into subproblems, solve recursively, combine solutions.
  - **Greedy Algorithms:** Make locally optimal choices at each step, hoping to find a global optimum.
  - **Dynamic Programming:** Solves problems with overlapping subproblems and optimal substructure by storing results of subproblems.
- **Pitfalls/Tricks:**
  - Mistaking a problem for one solvable by a greedy approach when it requires dynamic programming.
  - Incorrectly identifying optimal substructure or overlapping subproblems.
- **Techniques/Shortcuts:**
  - For DP, identify the state, recurrence relation, and base cases.
  - For Greedy, prove the greedy choice property and optimal substructure.

Asymptotic Notations

Definition: Mathematical notations used to describe the limiting behavior of a function when the argument tends towards a particular value or infinity, primarily used to classify algorithms by their running time or space requirements as input size grows.

- **Formulas/Theorems:**
  - **Big-O Notation (Upper Bound):** $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
  - **Omega Notation (Lower Bound):** $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.
  - **Theta Notation (Tight Bound):** $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.
  - **Little-o Notation (Strict Upper Bound):** $f(n) = o(g(n))$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$.
  - **Little-omega Notation (Strict Lower Bound):** $f(n) = \omega(g(n))$ if $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$.
- **Properties/Identities:**
  - Reflexivity: $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$, $f(n) = \Theta(f(n))$.
  - Transitivity: If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$. (Applies to all notations).
  - Symmetry: $f(n) = \Theta(g(n))$ iff $g(n) = \Theta(f(n))$.
  - Transpose Symmetry: $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$. Similarly for o and ω.
  - Sum Rule: If $f(n) = O(g(n))$ and $h(n) = O(g(n))$, then $f(n) + h(n) = O(g(n))$. (Take the maximum order term).
  - Product Rule: If $f(n) = O(g(n))$ and $h(n) = O(p(n))$, then $f(n) \cdot h(n) = O(g(n) \cdot p(n))$.
- **Pitfalls/Tricks:**
  - Confusing Big-O with Theta: Big-O is an upper bound, Theta is a tight bound.
  - Ignoring constants and lower-order terms: Asymptotic notation focuses on growth rate.
  - Assuming $O(f(n))$ means exactly $f(n)$ operations; it means "at most proportional to $f(n)$".
- **Techniques/Shortcuts:**
  - For polynomials, the highest degree term dominates. E.g., $3n^2 + 5n + 10 = O(n^2)$.
  - Logarithms grow slower than any polynomial: $\log n = O(n^\epsilon)$ for any $\epsilon > 0$.
  - Exponentials grow faster than any polynomial: $n^k = O(a^n)$ for any $a > 1$.

Bellman Ford

Definition: A single-source shortest path algorithm that can handle graphs with negative edge weights. It works by repeatedly relaxing all edges in the graph, detecting negative cycles if they exist.

- **Formulas/Theorems:**
  - **Relaxation Step:** For an edge (u, v) with weight $w(u, v)$, if $d[u] + w(u, v) < d[v]$, then $d[v] = d[u] + w(u, v)$.
  - **Number of Iterations:** $|V| - 1$ iterations are sufficient to find shortest paths in a graph with $|V|$ vertices, assuming no negative cycles reachable from the source.
  - **Negative Cycle Detection:** After $|V| - 1$ iterations, if any edge can still be relaxed in the $|V|$-th iteration, a negative cycle exists.
- **Properties/Identities:**
  - Works on directed and undirected graphs (undirected edges treated as two directed edges).
  - Can detect negative cycles.
  - Time Complexity: $O(|V| \cdot |E|)$.
  - Space Complexity: $O(|V|)$ for distance array, $O(|V|)$ for predecessor array.
- **Pitfalls/Tricks:**
  - Incorrectly assuming it's faster than Dijkstra's for non-negative weights (Dijkstra's is $O(E \log V)$ or $O(E + V \log V)$).
  - Forgetting to initialize distances (source to 0, others to infinity).
- **Techniques/Shortcuts:**
  - Visualize the relaxation process step-by-step.
  - For negative cycle detection, run one extra iteration after $|V| - 1$ iterations.

Binary Heap

Definition: A complete binary tree that satisfies the heap property: for a min-heap, every node's value is less than or equal to its children's values; for a max-heap, every node's value is greater than or equal to its children's values.

Typically implemented using an array.

- **Formulas/Theorems:**
  - **Parent of node i:** $\lfloor (i-1)/2 \rfloor$ (for 0-indexed array).
  - **Left child of node i:** $2i+1$ (for 0-indexed array).
  - **Right child of node i:** $2i+2$ (for 0-indexed array).
  - **Height of a heap with N nodes:** $\lfloor \log_2 N \rfloor$.
- **Properties/Identities:**
  - **Min-Heap:** Root is the minimum element.
  - **Max-Heap:** Root is the maximum element.
  - **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
  - **Operations Time Complexity:**
    - Insert: $O(\log N)$
    - Delete-Min/Max: $O(\log N)$
    - Build-Heap (from an array of N elements): $O(N)$
- **Pitfalls/Tricks:**
  - Confusing min-heap with max-heap properties.
  - Incorrectly calculating parent/child indices, especially for 1-indexed vs. 0-indexed arrays.
- **Techniques/Shortcuts:**
  - When building a heap, start from the last non-leaf node and call `heapify` upwards.
  - Heap sort uses a max-heap.

Binary Search

Definition: An efficient algorithm for finding an item from a sorted list of items. It repeatedly divides the search interval in half, eliminating half of the remaining elements at each step.

- **Formulas/Theorems:**
  - **Recurrence Relation:** $T(N) = T(N/2) + O(1)$.
  - **Time Complexity:** $O(\log N)$.
  - **Space Complexity:** $O(1)$ (iterative) or $O(\log N)$ (recursive due to call stack).
- **Properties/Identities:**
  - Requires the input array/list to be sorted.
  - Can be used to find the first/last occurrence of an element, or the ceiling/floor.
- **Pitfalls/Tricks:**
  - Forgetting to handle edge cases like empty arrays or target not found.
  - Off-by-one errors in index calculations (e.g., `mid = (low + high) / 2`). Use `mid = low + (high - low) / 2` to prevent overflow.
- **Techniques/Shortcuts:**
  - Always ensure the search space is correctly updated (`low = mid + 1` or `high = mid - 1`).
  - For finding first/last occurrence, adjust the update logic to keep searching in the relevant half even after finding a match.

Binary Search Tree (BST)

Definition: A binary tree where for each node, all values in its left subtree are less than the node's value, and all values in its right subtree are greater than the node's value. No duplicate values are typically allowed.

- **Properties/Identities:**
  - **Inorder Traversal:** Produces elements in sorted order.
  - **Average Time Complexity:**
    - Search, Insert, Delete: $O(\log N)$
  - **Worst-Case Time Complexity (skewed tree):**
    - Search, Insert, Delete: $O(N)$
  - **Space Complexity:** $O(N)$ for storing nodes.
- **Pitfalls/Tricks:**
  - Forgetting to handle the case of deleting a node with two children (replace with inorder successor/predecessor).
  - Assuming $O(\log N)$ performance always; a skewed BST degrades to $O(N)$.
- **Techniques/Shortcuts:**
  - To find minimum, traverse left until null. To find maximum, traverse right until null.

- For deletion, the inorder successor is the smallest node in the right subtree.

Binary Tree

Definition: A tree data structure where each node has at most two children, referred to as the left child and the right child. It's a fundamental non-linear data structure.

- **Formulas/Theorems:**
  - **Maximum nodes at level i (root at level 0):** 2^i.
  - **Maximum nodes in a binary tree of height h:** $2^{h+1} - 1$.
  - **Minimum height of a binary tree with N nodes:** $\lceil \log_2(N+1) \rceil - 1$.
- **Properties/Identities:**
  - **Full Binary Tree:** Every node has either 0 or 2 children.
  - **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
  - **Perfect Binary Tree:** All internal nodes have two children and all leaves are at the same level. (A perfect binary tree is always full and complete).
  - **Skewed Binary Tree:** All nodes have only one child (either left or right).
- **Pitfalls/Tricks:**
  - Confusing different types of binary trees (full, complete, perfect).
  - Incorrectly calculating height or number of nodes for specific tree types.
- **Techniques/Shortcuts:**
  - Understand the relationship between height and number of nodes for different tree types.
  - Practice drawing trees from traversals.

Breadth First Search (BFS)

Definition: A graph traversal algorithm that explores all the neighbor nodes at the present depth before moving on to nodes at the next depth level. It uses a queue data structure.

- **Properties/Identities:**
  - Finds the shortest path in terms of number of edges (unweighted graphs).
  - Guaranteed to visit all reachable vertices.
  - Time Complexity: $O(|V| + |E|)$ for adjacency list, $O(|V|^2)$ for adjacency matrix.
  - Space Complexity: $O(|V|)$ for queue and visited array.
- **Pitfalls/Tricks:**
  - Forgetting to mark nodes as visited to prevent cycles and redundant processing.
  - Using BFS for weighted shortest paths (Dijkstra's is needed).
- **Techniques/Shortcuts:**
  - Think of BFS as "level-by-level" exploration.
  - Useful for finding connected components, shortest path in unweighted graphs, and checking bipartiteness.

Bubble Sort

Definition: A simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted.

- **Formulas/Theorems:**
  - **Worst-case Time Complexity:** $O(N^2)$ (e.g., reverse sorted array).
  - **Average-case Time Complexity:** $O(N^2)$.
  - **Best-case Time Complexity:** $O(N)$ (already sorted array, with optimization to stop if no swaps).
  - **Space Complexity:** $O(1)$ (in-place).
  - **Number of Swaps (Worst Case):** $N(N-1)/2$.
  - **Number of Comparisons (Worst Case):** $N(N-1)/2$.
- **Properties/Identities:**
  - Stable sorting algorithm.
  - Adaptive (can detect if an array is sorted and stop early).
- **Pitfalls/Tricks:**
  - Often used as a baseline for comparison due to its simplicity and inefficiency.
  - Not suitable for large datasets.

- **Techniques/Shortcuts:**
  - Understand its passes: after k passes, the last k elements are in their correct sorted positions.

Computer Science

Definition: The study of computation and information, including theoretical foundations, algorithmic design, and practical implementation. Algorithms are a core theoretical and practical component.

- **Properties/Identities:**
  - Algorithms are fundamental to all areas of Computer Science.

Depth First Search (DFS)

Definition: A graph traversal algorithm that explores as far as possible along each branch before backtracking. It uses a stack (explicit or implicit via recursion).

- **Properties/Identities:**
  - Can be used to find paths between two nodes, detect cycles, find connected components, and perform topological sorting.
  - Time Complexity: $O(|V| + |E|)$ for adjacency list, $O(|V|^2)$ for adjacency matrix.
  - Space Complexity: $O(|V|)$ for recursion stack or explicit stack, and visited array.
- **Pitfalls/Tricks:**
  - Can lead to very deep recursion for long paths, potentially causing stack overflow.
  - Does not guarantee shortest paths in unweighted graphs (BFS does).
- **Techniques/Shortcuts:**
  - Think of DFS as "going deep" before "going wide".
  - Useful for cycle detection, topological sort, and finding strongly connected components.

Dijkstra's Algorithm

Definition: A single-source shortest path algorithm for graphs with non-negative edge weights. It uses a greedy approach and a priority queue to efficiently find the shortest paths from a source vertex to all other vertices.

- **Formulas/Theorems:**
  - **Relaxation Step:** Same as Bellman-Ford.
  - **Time Complexity:**
    - With min-priority queue (binary heap): $O(|E| \log |V|)$ or $O(|E| + |V| \log |V|)$ if using Fibonacci heap.
    - With adjacency matrix (no priority queue, simple array scan): $O(|V|^2)$.
  - **Space Complexity:** $O(|V| + |E|)$ for graph, $O(|V|)$ for distances/predecessors.
- **Properties/Identities:**
  - Does not work correctly with negative edge weights.
  - Greedy algorithm.
  - Guaranteed to find the shortest path.
- **Pitfalls/Tricks:**
  - Applying it to graphs with negative edge weights (use Bellman-Ford instead).
  - Forgetting to update distances in the priority queue (decrease-key operation).
- **Techniques/Shortcuts:**
  - Always pick the unvisited vertex with the smallest known distance.
  - Visualize the "settling" of vertices.

Directed Acyclic Graph (DAG)

Definition: A directed graph that contains no directed cycles. This property makes them useful for representing processes with dependencies, like task scheduling.

- **Properties/Identities:**
  - Can always be topologically sorted.
  - No path can revisit a vertex.
  - Every finite DAG has at least one source (in-degree 0) and at least one sink (out-degree 0).
- **Pitfalls/Tricks:**
  - Confusing DAGs with general directed graphs that might have cycles.
  - Trying to apply algorithms that assume cycles (e.g., finding strongly connected components) without

modification.
- **Techniques/Shortcuts:**
  - Topological sort is a key algorithm for DAGs.
  - Dynamic programming problems on graphs often involve DAGs.

Double Hashing

Definition: A collision resolution technique in hashing that uses two hash functions. When a collision occurs with the first hash function, the second hash function is used to determine the step size for probing the hash table.

- **Formulas/Theorems:**
  - **Primary Hash Function:** $h_1(k)$.
  - **Secondary Hash Function:** $h_2(k)$. $h_2(k)$ must never return 0. Often $h_2(k) = R - (k \pmod R)$ for a prime $R < \text{table\_size}$.
  - **Probe Sequence:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod M$, where M is table size and i is probe number (0, 1, 2, ...).
- **Properties/Identities:**
  - Minimizes clustering compared to linear probing.
  - Provides better distribution of keys.
  - Requires $h_2(k)$ to be relatively prime to M to ensure all slots are probed.
- **Pitfalls/Tricks:**
  - Choosing a secondary hash function that can return 0 or is not relatively prime to the table size, leading to incomplete probing.
  - Incorrectly implementing the modulo operation for negative results.
- **Techniques/Shortcuts:**
  - Ensure M is a prime number and R is a prime smaller than M for $h_2(k) = R - (k \pmod R)$.

Dynamic Programming

Definition: An algorithmic technique for solving complex problems by breaking them down into simpler subproblems. It's applicable when subproblems overlap and the problem exhibits optimal substructure. Solutions to subproblems are stored to avoid recomputation (memoization or tabulation).

- **Properties/Identities:**
  - **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems.
  - **Overlapping Subproblems:** The same subproblems are encountered multiple times.
  - **Memoization (Top-down):** Store results of subproblems as they are computed.
  - **Tabulation (Bottom-up):** Solve subproblems in a specific order, typically filling a table.
- **Pitfalls/Tricks:**
  - Incorrectly identifying the state of the DP problem.
  - Formulating an incorrect recurrence relation.
  - Forgetting base cases or handling them improperly.
- **Techniques/Shortcuts:**
  - Define the state: What information is needed to solve a subproblem?
  - Formulate the recurrence relation: How can a subproblem be solved using solutions to smaller subproblems?
  - Identify base cases.
  - Choose between memoization (recursive with caching) and tabulation (iterative).

Graph Algorithms

Definition: A category of algorithms designed to solve problems on graphs, which are mathematical structures used to model pairwise relations between objects. This includes traversal, shortest path, minimum spanning tree, and connectivity problems.

- **Properties/Identities:**
  - Common representations: Adjacency matrix, adjacency list.
  - Key problems: Shortest Path, MST, Graph Traversal, Connectivity.
- **Pitfalls/Tricks:**
  - Not considering graph type (directed/undirected, weighted/unweighted, cyclic/acyclic) when choosing an algorithm.
  - Memory limitations for dense graphs with adjacency matrices.
- **Techniques/Shortcuts:**

    - Always consider the graph representation that best suits the problem and its constraints.
    - Practice drawing graphs and tracing algorithms.

Graph Search

Definition: Algorithms for systematically visiting all vertices and edges in a graph. The two primary methods are Breadth-First Search (BFS) and Depth-First Search (DFS).

- **Properties/Identities:**
    - BFS: Level-by-level, uses queue, finds shortest path in unweighted graphs.
    - DFS: Explores deeply, uses stack/recursion, useful for cycle detection, topological sort, SCC.
- **Pitfalls/Tricks:**
    - Forgetting to use a 'visited' array/set to prevent infinite loops in cyclic graphs.
    - Choosing the wrong search algorithm for the problem (e.g., DFS for unweighted shortest path).
- **Techniques/Shortcuts:**
    - BFS is good for finding "closest" elements.
    - DFS is good for exploring "all paths" or "connectivity".

Greedy Algorithms

Definition: An algorithmic paradigm that makes the locally optimal choice at each stage with the hope of finding a global optimum. It doesn't always guarantee the globally optimal solution but works for specific problems.

- **Properties/Identities:**
    - **Greedy Choice Property:** A globally optimal solution can be reached by making a locally optimal (greedy) choice.
    - **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems.
- **Pitfalls/Tricks:**
    - Applying greedy approach to problems where it doesn't yield an optimal solution (e.g., general shortest path with negative weights, 0/1 Knapsack).
    - Not proving the greedy choice property and optimal substructure.
- **Techniques/Shortcuts:**
    - Common examples: Dijkstra's, Prim's, Kruskal's, Huffman Coding.
    - Try to prove correctness by contradiction or exchange argument.

Hashing

Definition: A technique used to map data of arbitrary size to fixed-size values (hash codes), typically used for efficient data retrieval in hash tables. Involves a hash function and collision resolution strategies.

- **Formulas/Theorems:**
    - **Hash Function:** $h(k) = k \pmod M$ (division method), $h(k) = \lfloor M(kA \pmod 1) \rfloor$ (multiplication method).
    - **Load Factor:** $\alpha = N/M$, where N is number of items, M is table size.
- **Properties/Identities:**
    - **Collision:** When two different keys map to the same hash value.
    - **Collision Resolution:** Open addressing (linear probing, quadratic probing, double hashing) or Separate chaining.
    - **Average Time Complexity (Search, Insert, Delete):** $O(1)$ (with good hash function and low load factor).
    - **Worst-case Time Complexity:** $O(N)$ (due to collisions).
- **Pitfalls/Tricks:**
    - Choosing a poor hash function that leads to many collisions (e.g., for string keys, summing ASCII values).
    - Ignoring the impact of load factor on performance.
- **Techniques/Shortcuts:**
    - For division method, choose M as a prime number not too close to a power of 2 or 10.
    - Understand the trade-offs between different collision resolution techniques.

Heap Sort

Definition: A comparison-based sorting algorithm that uses a binary heap data structure. It's an in-place algorithm that first builds a max-heap from the input data and then repeatedly extracts the maximum element and rebuilds the heap.

- **Formulas/Theorems:**

    - **Time Complexity:** $O(N \log N)$ for all cases (best, average, worst).
    - **Space Complexity:** $O(1)$ (in-place).
- **Properties/Identities:**
    - Not a stable sorting algorithm.
    - Uses a max-heap.
    - Build-heap step takes $O(N)$ time.
    - N extractions take $N \cdot O(\log N)$ time.
- **Pitfalls/Tricks:**
    - Confusing the heap property (min-heap vs. max-heap) with the sorting order.
    - Incorrectly implementing the `heapify` procedure.
- **Techniques/Shortcuts:**
    - Remember the two phases: build heap, then extract max and heapify.
    - The largest element is always at the root after heapify.

Huffman Code

Definition: A particular type of optimal prefix code used for lossless data compression. It's a greedy algorithm that builds a binary tree based on the frequencies of characters, assigning shorter codes to more frequent characters.

- **Properties/Identities:**
    - **Prefix Code:** No code is a prefix of another code, allowing unambiguous decoding.
    - **Optimal:** Produces the minimum possible expected code word length for a given set of character frequencies.
    - Uses a min-priority queue to repeatedly combine the two lowest-frequency nodes.
    - Time Complexity: $O(N \log N)$ where N is the number of unique characters.
- **Pitfalls/Tricks:**
    - Incorrectly building the Huffman tree (e.g., not using a min-priority queue, or incorrect combination logic).
    - Forgetting that it's a greedy algorithm.
- **Techniques/Shortcuts:**
    - Always combine the two nodes with the smallest frequencies.
    - The path from the root to a leaf defines the code word (e.g., left=0, right=1).

Identify Function

Definition: In a general mathematical or programming context, an identity function (often denoted id or I) is a function that always returns the same value that was used as its argument. $f(x) = x$. In algorithms, it might refer to a function used to uniquely identify elements or properties.

- **Properties/Identities:**
    - For any input x, $f(x) = x$.
    - It's the neutral element for function composition: $(f \circ id)(x) = f(x)$ and $(id \circ f)(x) = f(x)$.
- **Pitfalls/Tricks:**
    - This term is very generic; in an algorithmic context, it's usually implied or part of a larger concept (e.g., an identity hash function, or an identity matrix).

Insertion Sort

Definition: A simple sorting algorithm that builds the final sorted array (or list) one item at a time. It iterates through the input elements and removes one element at a time, finds the place where it belongs within the already sorted part, and inserts it there.

- **Formulas/Theorems:**
    - **Worst-case Time Complexity:** $O(N^2)$ (e.g., reverse sorted array).
    - **Average-case Time Complexity:** $O(N^2)$.
    - **Best-case Time Complexity:** $O(N)$ (already sorted array).
    - **Space Complexity:** $O(1)$ (in-place).
    - **Number of Swaps (Worst Case):** $N(N-1)/2$.
    - **Number of Comparisons (Worst Case):** $N(N-1)/2$.
- **Properties/Identities:**
    - Stable sorting algorithm.
    - Adaptive (efficient for nearly sorted arrays).
    - Good for small datasets.

- **Pitfalls/Tricks:**
  - Inefficient for large, unsorted arrays.
  - Understanding the "shift" operation correctly.
- **Techniques/Shortcuts:**
  - Think of it like sorting a hand of playing cards.
  - The first element is considered sorted.

Inversion

Definition: In an array A, an inversion is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. It measures how "unsorted" an array is.

- **Formulas/Theorems:**
  - **Maximum number of inversions in an array of size N:** $N(N-1)/2$ (for a reverse-sorted array).
  - **Minimum number of inversions:** 0 (for a sorted array).
- **Properties/Identities:**
  - The number of inversions can be counted efficiently using a modified Merge Sort algorithm in $O(N \log N)$ time.
- **Pitfalls/Tricks:**
  - Confusing inversions with just any pair of elements that are out of order; the index order $i < j$ is crucial.
- **Techniques/Shortcuts:**
  - To count inversions, adapt Merge Sort: when merging two sorted halves, if an element from the right half is taken before an element from the left half, it means all remaining elements in the left half form inversions with the taken element.

Linear Probing

Definition: A collision resolution technique in open addressing hashing where, upon a collision, the algorithm searches for the next available slot sequentially in the hash table (e.g., at $h(k)+1, h(k)+2, \ldots$).

- **Formulas/Theorems:**
  - **Probe Sequence:** $h(k, i) = (h(k) + i) \pmod M$, where M is table size and i is probe number (0, 1, 2, ...).
- **Properties/Identities:**
  - Simple to implement.
  - Suffers from primary clustering: long runs of occupied slots build up, increasing average search time.
- **Pitfalls/Tricks:**
  - Primary clustering significantly degrades performance, especially at high load factors.
  - Deletion is tricky: simply removing an element can break search chains. Often requires "lazy deletion" or re-hashing.
- **Techniques/Shortcuts:**
  - Understand that it's the simplest but often least efficient open addressing method.

Matrix Chain Ordering (Matrix Chain Multiplication)

Definition: A dynamic programming problem that seeks to find the most efficient way to multiply a sequence of matrices. The problem is not about performing the multiplications, but deciding the optimal parenthesization (order) to minimize the total number of scalar multiplications.

- **Formulas/Theorems:**
  - **Recurrence Relation:** Let $M[i, j]$ be the minimum number of scalar multiplications needed to compute the product $A_i A_{i+1} \ldots A_j$.

    $$M[i, j] = \min_{i \le k < j} (M[i, k] + M[k+1, j] + p_{i-1} p_k p_j)$$

    where A_i has dimensions $p_{i-1} \times p_i$.
  - **Base Case:** $M[i, i] = 0$ (single matrix requires no multiplications).
  - **Time Complexity:** $O(N^3)$ for N matrices.
  - **Space Complexity:** $O(N^2)$ for the DP table.
- **Properties/Identities:**
  - Exhibits optimal substructure and overlapping subproblems.
  - The order of multiplication matters for efficiency, not for the result.
- **Pitfalls/Tricks:**

    - Incorrectly setting up the dimensions array p. If there are N matrices $A_1, \ldots, A_N$, and A_i is $p_{i-1} \times p_i$, then the array p has $N+1$ elements.
    - Off-by-one errors in the recurrence relation indices.
- **Techniques/Shortcuts:**
    - Fill the DP table diagonally, starting with chain length 2, then 3, and so on.

Maximum Minimum

Definition: The problem of finding both the maximum and minimum elements in a given array or list. This can be done naively by iterating twice or more efficiently using a divide and conquer approach.

- **Formulas/Theorems:**
    - **Naive Approach (2N-2 comparisons):** Iterate once for max, once for min.
    - **Divide and Conquer Approach (approx. 3N/2 comparisons):**
        - If array size is 1, max=min=element.
        - If array size is 2, compare once.
        - Recursively find max/min in two halves, then compare the two maxes and two mins.
- **Properties/Identities:**
    - The divide and conquer approach is more efficient in terms of comparisons.
- **Pitfalls/Tricks:**
    - Forgetting to handle base cases (array of size 1 or 2) correctly in recursive solutions.
- **Techniques/Shortcuts:**
    - Pairwise comparison: process elements in pairs, comparing them. Then compare the smaller of the pair with the current min, and the larger with the current max. This also achieves approx. 3N/2 comparisons.

Merge Sort

Definition: A divide and conquer sorting algorithm that divides an unsorted list into N sublists, each containing one element (a list of one element is considered sorted), then repeatedly merges sublists to produce new sorted sublists until there is only one sorted list remaining.

- **Formulas/Theorems:**
    - **Recurrence Relation:** $T(N) = 2T(N/2) + O(N)$ (for dividing and merging).
    - **Time Complexity:** $O(N \log N)$ for all cases (best, average, worst).
    - **Space Complexity:** $O(N)$ (due to auxiliary array for merging).
- **Properties/Identities:**
    - Stable sorting algorithm.
    - Not an in-place algorithm (requires extra space).
    - Well-suited for external sorting.
- **Pitfalls/Tricks:**
    - Incorrectly implementing the merging step, which is crucial for correctness and efficiency.
    - Forgetting to copy remaining elements from one half if the other half is exhausted during merge.
- **Techniques/Shortcuts:**
    - The merge step is the core: combine two sorted arrays into one sorted array.
    - Can be used to count inversions.

Merging

Definition: The process of combining two or more sorted lists (or arrays) into a single sorted list. This is a fundamental operation in algorithms like Merge Sort.

- **Formulas/Theorems:**
    - **Time Complexity:** $O(N+M)$ to merge two sorted lists of sizes N and M.
    - **Space Complexity:** $O(N+M)$ for the new merged list.
- **Properties/Identities:**
    - Requires input lists to be sorted.
    - The output list is also sorted.
- **Pitfalls/Tricks:**
    - Off-by-one errors when handling array boundaries and indices.
    - Not correctly handling the case where one list is exhausted before the other.
- **Techniques/Shortcuts:**
    - Use two pointers, one for each input list, and a third pointer for the merged list.

Minimum Spanning Tree (MST)

Definition: For a connected, undirected, weighted graph, an MST is a subgraph that is a tree, connects all the vertices together, and has the minimum possible total edge weight. Algorithms include Prim's and Kruskal's.

- **Formulas/Theorems:**
  - **Cut Property:** For any cut (partition of vertices into two sets), if an edge crosses the cut and has strictly less weight than any other edge crossing the cut, then this edge must be in every MST.
  - **Cycle Property:** If an edge is the heaviest edge in any cycle of a graph, then it cannot be part of an MST.
  - An MST of a graph with $|V|$ vertices always has $|V| - 1$ edges.
- **Properties/Identities:**
  - Both Prim's and Kruskal's are greedy algorithms.
  - Prim's: Grows the MST from a starting vertex.
  - Kruskal's: Adds edges in increasing order of weight, avoiding cycles.
- **Pitfalls/Tricks:**
  - Applying MST algorithms to disconnected graphs (they will find an MST for each connected component, forming a Minimum Spanning Forest).
  - Confusing MST with shortest path problems.
- **Techniques/Shortcuts:**
  - Prim's uses a min-priority queue.
  - Kruskal's uses a Disjoint Set Union (DSU) data structure.

Number of Swap

Definition: A metric used to evaluate the efficiency of certain sorting algorithms, particularly comparison sorts. It counts how many times elements are exchanged during the sorting process.

- **Formulas/Theorems:**
  - **Bubble Sort (Worst Case):** $N(N-1)/2$.
  - **Selection Sort (Worst Case):** $N-1$.
  - **Insertion Sort (Worst Case):** $N(N-1)/2$.
  - **Quick Sort (Worst Case):** $O(N^2)$ swaps.
  - **Heap Sort (Worst Case):** $O(N \log N)$ swaps.
- **Properties/Identities:**
  - A lower number of swaps generally indicates better performance, especially when swap operations are costly.
  - Selection sort performs the minimum number of swaps among simple sorts.
- **Pitfalls/Tricks:**
  - Confusing swaps with comparisons. Both are important metrics but measure different aspects.
- **Techniques/Shortcuts:**
  - Memorize the worst-case swap counts for common sorting algorithms.

Prims Algorithm

Definition: A greedy algorithm that finds a Minimum Spanning Tree (MST) for a weighted undirected graph. It starts from an arbitrary vertex and grows the MST by adding the cheapest edge that connects a vertex in the MST to a vertex outside the MST.

- **Formulas/Theorems:**
  - **Time Complexity:**
    - With adjacency matrix (simple array scan): $O(|V|^2)$.
    - With adjacency list and binary min-priority queue: $O(|E| \log |V|)$ or $O(|E| + |V| \log |V|)$.
  - **Space Complexity:** $O(|V| + |E|)$ for graph, $O(|V|)$ for distances/parent array.
- **Properties/Identities:**
  - Greedy algorithm.
  - Builds the MST by adding vertices one by one.
  - Similar structure to Dijkstra's algorithm.
- **Pitfalls/Tricks:**
  - Forgetting to update edge weights in the priority queue when a shorter path to an unvisited vertex is found.
  - Applying to directed graphs (MST is for undirected graphs).
- **Techniques/Shortcuts:**
  - Maintain a set of vertices already in the MST and a min-priority queue of edges connecting to vertices outside

the MST.

Quick Sort

Definition: A highly efficient, in-place, divide and conquer sorting algorithm. It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

- **Formulas/Theorems:**
  - **Worst-case Time Complexity:** $O(N^2)$ (e.g., already sorted array with first/last element as pivot).
  - **Average-case Time Complexity:** $O(N \log N)$.
  - **Best-case Time Complexity:** $O(N \log N)$.
  - **Space Complexity:** $O(\log N)$ (average, for recursion stack) or $O(N)$ (worst-case, for recursion stack).
  - **Recurrence Relation (Average Case):** $T(N) = T(k) + T(N-k-1) + O(N)$, where k is the size of one partition. For balanced partitions, $T(N) = 2T(N/2) + O(N)$.
- **Properties/Identities:**
  - Not a stable sorting algorithm.
  - In-place sorting algorithm.
  - Performance heavily depends on pivot selection.
- **Pitfalls/Tricks:**
  - Poor pivot selection can lead to $O(N^2)$ performance.
  - Incorrect partitioning logic can lead to infinite recursion or incorrect sorting.
- **Techniques/Shortcuts:**
  - Randomized pivot selection helps achieve average-case performance reliably.
  - Hoare's partition scheme is often more efficient than Lomuto's.

Recurrence Relation

Definition: An equation that recursively defines a sequence or function, where each term or value is given as a function of preceding terms. Used to describe the time complexity of recursive algorithms.

- **Formulas/Theorems:**
  - **Master Theorem:** For recurrences of the form $T(N) = aT(N/b) + f(N)$ where $a \geq 1, b > 1$.
    1. If $f(N) = O(N^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(N) = \Theta(N^{\log_b a})$.
    2. If $f(N) = \Theta(N^{\log_b a})$, then $T(N) = \Theta(N^{\log_b a} \log N)$.
    3. If $f(N) = \Omega(N^{\log_b a + \epsilon})$ for some $\epsilon > 0$, AND $af(N/b) \leq cf(N)$ for some $c < 1$ and large N, then $T(N) = \Theta(f(N))$.
- **Properties/Identities:**
  - Describes the growth rate of recursive algorithms.
  - Methods to solve: Substitution, Recursion Tree, Master Theorem.
- **Pitfalls/Tricks:**
  - Incorrectly applying the Master Theorem (e.g., when $f(N)$ doesn't fit any case or the regularity condition is not met).
  - Algebraic errors in substitution or recursion tree methods.
- **Techniques/Shortcuts:**
  - Memorize the three cases of the Master Theorem.
  - For simple recurrences, try expanding a few terms to find a pattern.

Recursion

Definition: A programming technique where a function calls itself directly or indirectly to solve a problem. It involves a base case (stopping condition) and a recursive step (reducing the problem to a smaller instance of itself).

- **Properties/Identities:**
  - **Base Case:** A condition that terminates the recursion.
  - **Recursive Step:** The function calls itself with a modified (smaller) input.
  - Often leads to elegant and concise code for problems with recursive structure.
- **Pitfalls/Tricks:**
  - Missing or incorrect base case leading to infinite recursion (stack overflow).
  - High space complexity due to recursion stack.
  - Performance overhead compared to iterative solutions due to function call stack management.

- **Techniques/Shortcuts:**
  - Always identify the base case first.
  - Ensure that each recursive call moves closer to the base case.
  - For some problems, recursion can be converted to iteration using a stack.

Searching

Definition: The process of finding a specific item (or items) within a collection of items. Common search algorithms include Linear Search and Binary Search.

- **Formulas/Theorems:**
  - **Linear Search (Worst Case):** $O(N)$ comparisons.
  - **Binary Search (Worst Case):** $O(\log N)$ comparisons.
- **Properties/Identities:**
  - Linear Search: Works on unsorted or sorted data.
  - Binary Search: Requires sorted data.
- **Pitfalls/Tricks:**
  - Using Linear Search on a large sorted array when Binary Search is applicable.
  - Errors in Binary Search boundary conditions.
- **Techniques/Shortcuts:**
  - Always check if data is sorted before choosing a search algorithm.

Selection Sort

Definition: A simple sorting algorithm that repeatedly finds the minimum element from the unsorted part of the list and swaps it with the element at the current position. It maintains two subarrays: sorted and unsorted.

- **Formulas/Theorems:**
  - **Worst-case Time Complexity:** $O(N^2)$.
  - **Average-case Time Complexity:** $O(N^2)$.
  - **Best-case Time Complexity:** $O(N^2)$.
  - **Space Complexity:** $O(1)$ (in-place).
  - **Number of Swaps (All Cases):** $N-1$ (minimum possible swaps for any comparison sort).
  - **Number of Comparisons (All Cases):** $N(N-1)/2$.
- **Properties/Identities:**
  - Not a stable sorting algorithm.
  - Performs the minimum number of swaps among simple sorts.
- **Pitfalls/Tricks:**
  - Its performance is consistently $O(N^2)$ regardless of input order, unlike Bubble Sort or Insertion Sort.
- **Techniques/Shortcuts:**
  - The key idea is to find the minimum and place it, then find the next minimum and place it, and so on.

Shortest Path

Definition: The problem of finding a path between two vertices (or from a source to all other vertices) in a graph such that the sum of the weights of its constituent edges is minimized. Algorithms include BFS (unweighted), Dijkstra's (non-negative weights), and Bellman-Ford (negative weights).

- **Properties/Identities:**
  - **Unweighted Graphs:** BFS finds shortest path in terms of number of edges.
  - **Non-negative Weighted Graphs:** Dijkstra's algorithm.
  - **Negative Weighted Graphs (no negative cycles):** Bellman-Ford algorithm.
  - **All-Pairs Shortest Path:** Floyd-Warshall algorithm.
- **Pitfalls/Tricks:**
  - Using the wrong algorithm for the given graph properties (e.g., Dijkstra's on negative weights).
  - Not handling disconnected components or unreachable vertices.
- **Techniques/Shortcuts:**
  - Always check for negative edge weights and negative cycles.
  - Understand the relaxation principle.

Sorting

Definition: The process of arranging elements of a list or array in a specific order (e.g., numerical, alphabetical, ascending, descending). It's a fundamental operation in computer science.

- **Formulas/Theorems:**
  - **Comparison Sort Lower Bound:** Any comparison-based sorting algorithm requires $\Omega(N \log N)$ comparisons in the worst case.
- **Properties/Identities:**
  - **Stable Sort:** Preserves the relative order of equal elements.
  - **In-place Sort:** Requires $O(1)$ or $O(\log N)$ auxiliary space.
  - **Adaptive Sort:** Performance improves for partially sorted input.
- **Pitfalls/Tricks:**
  - Confusing different sorting algorithm properties (stability, in-place, worst-case vs. average-case).
  - Choosing an inefficient sort for specific data characteristics.
- **Techniques/Shortcuts:**
  - Know the time and space complexities, stability, and in-place nature of common sorts.
  - For GATE, often questions involve comparing two sorts or analyzing a modified sort.

Space Complexity

Definition: A measure of the amount of memory an algorithm needs to run to completion. It includes the space required for input, output, and temporary variables, often expressed using asymptotic notation.

- **Properties/Identities:**
  - **Auxiliary Space:** The extra space used by the algorithm beyond the input data.
  - **In-place Algorithm:** An algorithm that transforms input using only a small, constant amount of auxiliary space, typically $O(1)$ or $O(\log N)$ for recursion stack.
- **Pitfalls/Tricks:**
  - Forgetting to account for the recursion stack space in recursive algorithms.
  - Confusing total space with auxiliary space.
- **Techniques/Shortcuts:**
  - Identify data structures created by the algorithm (arrays, queues, stacks, hash tables).
  - For recursive calls, the maximum depth of the recursion stack contributes to space complexity.

Strongly Connected Components (SCC)

Definition: In a directed graph, a strongly connected component (SCC) is a maximal subgraph such that for every pair of vertices u and v in the subgraph, there is a path from u to v and a path from v to u. Algorithms include Kosaraju's and Tarjan's.

- **Properties/Identities:**
  - SCCs partition the vertices of a directed graph.
  - If we contract each SCC into a single vertex, the resulting graph is a Directed Acyclic Graph (DAG).
  - Kosaraju's Algorithm: Two DFS passes (one on original graph, one on transpose graph).
  - Tarjan's Algorithm: One DFS pass, uses discovery times and low-link values.
- **Formulas/Theorems:**
  - **Time Complexity (Kosaraju's, Tarjan's):** $O(|V| + |E|)$.
- **Pitfalls/Tricks:**
  - Confusing SCCs with connected components in undirected graphs.
  - Incorrectly performing DFS on the transpose graph for Kosaraju's.
- **Techniques/Shortcuts:**
  - Kosaraju's: DFS on G to get finishing times, then DFS on G transpose in decreasing order of finishing times.
  - Tarjan's: Uses a stack and `disc` (discovery time) and `low` (lowest discovery time reachable) arrays.

Time Complexity

Definition: A measure of the amount of time an algorithm takes to run as a function of the length of its input. It's typically expressed using asymptotic notations (Big-O, Omega, Theta).

- **Properties/Identities:**
  - Focuses on the growth rate of operations as input size N increases.
  - Usually refers to worst-case time complexity, but average-case and best-case are also important.
  - Independent of machine specifics (CPU speed, memory access time).

- **Pitfalls/Tricks:**
  - Confusing constant factors with asymptotic growth.
  - Incorrectly analyzing loops or recursive calls.
  - Assuming best-case performance for general analysis.
- **Techniques/Shortcuts:**
  - Count dominant operations (comparisons, assignments, arithmetic operations).
  - For nested loops, multiply loop counts.
  - For recursive functions, use recurrence relations and Master Theorem.

Topological Sort

Definition: A linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge (u, v), vertex u comes before vertex v in the ordering. Not unique for all DAGs.

- **Properties/Identities:**
  - Only possible for DAGs.
  - Can be implemented using DFS or Kahn's algorithm (using in-degrees and a queue).
  - **Time Complexity:** $O(|V| + |E|)$.
- **Pitfalls/Tricks:**
  - Attempting to topologically sort a graph with cycles (it's impossible).
  - Incorrectly handling multiple possible topological sorts.
- **Techniques/Shortcuts:**
  - **DFS-based:** Perform DFS, then reverse the order of finishing times.
  - **Kahn's Algorithm:** Find all nodes with in-degree 0, add to queue. While queue not empty, dequeue, add to sorted list, decrement in-degree of neighbors. If neighbor's in-degree becomes 0, enqueue.

Tree Traversal

Definition: The process of visiting each node in a tree data structure exactly once. Common methods include Inorder, Preorder, Postorder (for binary trees), and Level-order traversal.

- **Properties/Identities:**
  - **Inorder (Left-Root-Right):** For BSTs, gives sorted elements.
  - **Preorder (Root-Left-Right):** Useful for creating a copy of the tree.
  - **Postorder (Left-Right-Root):** Useful for deleting a tree.
  - **Level-order (BFS-like):** Visits nodes level by level, uses a queue.
  - **Time Complexity:** $O(N)$ for a tree with N nodes.
  - **Space Complexity:** $O(H)$ for DFS-based (recursion stack, where H is height), $O(W)$ for BFS-based (queue, where W is max width).
- **Pitfalls/Tricks:**
  - Confusing the order of visits for different traversal types.
  - Incorrectly handling null nodes in recursive traversals.
- **Techniques/Shortcuts:**
  - Practice drawing the traversal paths on sample trees.
  - Remember the "Root" position in the name (Pre-Root-Order, In-Root-Order, Post-Root-Order).

Uniform Hashing

Definition: An idealized theoretical model for hashing where each key is equally likely to hash to any slot in the hash table, independent of where other keys hash. It implies a perfectly random distribution of keys.

- **Properties/Identities:**
  - Assumed for theoretical analysis of hashing algorithms (e.g., average case performance of open addressing).
  - Difficult to achieve in practice with real-world data.
  - Minimizes collisions and maximizes efficiency.
- **Pitfalls/Tricks:**
  - Assuming practical hash functions achieve uniform hashing, which is rarely true.
  - Not understanding that it's a theoretical ideal, not a practical hash function.
- **Techniques/Shortcuts:**
  - When analyzing hashing, remember that uniform hashing is the best-case scenario for collision distribution.

Quick Formula Reference

- **Asymptotic Notations:**
  - Big-O: $f(n) = O(g(n)) \implies \exists c, n_0 > 0 \text{ s.t. } 0 \le f(n) \le c \cdot g(n) \text{ for } n \ge n_0$.
  - Omega: $f(n) = \Omega(g(n)) \implies \exists c, n_0 > 0 \text{ s.t. } 0 \le c \cdot g(n) \le f(n) \text{ for } n \ge n_0$.
  - Theta: $f(n) = \Theta(g(n)) \implies \exists c_1, c_2, n_0 > 0 \text{ s.t. } 0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n) \text{ for } n \ge n_0$.
- **Binary Heap:**
  - Parent of i (0-indexed): $\lfloor (i-1)/2 \rfloor$.
  - Left child of i: $2i + 1$.
  - Right child of i: $2i + 2$.
  - Height: $\lfloor \log_2 N \rfloor$.
- **Binary Tree:**
  - Max nodes at level i: 2^i.
  - Max nodes in height h: $2^{h+1} - 1$.
  - Min height with N nodes: $\lceil \log_2(N+1) \rceil - 1$.
- **Hashing (Open Addressing):**
  - Linear Probing: $h(k, i) = (h(k) + i) \pmod M$.
  - Double Hashing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod M$.
  - Load Factor: $\alpha = N/M$.
- **Matrix Chain Ordering:**

$$M[i, j] = \min_{i \le k < j} (M[i, k] + M[k+1, j] + p_{i-1} p_k p_j)$$

  Base Case: $M[i, i] = 0$.
- **Recurrence Relation (Master Theorem):** $T(N) = aT(N/b) + f(N)$.
  1. If $f(N) = O(N^{\log_b a - \epsilon})$, $T(N) = \Theta(N^{\log_b a})$.
  2. If $f(N) = \Theta(N^{\log_b a})$, $T(N) = \Theta(N^{\log_b a} \log N)$.
  3. If $f(N) = \Omega(N^{\log_b a + \epsilon})$ and $af(N/b) \le cf(N)$, $T(N) = \Theta(f(N))$.
- **Sorting Algorithm Complexities:**

Algorithm	Time (Worst)	Time (Avg)	Space	Stable	In-place
Bubble Sort	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Yes
Insertion Sort	$O(N^2)$	$O(N^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(N^2)$	$O(N^2)$	$O(1)$	No	Yes
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N)$	Yes	No
Quick Sort	$O(N^2)$	$O(N \log N)$	$O(\log N)$	No	Yes
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(1)$	No	Yes

- **Graph Algorithm Complexities (Adjacency List):**
  - BFS: $O(|V| + |E|)$
  - DFS: $O(|V| + |E|)$
  - Dijkstra's (Binary Heap): $O(|E| \log |V|)$
  - Bellman-Ford: $O(|V| \cdot |E|)$
  - Prim's (Binary Heap): $O(|E| \log |V|)$
  - Kruskal's (DSU): $O(|E| \log |E|)$ or $O(|E| \log |V|)$
  - Topological Sort: $O(|V| + |E|)$
  - SCC (Kosaraju's/Tarjan's): $O(|V| + |E|)$

Important Tips for GATE

1. **Master Asymptotic Notations:** A significant portion of algorithm questions revolves around time and space complexity. Understand the definitions of $O, \Omega, \Theta, o, \omega$ thoroughly, and be adept at applying them to various code snippets and recurrence relations, especially using the Master Theorem.
2. **Understand Algorithm Design Paradigms:** Don't just memorize algorithms; understand *why* they work and which paradigm they belong to (Divide and Conquer, Greedy, Dynamic Programming). This helps in identifying the correct approach for new problems.
3. **Practice Recurrence Relations:** Solving recurrence relations is a frequently tested skill. Be comfortable with the substitution method, recursion tree method, and especially the Master Theorem. Pay attention to base cases and boundary conditions.
4. **Graph Algorithms are Crucial:** BFS, DFS, Dijkstra's, Bellman-Ford, Prim's, Kruskal's, Topological Sort, and SCCs are high-yield topics. Know their complexities, applications, and limitations (e.g., negative weights for

1.1 Algorithm Design (8)

✍ **Practice Test:** Test 1 (9Q)

1.1.1 Algorithm Design: GATE CSE 1992 | Question: 8

Let T be a Depth First Tree of a undirected graph G. An array P indexed by the vertices of G is given. $P[V]$ is the parent of vertex V, in T. Parent of the root is the root itself.

Give a method for finding and printing the cycle formed if the edge (u, v) of G not in T (i.e., $e \in G - T$) is now added to T.

Time taken by your method must be proportional to the length of the cycle.

Describe the algorithm in a PASCAL (C) – like language. Assume that the variables have been suitably declared.

gate1992 algorithms descriptive algorithm-design

Answer key

1.1.2 Algorithm Design: GATE CSE 1994 | Question: 7

An array A contains n integers in locations $A[0], A[1], \ldots A[n-1]$. It is required to shift the elements of the array cyclically to the left by K places, where $1 \leq K \leq n-1$. An incomplete algorithm for doing this in linear time, without using another array is given below. Complete the algorithm by filling in the blanks. Assume all variables are suitably declared.

```
min:=n;
i=0;
while _____ do
begin
   temp:=A[i];
   j:=i;
   while ____ do
   begin
      A[j]:=____;
      j:=(j+K) mod n;
     if j<min then
      min:=j;
   end;
   A[(n+i-K)mod n]:=____;
   i:=_____;
end;
```

gate1994 algorithms normal algorithm-design fill-in-the-blanks

Answer key

1.1.3 Algorithm Design: GATE CSE 2006 | Question: 17

An element in an array X is called a leader if it is greater than all elements to the right of it in X. The best algorithm to find all leaders in an array

A. solves it in linear time using a left to right pass of the array
B. solves it in linear time using a right to left pass of the array
C. solves it using divide and conquer in time $\Theta(n \log n)$
D. solves it in time $\Theta(n^2)$

gatecse-2006 algorithms normal algorithm-design

Answer key

1.1.4 Algorithm Design: GATE CSE 2006 | Question: 54

Given two arrays of numbers $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$ where each number is 0 or 1, the fastest algorithm to find the largest span (i, j) such that $a_i + a_{i+1} + \cdots + a_j = b_i + b_{i+1} + \cdots + b_j$ or report that there is not such span,

A. Takes $O(3^n)$ and $\Omega(2^n)$ time if hashing is permitted
B. Takes $O(n^3)$ and $\Omega(n^{2.5})$ time in the key comparison mode
C. Takes $\Theta(n)$ time and space
D. Takes $O(\sqrt{n})$ time only if the sum of the $2n$ elements is an even number

gatecse-2006 algorithms normal algorithm-design time-complexity

Answer key

1.1.5 Algorithm Design: GATE CSE 2014 | Set 1 | Question: 37

There are 5 bags labeled 1 to 5. All the coins in a given bag have the same weight. Some bags have coins of weight 10 gm, others have coins of weight 11 gm. I pick $1, 2, 4, 8, 16$ coins respectively from bags 1 to 5 Their total weight comes out to 323 gm. Then the product of the labels of the bags having 11 gm coins is ___.

gatecse-2014-set1 algorithms numerical-answers normal algorithm-design

Answer key

1.1.6 Algorithm Design: GATE CSE 2019 | Question: 25

Consider a sequence of 14 elements: $A = [-5, -10, 6, 3, -1, -2, 13, 4, -9, -1, 4, 12, -3, 0]$. The sequence sum $S(i, j) = \Sigma_{k=i}^{j} A[k]$. Determine the maximum of $S(i, j)$, where $0 \leq i \leq j < 14$. (Divide and conquer approach may be used.)

Answer: _____

gatecse-2019 numerical-answers algorithms algorithm-design one-mark

Answer key

1.1.7 Algorithm Design: GATE CSE 2021 | Set 1 | Question: 40

Define R_n to be the maximum amount earned by cutting a rod of length n meters into one or more pieces of integer length and selling them. For $i > 0$, let $p[i]$ denote the selling price of a rod whose length is i meters. Consider the array of prices:

$$\mathrm{p}[1] = 1, \mathrm{p}[2] = 5, \mathrm{p}[3] = 8, \mathrm{p}[4] = 9, \mathrm{p}[5] = 10, \mathrm{p}[6] = 17, \mathrm{p}[7] = 18$$

Which of the following statements is/are correct about R_7?

A. $R_7 = 18$
B. $R_7 = 19$
C. R_7 is achieved by three different solutions
D. R_7 cannot be achieved by a solution consisting of three pieces

gatecse-2021-set1 multiple-selects algorithms algorithm-design two-marks

Answer key

1.1.8 Algorithm Design: GATE CSE 2024 | Set 2 | Question: 32

Consider an array X that contains n positive integers. A subarray of X is defined to be a sequence of array locations with consecutive indices.

The C code snippet given below has been written to compute the length of the longest subarray of X that contains at most two distinct integers. The code has two missing expressions labelled (P) and (Q).

```
int first=0, second=0, len1=0, len2=0, maxlen=0;
for (int i=0; i < n; i++) {
   if (X[i] == first) {
      len2++; len1++;
   } else if (X[i] == second) {
      len2++;
      len1 =      (P)      ;

      second = first;
```

```
    } else {
        len2 =      (Q)      ;

        len1 = 1; second = first;
    }
    if (len2 > maxlen) {
        maxlen = len2;
    }
    first = X[i];
}
```

Which one of the following options gives the CORRECT missing expressions?

(**Hint:** At the end of the i-th iteration, the value of $len1$ is the length of the longest subarray ending with $\mathrm{X}[i]$ that contains all equal values, and $len2$ is the length of the longest subarray ending with $\mathrm{X}[i]$ that contains at most two distinct values.)

A. $(\mathrm{P})\ \mathrm{len1}+1 \quad (\mathrm{Q})\ \mathrm{len2}+1$
B. $(\mathrm{P})\ 1 \quad (\mathrm{Q})\ \mathrm{len1}+1$
C. $(\mathrm{P})\ 1 \quad (\mathrm{Q})\ \mathrm{len2}+1$
D. $(\mathrm{P})\ \mathrm{len2}+1 \quad (\mathrm{Q})\ \mathrm{len1}+1$

gatecse-2024-set2 algorithms algorithm-design two-marks

Answer key

1.2 Algorithm Design Techniques (9)

1.2.1 Algorithm Design Techniques: GATE CSE 1990 | Question: 12b

Consider the following problem. Given n positive integers $a_1, a_2 \ldots a_n$, it is required to partition them in to two parts A and B such that, $\left|\sum_{i\in A} a_i - \sum_{i\in B} a_i\right|$ is minimised

Consider a greedy algorithm for solving this problem. The numbers are ordered so that $a_1 \geq a_2 \geq \ldots a_n$, and at i^{th} step, a_i is placed in that part whose sum in smaller at that step. Give an example with $n = 5$ for which the solution produced by the greedy algorithm is not optimal.

gate1990 descriptive algorithms algorithm-design-techniques

Answer key

1.2.2 Algorithm Design Techniques: GATE CSE 1990 | Question: 2-vii

Match the pairs in the following questions:

(a)	Strassen's matrix multiplication algorithm	(p)	Greedy method
(b)	Kruskal's minimum spanning tree algorithm	(q)	Dynamic programming
(c)	Biconnected components algorithm	(r)	Divide and Conquer
(d)	Floyd's shortest path algorithm	(s)	Depth-first search

gate1990 match-the-following algorithms algorithm-design-techniques easy

Answer key

1.2.3 Algorithm Design Techniques: GATE CSE 1994 | Question: 1.19, ISRO2016-31

Algorithm design technique used in quicksort algorithm is?

A. Dynamic programming
B. Backtracking
C. Divide and conquer
D. Greedy method

gate1994 algorithms algorithm-design-techniques quick-sort easy isro2016

Answer key

1.2.4 Algorithm Design Techniques: GATE CSE 1995 | Question: 1.5

Merge sort uses:

A. Divide and conquer strategy
B. Backtracking approach
C. Heuristic search
D. Greedy approach

gate1995 algorithms sorting easy algorithm-design-techniques merge-sort

Answer key

1.2.5 Algorithm Design Techniques: GATE CSE 1997 | Question: 1.5

The correct matching for the following pairs is

A.	All pairs shortest path	1.	Greedy
B.	Quick Sort	2.	Depth-First Search
C.	Minimum weight spanning tree	3.	Dynamic Programming
D.	Connected Components	4.	Divide and Conquer

A. A-2 B-4 C-1 D-3 B. A-3 B-4 C-1 D-2 C. A-3 B-4 C-2 D-1 D. A-4 B-1 C-2 D-3

gate1997 algorithms normal algorithm-design-techniques easy match-the-following

Answer key

1.2.6 Algorithm Design Techniques: GATE CSE 1998 | Question: 1.21, ISRO2008-16

Which one of the following algorithm design techniques is used in finding all pairs of shortest distances in a graph?

A. Dynamic programming
B. Backtracking
C. Greedy
D. Divide and Conquer

gate1998 algorithms algorithm-design-techniques easy isro2008

Answer key

1.2.7 Algorithm Design Techniques: GATE CSE 2015 | Set 1 | Question: 6

Match the following:

P.	Prim's algorithm for minimum spanning tree	i.	Backtracking
Q.	Floyd-Warshall algorithm for all pairs shortest path	ii.	Greedy method
R.	Merge sort	iii.	Dynamic programming
S.	Hamiltonian circuit	iv.	Divide and conquer

A. P-iii, Q-ii, R-iv, S-i
B. P-i, Q-ii, R-iv, S-iii
C. P-ii, Q-iii, R-iv, S-i
D. P-ii, Q-i, R-iii, S-iv

gatecse-2015-set1 algorithms normal match-the-following algorithm-design-techniques

Answer key

1.2.8 Algorithm Design Techniques: GATE CSE 2015 | Set 2 | Question: 36

Given below are some algorithms, and some algorithm design paradigms.

1.	Dijkstra's Shortest Path	i.	Divide and Conquer
2.	Floyd-Warshall algorithm to compute all pair shortest path	ii.	Dynamic Programming
3.	Binary search on a sorted array	iii.	Greedy design
4.	Backtracking search on a graph	iv.	Depth-first search
		v.	Breadth-first search

Match the above algorithms on the left to the corresponding design paradigm they follow.

A. 1-i, 2-iii, 3-i, 4-v
B. 1-iii, 2-iii, 3-i, 4-v
C. 1-iii, 2-ii, 3-i, 4-iv
D. 1-iii, 2-ii, 3-i, 4-v

gatecse-2015-set2 algorithms easy algorithm-design-techniques match-the-following

Answer key

1.2.9 Algorithm Design Techniques: GATE CSE 2017 | Set 1 | Question: 05

Consider the following table:

	Algorithms		Design Paradigms
(P)	Kruskal	(i)	Divide and Conquer
(Q)	Quicksort	(ii)	Greedy
(R)	Floyd-Warshall	(iii)	Dynamic Programming

Match the algorithms to the design paradigms they are based on.

A. $(P) \leftrightarrow (ii), (Q) \leftrightarrow (iii), (R) \leftrightarrow (i)$
B. $(P) \leftrightarrow (iii), (Q) \leftrightarrow (i), (R) \leftrightarrow (ii)$
C. $(P) \leftrightarrow (ii), (Q) \leftrightarrow (i), (R) \leftrightarrow (iii)$
D. $(P) \leftrightarrow (i), (Q) \leftrightarrow (ii), (R) \leftrightarrow (iii)$

gatecse-2017-set1 algorithms algorithm-design-techniques easy match-the-following

Answer key

1.3 Asymptotic Notations (22)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (4Q)

1.3.1 Asymptotic Notations: GATE CSE 1994 | Question: 1.23

Consider the following two functions:

$$g_1(n) = \begin{cases} n^3 \text{ for } 0 \leq n \leq 10,000 \\ n^2 \text{ for } n > 10,000 \end{cases}$$

$$g_2(n) = \begin{cases} n \text{ for } 0 \leq n \leq 100 \\ n^3 \text{ for } n > 100 \end{cases}$$

Which of the following is true?

A. $g_1(n)$ is $O(g_2(n))$
B. $g_1(n)$ is $O(n^3)$
C. $g_2(n)$ is $O(g_1(n))$
D. $g_2(n)$ is $O(n)$

gate1994 algorithms asymptotic-notations normal multiple-selects

Answer key

1.3.2 Asymptotic Notations: GATE CSE 1996 | Question: 1.11

Which of the following is false?

A. $100n \log n = O(\frac{n \log n}{100})$
B. $\sqrt{\log n} = O(\log \log n)$
C. If $0 < x < y$ then $n^x = O(n^y)$
D. $2^n \neq O(nk)$

gate1996 algorithms asymptotic-notations normal

Answer key

1.3.3 Asymptotic Notations: GATE CSE 1999 | Question: 2.21

If $T_1 = O(1)$, give the correct matching for the following pairs:

(M) $T_n = T_{n-1} + n$	(U) $T_n = O(n)$
(N) $T_n = T_{n/2} + n$	(V) $T_n = O(n \log n)$
(O) $T_n = T_{n/2} + n \log n$	(W) $T_n = O(n^2)$
(P) $T_n = T_{n-1} + \log n$	(X) $T_n = O(\log^2 n)$

A. M-W, N-V, O-U, P-X
B. M-W, N-U, O-X, P-V
C. M-V, N-W, O-X, P-U
D. M-W, N-U, O-V, P-X

gate1999 algorithms recurrence-relation asymptotic-notations normal match-the-following

Answer key

1.3.4 Asymptotic Notations: GATE CSE 2000 | Question: 2.17

Consider the following functions

- $f(n) = 3n^{\sqrt{n}}$
- $g(n) = 2^{\sqrt{n} \log_2 n}$
- $h(n) = n!$

Which of the following is true?

A. $h(n)$ is $O(f(n))$
B. $h(n)$ is $O(g(n))$
C. $g(n)$ is not $O(f(n))$
D. $f(n)$ is $O(g(n))$

gatecse-2000 algorithms asymptotic-notations normal

Answer key

1.3.5 Asymptotic Notations: GATE CSE 2001 | Question: 1.16

Let $f(n) = n^2 \log n$ and $g(n) = n(\log n)^{10}$ be two positive functions of n. Which of the following statements is correct?

A. $f(n) = O(g(n))$ and $g(n) \neq O(f(n))$
B. $g(n) = O(f(n))$ and $f(n) \neq O(g(n))$
C. $f(n) \neq O(g(n))$ and $g(n) \neq O(f(n))$
D. $f(n) = O(g(n))$ and $g(n) = O(f(n))$

gatecse-2001 algorithms asymptotic-notations time-complexity normal

Answer key

1.3.6 Asymptotic Notations: GATE CSE 2003 | Question: 20

Consider the following three claims:

I. $(n+k)^m = \Theta(n^m)$ where k and m are constants
II. $2^{n+1} = O(2^n)$
III. $2^{2n+1} = O(2^n)$

Which of the following claims are correct?

A. I and II
B. I and III
C. II and III
D. I, II, and III

gatecse-2003 algorithms asymptotic-notations normal

Answer key

1.3.7 Asymptotic Notations: GATE CSE 2004 | Question: 29

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A. n
B. n^2
C. $n \log n$
D. $n \log^2 n$

gatecse-2004 algorithms sorting asymptotic-notations easy

Answer key

1.3.8 Asymptotic Notations: GATE CSE 2005 | Question: 37

Suppose $T(n) = 2T(\frac{n}{2}) + n, T(0) = T(1) = 1$

Which one of the following is FALSE?

A. $T(n) = O(n^2)$
B. $T(n) = \Theta(n \log n)$
C. $T(n) = \Omega(n^2)$
D. $T(n) = O(n \log n)$

gatecse-2005 algorithms asymptotic-notations recurrence-relation normal

Answer key

1.3.9 Asymptotic Notations: GATE CSE 2008 | Question: 39

Consider the following functions:

- $f(n) = 2^n$
- $g(n) = n!$
- $h(n) = n^{\log n}$

Which of the following statements about the asymptotic behavior of $f(n)$, $g(n)$ and $h(n)$ is true?

A. $f(n) = O(g(n)); g(n) = O(h(n))$
B. $f(n) = \Omega(g(n)); g(n) = O(h(n))$
C. $g(n) = O(f(n)); h(n) = O(f(n))$
D. $h(n) = O(f(n)); g(n) = \Omega(f(n))$

gatecse-2008 algorithms asymptotic-notations normal

Answer key

1.3.10 Asymptotic Notations: GATE CSE 2011 | Question: 37

Which of the given options provides the increasing order of asymptotic complexity of functions f_1, f_2, f_3 and f_4?

- $f_1(n) = 2^n$
- $f_2(n) = n^{3/2}$
- $f_3(n) = n \log_2 n$
- $f_4(n) = n^{\log_2 n}$

A. f_3, f_2, f_4, f_1
B. f_3, f_2, f_1, f_4
C. f_2, f_3, f_1, f_4
D. f_2, f_3, f_4, f_1

gatecse-2011 algorithms asymptotic-notations normal

Answer key

1.3.11 Asymptotic Notations: GATE CSE 2012 | Question: 18

Let $W(n)$ and $A(n)$ denote respectively, the worst case and average case running time of an algorithm executed on an input of size n. Which of the following is **ALWAYS TRUE**?

A. $A(n) = \Omega(W(n))$
B. $A(n) = \Theta(W(n))$
C. $A(n) = \mathrm{O}(W(n))$
D. $A(n) = \mathrm{o}(W(n))$

gatecse-2012 algorithms easy asymptotic-notations

Answer key

1.3.12 Asymptotic Notations: GATE CSE 2015 | Set 3 | Question: 4

Consider the equality $\sum_{i=0}^{n} i^3 = X$ and the following choices for X:

I. $\Theta(n^4)$

II. $\Theta(n^5)$
III. $O(n^5)$
IV. $\Omega(n^3)$

The equality above remains correct if X is replaced by

A. Only I
B. Only II
C. I or III or IV but not II
D. II or III or IV but not I

gatecse-2015-set3 algorithms asymptotic-notations normal

Answer key

1.3.13 Asymptotic Notations: GATE CSE 2015 | Set 3 | Question: 42

Let $f(n) = n$ and $g(n) = n^{(1+\sin\ n)}$, where n is a positive integer. Which of the following statements is/are correct?

I. $f(n) = O(g(n))$
II. $f(n) = \Omega(g(n))$

A. Only I
B. Only II
C. Both I and II
D. Neither I nor II

gatecse-2015-set3 algorithms asymptotic-notations normal

Answer key

1.3.14 Asymptotic Notations: GATE CSE 2017 | Set 1 | Question: 04

Consider the following functions from positive integers to real numbers:

$10, \sqrt{n}, n, \log_2 n, \frac{100}{n}$.
The CORRECT arrangement of the above functions in increasing order of asymptotic complexity is:

A. $\log_2 n, \frac{100}{n}, 10, \sqrt{n}, n$
B. $\frac{100}{n}, 10, \log_2 n, \sqrt{n}, n$
C. $10, \frac{100}{n}, \sqrt{n}, \log_2 n, n$
D. $\frac{100}{n}, \log_2 n, 10, \sqrt{n}, n$

gatecse-2017-set1 algorithms asymptotic-notations normal

Answer key

1.3.15 Asymptotic Notations: GATE CSE 2021 | Set 1 | Question: 3

Consider the following three functions.

$$f_1 = 10^n \quad f_2 = n^{\log n} \quad f_3 = n^{\sqrt{n}}$$

Which one of the following options arranges the functions in the increasing order of asymptotic growth rate?

A. f_3, f_2, f_1
B. f_2, f_1, f_3
C. f_1, f_2, f_3
D. f_2, f_3, f_1

gatecse-2021-set1 algorithms asymptotic-notations one-mark

Answer key

1.3.16 Asymptotic Notations: GATE CSE 2022 | Question: 1

Which one of the following statements is TRUE for all positive functions $f(n)$?

A. $f(n^2) = \theta(f(n)^2)$, when $f(n)$ is a polynomial
B. $f(n^2) = o(f(n)^2)$
C. $f(n^2) = O(f(n)^2)$, when $f(n)$ is an exponential function
D. $f(n^2) = \Omega(f(n)^2)$

gatecse-2022 algorithms asymptotic-notations one-mark

Answer key

1.3.17 Asymptotic Notations: GATE CSE 2023 | Question: 19

Let f and g be functions of natural numbers given by $f(n) = n$ and $g(n) = n^2$. Which of the following statements is/are TRUE?

A. $f \in O(g)$
B. $f \in \Omega(g)$
C. $f \in o(g)$
D. $f \in \Theta(g)$

gatecse-2023 algorithms asymptotic-notations multiple-selects one-mark

Answer key

1.3.18 Asymptotic Notations: GATE CSE 2023 | Question: 44

Consider functions Function_1 and Function_2 expressed in pseudocode as follows:

```
Function_1
  while n>1 do
    for i=1 to n do
      x = x + 1;
    end for
    n = ⌊n/2⌋;
  end while
```

```
Function_2
  for i = 1 to 100 * n do
    x = x + 1;
  end for
```

Let $f_1(n)$ and $f_2(n)$ denote the number of times the statement "$x = x + 1$" is executed in Function_1 and Function_2, respectively.

Which of the following statements is/are TRUE?

A. $f_1(n) \in \Theta(f_2(n))$
B. $f_1(n) \in o(f_2(n))$
C. $f_1(n) \in \omega(f_2(n))$
D. $f_1(n) \in O(n)$

gatecse-2023 algorithms asymptotic-notations multiple-selects two-marks

Answer key

1.3.19 Asymptotic Notations: GATE CSE 2024 | Set 2 | Question: 5

Let $\mathrm{T(n)}$ be the recurrence relation defined as follows:

$$T(0) = 1,$$
$$T(1) = 2, \text{ and}$$
$$T(n) = 5T(n-1) - 6T(n-2) \text{ for } n \geq 2$$

Which one of the following statements is TRUE?

A. $T(n) = \Theta(2^n)$
B. $T(n) = \Theta(n2^n)$
C. $T(n) = \Theta(3^n)$
D. $T(n) = \Theta(n3^n)$

gatecse-2024-set2 algorithms recurrence-relation asymptotic-notations one-mark

Answer key

1.3.20 Asymptotic Notations: GATE CSE 2026 | Set 2 | Question: 14

Consider the following functions, where n is a positive integer.

$$n^{1/3}, \log(n), \log(n!), 2^{\log(n)}$$

Which one of the following options lists the functions in increasing order of asymptotic growth rate?

Note: Assume the base of log to be 2.

A. $\log(n), n^{1/3}, 2^{\log(n)}, \log(n!)$
B. $n^{1/3}, \log(n), \log(n!), 2^{\log(n)}$

C. $\log(n), n^{1/3}, \log(n!), 2^{\log(n)}$
D. $2^{\log(n)}, n^{1/3}, \log(n), \log(n!)$

gatecse-2026-set2 algorithms asymptotic-notations one-mark

Answer key

1.3.21 Asymptotic Notations: GATE IT 2004 | Question: 55

Let $f(n)$, $g(n)$ and $h(n)$ be functions defined for positive integers such that $f(n) = O(g(n))$, $g(n) \neq O(f(n))$, $g(n) = O(h(n))$, and $h(n) = O(g(n))$.
Which one of the following statements is FALSE?

A. $f(n) + g(n) = O(h(n) + h(n))$
B. $f(n) = O(h(n))$
C. $h(n) \neq O(f(n))$
D. $f(n)h(n) \neq O(g(n)h(n))$

gateit-2004 algorithms asymptotic-notations normal

Answer key

1.3.22 Asymptotic Notations: GATE IT 2008 | Question: 10

Arrange the following functions in increasing asymptotic order:

a. $n^{1/3}$
b. e^n
c. $n^{7/4}$
d. $n \log^9 n$
e. 1.0000001^n

A. a, d, c, e, b
B. d, a, c, e, b
C. a, c, d, e, b
D. a, c, d, b, e

gateit-2008 algorithms asymptotic-notations normal

Answer key

1.4 Bellman Ford (2)

1.4.1 Bellman Ford: GATE CSE 2009 | Question: 13

Which of the following statement(s) is/are correct regarding Bellman-Ford shortest path algorithm?

P: Always finds a negative weighted cycle, if one exists.

Q: Finds whether any negative weighted cycle is reachable from the source.

A. P only
B. Q only
C. Both P and Q
D. Neither P nor Q

gatecse-2009 algorithms graph-algorithms normal bellman-ford

Answer key

1.4.2 Bellman Ford: GATE CSE 2013 | Question: 19

What is the time complexity of Bellman-Ford single-source shortest path algorithm on a complete graph of n vertices?

A. $\theta(n^2)$
B. $\theta(n^2 \log n)$
C. $\theta(n^3)$
D. $\theta(n^3 \log n)$

gatecse-2013 algorithms graph-algorithms normal bellman-ford

Answer key

1.5 Binary Heap (1)

1.5.1 Binary Heap: GATE CSE 2026 | Set 1 | Question: 13

Let n be an odd number greater than 100. Consider a binary minheap with n elements stored in an array P whose index starts from 1.

Which of the following indices of P do/does NOT correspond to any leaf node of the minheap?

A. $\frac{n+1}{2}$ B. $\frac{n-1}{2}$ C. $\frac{n-3}{2}$ D. n

gatecse-2026-set1 algorithms binary-heap multiple-selects one-mark

Answer key

1.6 Binary Search (4)

Practice Test: Test 1 (6Q)

1.6.1 Binary Search: GATE CSE 2021 | Set 2 | Question: 8

What is the worst-case number of arithmetic operations performed by recursive binary search on a sorted array of size n?

A. $\Theta(\sqrt{n})$
B. $\Theta(\log_2(n))$
C. $\Theta(n^2)$
D. $\Theta(n)$

gatecse-2021-set2 algorithms binary-search time-complexity one-mark

Answer key

1.6.2 Binary Search: GATE DA 2025 | Question: 17

For which of the following inputs does binary search take time $O(\log n)$ in the worst case?

A. An array of n integers in any order
B. A linked list of n integers in any order
C. An array of n integers in increasing order
D. A linked list of n integers in increasing order

gateda-2025 algorithms binary-search multiple-selects one-mark

Answer key

1.6.3 Binary Search: GATE DA 2026 | Question: 21

Let A be a sorted array containing 1000 distinct integers. You perform a recursive binary search on A to find an element y. Suppose each comparison checks whether the middle element computed during the current recursive step is equal to, less than, or greater than y.

The maximum number of comparisons that may have to be performed if y is not an element of A is ____. (Answer in integer)

gateda-2026 algorithms binary-search numerical-answers one-mark

Answer key

1.6.4 Binary Search: GATE DS&AI 2024 | Question: 30

Let $F(n)$ denote the maximum number of comparisons made while searching for an entry in a sorted array of size n using binary search.

Which ONE of the following options is TRUE?

A. $F(n) = F(\lfloor n/2 \rfloor) + 1$
B. $F(n) = F(\lfloor n/2 \rfloor) + F(\lceil n/2 \rceil)$
C. $F(n) = F(\lfloor n/2 \rfloor)$
D. $F(n) = F(n-1) + 1$

gate-ds-ai-2024 algorithms binary-search two-marks

Answer key

1.7 Binary Search Tree (3)

1.7.1 Binary Search Tree: GATE CSE 2026 | Set 1 | Question: 30

Let P be the set of all integers from 1 to 15. Consider any order of insertion of the elements of P into a binary search tree that creates a complete binary tree.

Which one of the following elements can NEVER be the third element that is inserted?

A. 4 B. 2 C. 10 D. 5

gatecse-2026-set1 algorithms binary-search-tree two-marks

Answer key

1.7.2 Binary Search Tree: GATE CSE 2026 | Set 1 | Question: 52

The following sequence corresponds to the preorder traversal of a binary search tree T :

$$50, 25, 13, 40, 30, 47, 75, 60, 70, 80, 77$$

The position of the element 60 in the postorder traversal of T is ____ . (answer in integer)

Note: The position begins with 1.

gatecse-2026-set1 numerical-answers two-marks binary-search-tree algorithms

Answer key

1.7.3 Binary Search Tree: GATE CSE 2026 | Set 2 | Question: 39

Consider a binary search tree (BST) with n leaf nodes $(n > 0)$. Given any node V, the key present in the node is denoted as $\text{Val}(V)$. All the keys present in the given BST are distinct. The keys belong to the set of real numbers.

For a node V, let $\text{Suc}(V)$ denote the node that is its inorder successor. If a node V does not have an inorder successor, then $\text{Suc}(V)$ is $NULL$. As there are no duplicates, if $\text{Suc}(V)$ is not $NULL$, then $\text{Val}(V) < \text{Val}(\text{Suc}(V))$.

Corresponding to every leaf node L_i that has a non-NULL $\text{Suc}(L_i)$, a new key k_i with the following property is to be inserted into the BST.

$$\text{Val}(L_i) < k_i < Val(\text{Suc}(L_i))$$

Let K represent the list of all such new keys to be inserted into the BST.

Which of the following statements is/are true?

A. K cannot have any duplicates
B. K will have at least one element
C. After inserting all keys from K, the height of the BST can increase at most by one
D. Number of nodes in the BST will double after inserting all keys from K

gatecse-2026-set2 algorithms binary-search-tree multiple-selects two-marks

Answer key

1.8 Binary Tree (1)

1.8.1 Binary Tree: GATE CSE 2026 | Set 1 | Question: 23

The height of a binary tree is the number of edges in the longest path from the root to a leaf in the tree. The maximum possible height of a full binary tree with 23 nodes is ____.(answer in integer)

gatecse-2026-set1 algorithms binary-tree numerical-answers one-mark

Answer key

1.9 Breadth First Search (3)

1.9.1 Breadth First Search: GATE CSE 2025 | Set 1 | Question: 33

Let $G(V, E)$ be an undirected and unweighted graph with 100 vertices. Let $d(u, v)$ denote the number of edges in a shortest path between vertices u and v in V. Let the maximum value of $d(u, v), u, v \in V$ such that $u \neq v$, be 30. Let T be any breadth-first-search tree of G. Which ONE of the given options is CORRECT for every such graph G ?

A. The height of T is exactly 15.
B. The height of T is exactly 30.
C. The height of T is at least 15.
D. The height of T is at least 30.

gatecse2025-set1 algorithms breadth-first-search shortest-path two-marks

Answer key

1.9.2 Breadth First Search: GATE DA 2026 | Question: 30

Consider a directed graph $G = (V, E)$, where V is the finite set of vertices and E is the set of directed edges between the vertices. G may contain cycles but there is no self-loop. Further, G may not be strongly connected.

Let G^R be the graph obtained by reversing the directions of all the edges in G without changing the set of vertices. Assume that Breadth First Search (BFS) or Depth First Search (DFS) from any given vertex v of a graph visits only the reachable vertices from v in that graph.

Which of the following statements must always be true, regardless of the structure of G?

A. If u is a reachable vertex in the BFS of G^R from v, then u is also a reachable vertex in the DFS of G from v.
B. In G^R, the BFS traversal from v will visit exactly the same set of vertices as the DFS from v in G.
C. The order of vertices visited in the BFS of G^R from v is the reverse of the order of vertices visited in the DFS of G from v.
D. If u is a reachable vertex in the DFS of G from v, then v is also a reachable vertex in the BFS of G^R from u.

gateda-2026 algorithms graph-algorithms breadth-first-search depth-first-search two-marks

1.9.3 Breadth First Search: GATE DS&AI 2024 | Question: 34

Consider a state space where the start state is number 1. The successor function for the state numbered n returns two states numbered $n + 1$ and $n + 2$. Assume that the states in the unexpanded state list are expanded in the ascending order of numbers and the previously expanded states are not added to the unexpanded state list.

Which ONE of the following statements about breadth-first search (BFS) and depth-first search (DFS) is true, when reaching the goal state number 6?

A. BFS expands more states than DFS.
B. DFS expands more states than BFS.
C. Both BFS and DFS expand equal number of states.
D. Both BFS and DFS do not reach the goal state number 6.

gate-ds-ai-2024 algorithms breadth-first-search depth-first-search two-marks

Answer key

1.10 Bubble Sort (1)

1.10.1 Bubble Sort: GATE CSE 1995 | Question: 12

Consider the following sequence of numbers:

$$92, 37, 52, 12, 11, 25$$

Use Bubble sort to arrange the sequence in ascending order. Give the sequence at the end of each of the first five passes.

gate1995 algorithms sorting easy descriptive bubble-sort

Answer key

1.11 Computer Science (1)

1.11.1 Computer Science: GATE CS Practice : The "Master Theorem" (Algorithms)

Consider the following recurrence relation describing the running time of an algorithm:

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$$

$$(Base\ condition: T(1) = 1)$$

What is the asymptotic tight bound $\Theta(T(n))$?

A. $\Theta(n)$

B. $\Theta(n \log n)$

C. $\Theta(n \log \log n)$

D. $\Theta(n(\log n)^2)$

algorithms master-theorem recurrence-relation computer-science gate-preparation

Answer key

1.12 Depth First Search (2)

✍ **Practice Test:** Test 1 (12Q)

1.12.1 Depth First Search: GATE CSE 2026 | Set 1 | Question: 40

Consider the following pseudocode for depth-first search (DFS) algorithm which takes a directed graph $G(V, E)$ as input, where $d[v]$ and $f[v]$ are the discovery time and finishing time, respectively, of the vertex $v \in V$.

```
DFS(G):
    unmark all v ∈ V
    t ← 0
    for each v ∈ V
        if v is unmarked
            t ← Explore(G, v, t)
        end if
    end for
```

```
Explore(G, v, t):
    mark v
    t ← t + 1
    d[v] ← t
    for each (v, w) ∈ E
        if w is unmarked
            t ← Explore(G, w, t)
        end if
    end for
    t ← t + 1
    f[v] ← t
    return t
```

Suppose that the input directed graph $G(V, E)$ is a directed acyclic graph (DAG).
For an edge $(u, v) \in E$, which of the following options will NEVER be correct?

A. $d[u] < d[v] < f[v] < f[u]$

B. $d[v] < d[u] < f[u] < f[v]$

C. $d[v] < f[v] < d[u] < f[u]$

D. $d[u] < d[v] < f[u] < f[v]$

gatecse-2026-set1 algorithms depth-first-search two-marks multiple-selects

Answer key

1.12.2 Depth First Search: GATE DA 2025 | Question: 55

Consider a directed graph $G = (V, E)$, where $V = \{0, 1, 2, \ldots, 100\}$ and $E = \{(i, j) : 0 < j - i \leq 2, \text{ for all } i, j \in V\}$. Suppose the adjacency list of each vertex is in decreasing order of vertex number, and depth-first search (DFS) is performed at vertex 0. The number of vertices that will be discovered after vertex 50 is _____ (Answer in integer)

gateda-2025 algorithms depth-first-search numerical-answers two-marks

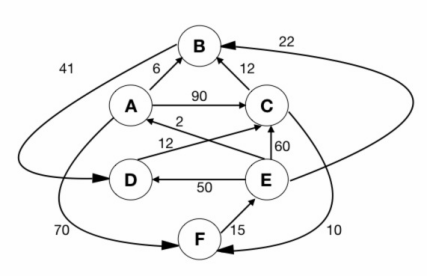
Answer key

1.13 Dijkstras Algorithm (5)

Practice Test: Test 1 (12Q)

1.13.1 Dijkstras Algorithm: GATE CSE 1996 | Question: 17

Let G be the directed, weighted graph shown in below figure



We are interested in the shortest paths from A.

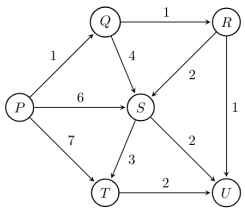
a. Output the sequence of vertices identified by the Dijkstra's algorithm for single source shortest path when the algorithm is started at node A
b. Write down sequence of vertices in the shortest path from A to E
c. What is the cost of the shortest path from A to E?

gate1996 algorithms graph-algorithms normal dijkstras-algorithm descriptive

Answer key

1.13.2 Dijkstras Algorithm: GATE CSE 2004 | Question: 44

Suppose we run Dijkstra's single source shortest path algorithm on the following edge-weighted directed graph with vertex P as the source.



In what order do the nodes get included into the set of vertices for which the shortest path distances are finalized?

A. P, Q, R, S, T, U B. P, Q, R, U, S, T C. P, Q, R, U, T, S D. P, Q, T, R, U, S

gatecse-2004 algorithms graph-algorithms normal dijkstras-algorithm

Answer key

1.13.3 Dijkstras Algorithm: GATE CSE 2005 | Question: 38

Let $G(V, E)$ be an undirected graph with positive edge weights. Dijkstra's single source shortest path algorithm can be implemented using the binary heap data structure with time complexity:

A. $O\left(|V|^2\right)$ B. $O(|E| + |V| \log |V|)$

C. $O(|V|\log|V|)$

D. $O((|E|+|V|)\log|V|)$

gatecse-2005 algorithms graph-algorithms normal dijkstras-algorithm

Answer key

1.13.4 Dijkstras Algorithm: GATE CSE 2006 | Question: 12

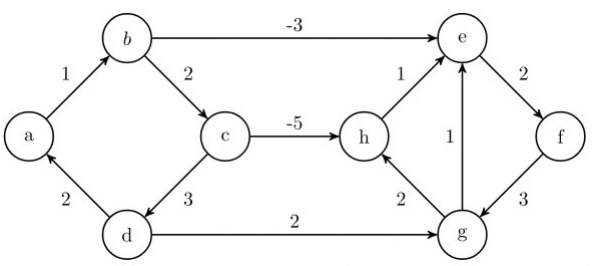
To implement Dijkstra's shortest path algorithm on unweighted graphs so that it runs in linear time, the data structure to be used is:

A. Queue B. Stack C. Heap D. B-Tree

gatecse-2006 algorithms graph-algorithms easy dijkstras-algorithm

Answer key

1.13.5 Dijkstras Algorithm: GATE CSE 2008 | Question: 45



Dijkstra's single source shortest path algorithm when run from vertex a in the above graph, computes the correct shortest path distance to

A. only vertex a

B. only vertices a, e, f, g, h

C. only vertices a, b, c, d

D. all the vertices

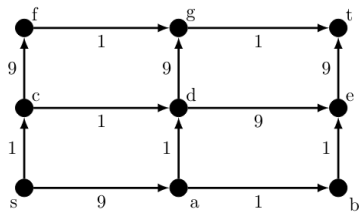
gatecse-2008 algorithms graph-algorithms normal dijkstras-algorithm

Answer key

1.14 Directed Acyclic Graph (2)

1.14.1 Directed Acyclic Graph: GATE CSE 2021 | Set 2 | Question: 55

In a directed acyclic graph with a source vertex **s**, the $quality\text{-}score$ of a directed path is defined to be the product of the weights of the edges on the path. Further, for a vertex v other than **s**, the quality-score of v is defined to be the maximum among the quality-scores of all the paths from **s** to v. The quality-score of **s** is assumed to be 1.



The sum of the quality-scores of all vertices on the graph shown above is _____

gatecse-2021-set2 algorithms graph-algorithms directed-acyclic-graph numerical-answers two-marks

Answer key

1.14.2 Directed Acyclic Graph: GATE CSE 2026 | Set 2 | Question: 27

Let G be a weighted directed acyclic graph with m edges and n vertices. Given G and a source vertex s in G, which one of the following options gives the worst case time complexity of the fastest algorithm to find the lengths of shortest paths from s to all vertices that are reachable from s in G?

A. $\Theta(m+n)$

B. $\Theta(m+n\log(n))$

C. $\Theta(nm)$ D. $\Theta(n^3)$

gatecse-2026-set2 algorithms shortest-path directed-acyclic-graph time-complexity two-marks

Answer key

1.15 Double Hashing (2)

1.15.1 Double Hashing: GATE CSE 2020 | Question: 23

Consider a double hashing scheme in which the primary hash function is $h_1(k) = k \bmod 23$, and the secondary hash function is $h_2(k) = 1 + (k \bmod 19)$. Assume that the table size is 23. Then the address returned by probe 1 in the probe sequence (assume that the probe sequence begins at probe 0) for key value $k = 90$ is_____.

gatecse-2020 numerical-answers algorithms hashing one-mark double-hashing

Answer key

1.15.2 Double Hashing: GATE CSE 2025 | Set 1 | Question: 55

In a double hashing scheme, $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 7)$ are the auxiliary hash functions. The size m of the hash table is 11. The hash function for the i-th probe in the open address table is $[h_1(k) + ih_2(k)] \bmod m$. The following keys are inserted in the given order: $63, 50, 25, 79, 67, 24$.

The slot at which key 24 gets stored is _____ . (Answer in integer)

gatecse2025-set1 algorithms hashing double-hashing numerical-answers easy two-marks

Answer key

1.16 Dynamic Programming (10)

✍ **Practice Test:** Test 1 (13Q)

1.16.1 Dynamic Programming: GATE CSE 2008 | Question: 80

The subset-sum problem is defined as follows. Given a set of n positive integers, $S = \{a_1, a_2, a_3, \ldots, a_n\}$, and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array, X, with n rows and $W + 1$ columns. $X[i, j], 1 \leq i \leq n, 0 \leq j \leq W$, is TRUE, if and only if there is a subset of $\{a_1, a_2, \ldots, a_i\}$ whose elements sum to j.

Which of the following is valid for $2 \leq i \leq n$, and $a_i \leq j \leq W$?

A. $X[i, j] = X[i-1, j] \vee X[i, j-a_i]$
B. $X[i, j] = X[i-1, j] \vee X[i-1, j-a_i]$
C. $X[i, j] = X[i-1, j] \wedge X[i, j-a_i]$
D. $X[i, j] = X[i-1, j] \wedge X[i-1, j-a_i]$

gatecse-2008 algorithms difficult dynamic-programming

Answer key

1.16.2 Dynamic Programming: GATE CSE 2008 | Question: 81

The subset-sum problem is defined as follows. Given a set of n positive integers, $S = \{a_1, a_2, a_3, \ldots, a_n\}$, and positive integer W, is there a subset of S whose elements sum to W? A dynamic program for solving this problem uses a 2-dimensional Boolean array, X, with n rows and $W + 1$ columns. $X[i, j], 1 \leq i \leq n, 0 \leq j \leq W$, is TRUE, if and only if there is a subset of $\{a_1, a_2, \ldots, a_i\}$ whose elements sum to j.

Which entry of the array X, if TRUE, implies that there is a subset whose elements sum to W?

A. $X[1, W]$ B. $X[n, 0]$ C. $X[n, W]$ D. $X[n-1, n]$

gatecse-2008 algorithms normal dynamic-programming

Answer key

1.16.3 Dynamic Programming: GATE CSE 2009 | Question: 53

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences $X[m]$ and $Y[n]$ of lengths m and n, respectively with indexes of X and Y starting from 0.

We wish to find the length of the longest common sub-sequence (LCS) of $X[m]$ and $Y[n]$ as $l(m,n)$, where an incomplete recursive definition for the function $I(i,j)$ to compute the length of the LCS of $X[m]$ and $Y[n]$ is given below:

l(i,j) = 0, if either i = 0 or j = 0
= expr1, if i,j > 0 and X[i-1] = Y[j-1]
= expr2, if i,j > 0 and X[i-1] ≠ Y[j-1]

Which one of the following options is correct?

A. $\text{expr1} = l(i-1,j)+1$
B. $\text{expr1} = l(i,j-1)$
C. $\text{expr2} = \max(l(i-1,j), l(i,j-1))$
D. $\text{expr2} = \max(l(i-1,j-1), l(i,j))$

gatecse-2009 algorithms normal dynamic-programming recursion

Answer key

1.16.4 Dynamic Programming: GATE CSE 2009 | Question: 54

A sub-sequence of a given sequence is just the given sequence with some elements (possibly none or all) left out. We are given two sequences $X[m]$ and $Y[n]$ of lengths m and n, respectively with indexes of X and Y starting from 0.

We wish to find the length of the longest common sub-sequence (LCS) of $X[m]$ and $Y[n]$ as $l(m,n)$, where an incomplete recursive definition for the function $I(i,j)$ to compute the length of the LCS of $X[m]$ and $Y[n]$ is given below:

l(i,j) = 0, if either i = 0 or j = 0
= expr1, if i,j > 0 and X[i-1] = Y[j-1]
= expr2, if i,j > 0 and X[i-1] ≠ Y[j-1]

The value of $l(i,j)$ could be obtained by dynamic programming based on the correct recursive definition of $l(i,j)$ of the form given above, using an array $L[M,N]$, where $M = m+1$ and $N = n+1$, such that $L[i,j] = l(i,j)$.

Which one of the following statements would be TRUE regarding the dynamic programming solution for the recursive definition of $l(i,j)$?

A. All elements of L should be initialized to 0 for the values of $l(i,j)$ to be properly computed.
B. The values of $l(i,j)$ may be computed in a row major order or column major order of $L[M,N]$.
C. The values of $l(i,j)$ cannot be computed in either row major order or column major order of $L[M,N]$.
D. $L[p,q]$ needs to be computed before $L[r,s]$ if either $p < r$ or $q < s$.

gatecse-2009 normal algorithms dynamic-programming recursion

Answer key

1.16.5 Dynamic Programming: GATE CSE 2010 | Question: 34

The weight of a sequence $a_0, a_1, \ldots, a_{n-1}$ of real numbers is defined as $a_0 + a_1/2 + \cdots + a_{n-1}/2^{n-1}$. A subsequence of a sequence is obtained by deleting some elements from the sequence, keeping the order of the remaining elements the same. Let X denote the maximum possible weight of a subsequence of $a_o, a_1, \ldots, a_{n-1}$ and Y the maximum possible weight of a subsequence of $a_1, a_2, \ldots, a_{n-1}$. Then X is equal to

A. $max(Y, a_0 + Y)$
B. $max(Y, a_0 + Y/2)$
C. $max(Y, a_0 + 2Y)$
D. $a_0 + Y/2$

gatecse-2010 algorithms dynamic-programming normal

Answer key

1.16.6 Dynamic Programming: GATE CSE 2011 | Question: 25

An algorithm to find the length of the longest monotonically increasing sequence of numbers in an array $A[0:n-1]$ is given below.

Let L_i, denote the length of the longest monotonically increasing sequence starting at index i in the array.

Initialize $L_{n-1} = 1$.

For all i such that $0 \leq i \leq n-2$

$$L_i = \begin{cases} 1 + L_{i+1} & \text{if A[i]} < \text{A[i+1]} \\ 1 & \text{Otherwise} \end{cases}$$

Finally, the length of the longest monotonically increasing sequence is $\max(L_0, L_1, \ldots, L_{n-1})$.

Which of the following statements is **TRUE**?

A. The algorithm uses dynamic programming paradigm
B. The algorithm has a linear complexity and uses branch and bound paradigm
C. The algorithm has a non-linear polynomial complexity and uses branch and bound paradigm
D. The algorithm uses divide and conquer paradigm

gatecse-2011 algorithms easy dynamic-programming

Answer key

1.16.7 Dynamic Programming: GATE CSE 2014 | Set 2 | Question: 37

Consider two strings A = *"qpqrr"* and B = *"pqprqrp"*. Let x be the length of the longest common subsequence *(not necessarily contiguous)* between A and B and let y be the number of such longest common subsequences between A and B. Then $x + 10y =$ ___.

gatecse-2014-set2 algorithms normal numerical-answers dynamic-programming

Answer key

1.16.8 Dynamic Programming: GATE CSE 2014 | Set 3 | Question: 37

Suppose you want to move from 0 to 100 on the number line. In each step, you either move right by a unit distance or you take a *shortcut*. A shortcut is simply a pre-specified pair of integers i, j with $i < j$. Given a shortcut (i, j), if you are at position i on the number line, you may directly move to j. Suppose $T(k)$ denotes the smallest number of steps needed to move from k to 100. Suppose further that there is at most 1 shortcut involving any number, and in particular, from 9 there is a shortcut to 15. Let y and z be such that $T(9) = 1 + \min(T(y), T(z))$. Then the value of the product yz is _____.

gatecse-2014-set3 algorithms normal numerical-answers dynamic-programming

Answer key

1.16.9 Dynamic Programming: GATE CSE 2016 | Set 2 | Question: 14

The Floyd-Warshall algorithm for all-pair shortest paths computation is based on

A. Greedy paradigm.
B. Divide-and-conquer paradigm.
C. Dynamic Programming paradigm.
D. Neither Greedy nor Divide-and-Conquer nor Dynamic Programming paradigm.

gatecse-2016-set2 algorithms dynamic-programming easy

Answer key

1.16.10 Dynamic Programming: GATE CSE 2026 | Set 2 | Question: 29

Consider a table T, where the elements $T[i][j], 0 \leq i, j \leq n$, represent the cost of the optimal solutions of different subproblems of a problem that is being solved using a dynamic programming algorithm. The recursive formulation to compute the table entries is as follows:

$$T[0][k] = T[k][0] = 1 \qquad \text{for } k = 0, 1, 2, \ldots, n$$
$$T[i][j] = 2T[i-1][j] + 3T[i][j-1] \qquad \text{for } 1 \leq i, j \leq n$$

Consider the following two algorithms to compute entries of T. Assume that for both the algorithms, for all $0 \leq i, j \leq n, T[i][j]$ has been initialized to 1.
Algorithm B_1 : For $i = 1, 2, \ldots, n$

$$\begin{aligned}&\text{For } j = 1, 2, \ldots, n\\ &\quad T[i][j] = 2T[i-1][j] + 3T[i][j-1]\end{aligned}$$

Algorithm B_2 : For $s = 2, 3, \ldots, 2n$

$$\begin{aligned}&\text{For } i = 1, 2, \ldots, n\\ &\quad \text{For } j = 1, 2, \ldots, n\\ &\quad\quad \text{If } (i + j == s)\\ &\quad\quad\quad T[i][j] = 2T[i-1][j] + 3T[i][j-1]\end{aligned}$$

Algorithm $B_k, k \in \{1, 2\}$ is said to be correct if and only if it calculates the correct values of $T[i][j]$, for all $0 \leq i, j \leq n$, (as per the recursive formulation) at the end of the execution of the algorithm B_k.

Which one of the following statements is true?

A. Both algorithms B_1 and B_2 are correct
B. Algorithm B_1 is correct, but algorithm B_2 is incorrect
C. Algorithm B_2 is correct, but algorithm B_1 is incorrect
D. Both algorithms B_1 and B_2 are incorrect

gatecse-2026-set2 algorithms dynamic-programming two-marks

Answer key

1.17 Graph Algorithms (11)

Practice Test: Test 1 (14Q)

1.17.1 Graph Algorithms: GATE CSE 1994 | Question: 1.22

Which of the following statements is false?

A. Optimal binary search tree construction can be performed efficiently using dynamic programming
B. Breadth-first search cannot be used to find connected components of a graph
C. Given the prefix and postfix walks over a binary tree, the binary tree cannot be uniquely constructed.
D. Depth-first search can be used to find connected components of a graph

gate1994 algorithms normal graph-algorithms

Answer key

1.17.2 Graph Algorithms: GATE CSE 2003 | Question: 70

Let $G = (V, E)$ be a directed graph with n vertices. A path from v_i to v_j in G is a sequence of vertices ($v_i, v_{i+1}, \ldots, v_j$) such that $(v_k, v_{k+1}) \in E$ for all k in i through $j - 1$. A simple path is a path in which no vertex appears more than once.

Let A be an $n \times n$ array initialized as follows:

$$A[j,k] = \begin{cases} 1, & \text{if } (j,k) \in E \\ 0, & \text{otherwise} \end{cases}$$

Consider the following algorithm:

```
for i=1 to n
   for j=1 to n
      for k=1 to n
         A[j,k] = max(A[j,k], A[j,i] + A[i,k]);
```

Which of the following statements is necessarily true for all j and k after termination of the above algorithm?

A. $A[j,k] \leq n$
B. If $A[j,j] \geq n-1$ then G has a Hamiltonian cycle
C. If there exists a path from j to k, $A[j,k]$ contains the longest path length from j to k
D. If there exists a path from j to k, every simple path from j to k contains at most $A[j,k]$ edges

gatecse-2003 algorithms graph-algorithms normal

Answer key

1.17.3 Graph Algorithms: GATE CSE 2005 | Question: 82a

Let s and t be two vertices in a undirected graph $G = (V, E)$ having distinct positive edge weights. Let $[X, Y]$ be a partition of V such that $s \in X$ and $t \in Y$. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y.

The edge e must definitely belong to:

A. the minimum weighted spanning tree of G
B. the weighted shortest path from s to t
C. each path from s to t
D. the weighted longest path from s to t

gatecse-2005 algorithms graph-algorithms normal

Answer key

1.17.4 Graph Algorithms: GATE CSE 2005 | Question: 82b

Let s and t be two vertices in a undirected graph $G = (V, E)$ having distinct positive edge weights. Let $[X, Y]$ be a partition of V such that $s \in X$ and $t \in Y$. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y.

Let the weight of an edge e denote the congestion on that edge. The congestion on a path is defined to be the maximum of the congestions on the edges of the path. We wish to find the path from s to t having minimum congestion. Which of the following paths is always such a path of minimum congestion?

A. a path from s to t in the minimum weighted spanning tree
B. a weighted shortest path from s to t
C. an Euler walk from s to t
D. a Hamiltonian path from s to t

gatecse-2005 algorithms graph-algorithms normal

Answer key

1.17.5 Graph Algorithms: GATE CSE 2016 | Set 2 | Question: 41

In an adjacency list representation of an undirected simple graph $G = (V, E)$, each edge (u, v) has two adjacency list entries: $[v]$ in the adjacency list of u, and $[u]$ in the adjacency list of v. These are called twins of each other. A twin pointer is a pointer from an adjacency list entry to its twin. If $|E| = m$ and $|V| = n$, and the memory size is not a constraint, what is the time complexity of the most efficient algorithm to set the twin pointer in each entry in each adjacency list?

A. $\Theta(n^2)$
B. $\Theta(n+m)$
C. $\Theta(m^2)$
D. $\Theta(n^4)$

gatecse-2016-set2 algorithms graph-algorithms normal

Answer key

1.17.6 Graph Algorithms: GATE CSE 2017 | Set 1 | Question: 26

Let $G = (V, E)$ be *any* connected, undirected, edge-weighted graph. The weights of the edges in E are positive and distinct. Consider the following statements:

I. Minimum Spanning Tree of G is always unique.
II. Shortest path between any two vertices of G is always unique.

Which of the above statements is/are necessarily true?

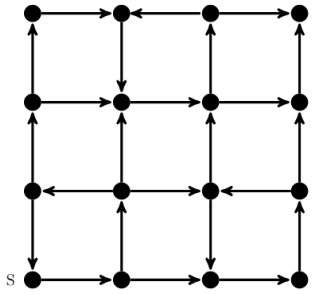
A. I only B. II only C. both I and II D. neither I nor II

gatecse-2017-set1 algorithms graph-algorithms normal

Answer key

1.17.7 Graph Algorithms: GATE CSE 2021 | Set 2 | Question: 46

Consider the following directed graph:



Which of the following is/are correct about the graph?

A. The graph does not have a topological order
B. A depth-first traversal starting at vertex S classifies three directed edges as back edges
C. The graph does not have a strongly connected component
D. For each pair of vertices u and v, there is a directed path from u to v

gatecse-2021-set2 multiple-selects algorithms graph-algorithms two-marks

Answer key

1.17.8 Graph Algorithms: GATE CSE 2026 | Set 1 | Question: 31

Let $G(V, E)$ be an undirected, edge-weighted graph with integer weights. The weight of a path is the sum of the weights of the edges in that path. The length of a path is the number of edges in that path.

Let $s \in V$ be a vertex in G. For every $u \in V$ and for every $k \geq 0$, let $d_k(u)$ denote the weight of a shortest path (in terms of weight) from s to u of length at most k. If there is no path from s to u of length at most k, then $d_k(u) = \infty$.

Consider the statements:

S1: For every $k \geq 0$ and $u \in V, d_{k+1}(u) \leq d_k(u)$.

S2: For every $(u, v) \in E$, if (u, v) is part of a shortest path (in terms of weight) from s to v, then for every $k \geq 0, d_k(u) \leq d_k(v)$.

Which one of the following options is correct?

A. Only $\mathrm{S1}$ is true
B. Only $\mathrm{S2}$ is true
C. Both $\mathrm{S1}$ and $\mathrm{S2}$ are true
D. Neither $\mathrm{S1}$ nor $\mathrm{S2}$ is true

gatecse-2026-set1 algorithms two-marks graph-algorithms

Answer key

1.17.9 Graph Algorithms: GATE IT 2005 | Question: 15

In the following table, the left column contains the names of standard graph algorithms and the right column contains the time complexities of the algorithms. Match each algorithm with its time complexity.

1.	Bellman-Ford algorithm	A:	$O(m \log n)$
2.	Kruskal's algorithm	B:	$O(n^3)$
3.	Floyd-Warshall algorithm	C:	$O(nm)$
4.	Topological sorting	D:	$O(n+m)$

A. $1 \rightarrow \text{C}, 2 \rightarrow \text{A}, 3 \rightarrow \text{B}, 4 \rightarrow \text{D}$
B. $1 \rightarrow \text{B}, 2 \rightarrow \text{D}, 3 \rightarrow \text{C}, 4 \rightarrow \text{A}$
C. $1 \rightarrow \text{C}, 2 \rightarrow \text{D}, 3 \rightarrow \text{A}, 4 \rightarrow \text{B}$
D. $1 \rightarrow \text{B}, 2 \rightarrow \text{A}, 3 \rightarrow \text{C}, 4 \rightarrow \text{D}$

gateit-2005 algorithms graph-algorithms match-the-following easy

Answer key

1.17.10 Graph Algorithms: GATE IT 2005 | Question: 84a

A sink in a directed graph is a vertex i such that there is an edge from every vertex $j \neq i$ to i and there is no edge from i to any other vertex. A directed graph G with n vertices is represented by its adjacency matrix A, where $A[i][j] = 1$ if there is an edge directed from vertex i to j and 0 otherwise. The following algorithm determines whether there is a sink in the graph G.

```
i = 0;
do {
   j = i + 1;
   while ((j < n) && E1) j++;
   if (j < n) E2;
} while (j < n);
flag = 1;
for (j = 0; j < n; j++)
   if ((j! = i) && E3) flag = 0;
if (flag) printf("Sink exists");
else printf ("Sink does not exist");
```

Choose the correct expressions for E_1 and E_2

A. $E_1 : A[i][j]$ and $E_2 : i = j;$
B. $E_1 : \ !A[i][j]$ and $E_2 : i = j + 1;$
C. $E_1 : \ !A[i][j]$ and $E_2 : i = j;$
D. $E_1 : A[i][j]$ and $E_2 : i = j + 1;$

gateit-2005 algorithms graph-algorithms normal

Answer key

1.17.11 Graph Algorithms: GATE IT 2005 | Question: 84b

A sink in a directed graph is a vertex i such that there is an edge from every vertex $j \neq i$ to i and there is no edge from i to any other vertex. A directed graph G with n vertices is represented by its adjacency matrix A, where $A[i][j] = 1$ if there is an edge directed from vertex i to j and 0 otherwise. The following algorithm determines whether there is a sink in the graph G.

```
i = 0;
do {
   j = i + 1;
   while ((j < n) && E1) j++;
   if (j < n) E2;
} while (j < n);
flag = 1;
for (j = 0; j < n; j++)
   if ((j! = i) && E3) flag = 0;
if (flag) printf("Sink exists") ;
else printf ("Sink does not exist");
```

Choose the correct expression for E_3

A. $(A[i][j] \ \&\& \ !A[j][i])$
B. $(!A[i][j] \ \&\& \ A[j][i])$
C. $(!A[i][j] \ || \ A[j][i])$
D. $(A[i][j] \ || \ !A[j][i])$

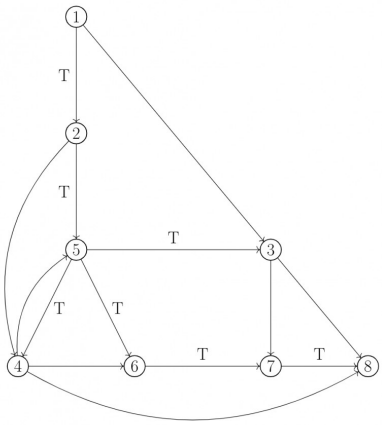
gateit-2005 algorithms graph-algorithms normal

Answer key

1.18 Graph Search (23)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (7Q)

1.18.1 Graph Search: GATE CSE 1989 | Question: 4-vii



In the graph shown above, the depth-first spanning tree edges are marked with a $'T'$. Identify the forward, backward, and cross edges.

gate1989 descriptive algorithms graph-algorithms depth-first-search graph-search

Answer key

1.18.2 Graph Search: GATE CSE 2000 | Question: 1.13

The most appropriate matching for the following pairs

X: depth first search	1: heap
Y: breadth first search	2: queue
Z: sorting	3: stack

is:

A. $\mathrm{X}-1,\ \mathrm{Y}-2,\ \mathrm{Z}-3$
B. $\mathrm{X}-3,\ \mathrm{Y}-1,\ \mathrm{Z}-2$
C. $\mathrm{X}-3,\ \mathrm{Y}-2,\ \mathrm{Z}-1$
D. $\mathrm{X}-2,\ \mathrm{Y}-3,\ \mathrm{Z}-1$

gatecse-2000 algorithms easy graph-algorithms graph-search match-the-following

Answer key

1.18.3 Graph Search: GATE CSE 2000 | Question: 2.19

Let G be an undirected graph. Consider a depth-first traversal of G, and let T be the resulting depth-first search tree. Let u be a vertex in G and let v be the first new (unvisited) vertex visited after visiting u in the traversal. Which of the following statement is always true?

A. $\{u, v\}$ must be an edge in G, and u is a descendant of v in T
B. $\{u, v\}$ must be an edge in G, and v is a descendant of u in T
C. If $\{u, v\}$ is not an edge in G then u is a leaf in T
D. If $\{u, v\}$ is not an edge in G then u and v must have the same parent in T

gatecse-2000 algorithms graph-algorithms normal graph-search

Answer key

1.18.4 Graph Search: GATE CSE 2001 | Question: 2.14

Consider an undirected, unweighted graph G. Let a breadth-first traversal of G be done starting from a node r. Let $d(r, u)$ and $d(r, v)$ be the lengths of the shortest paths from r to u and v respectively in G. If u is visited before v during the breadth-first traversal, which of the following statements is correct?

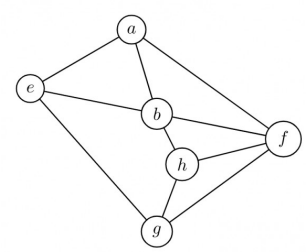
A. $d(r,u) < d(r,v)$
B. $d(r,u) > d(r,v)$
C. $d(r,u) \leq d(r,v)$
D. None of the above

gatecse-2001 algorithms graph-algorithms normal graph-search

Answer key

1.18.5 Graph Search: GATE CSE 2003 | Question: 21

Consider the following graph:



Among the following sequences:

I. abeghf
II. abfehg
III. abfhge
IV. afghbe

Which are the depth-first traversals of the above graph?

A. I, II and IV only
B. I and IV only
C. II, III and IV only
D. I, III and IV only

gatecse-2003 algorithms graph-algorithms normal graph-search

Answer key

1.18.6 Graph Search: GATE CSE 2006 | Question: 48

Let T be a depth first search tree in an undirected graph G. Vertices u and v are leaves of this tree T. The degrees of both u and v in G are at least 2. which one of the following statements is true?

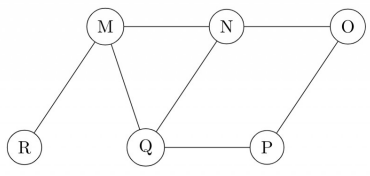
A. There must exist a vertex w adjacent to both u and v in G
B. There must exist a vertex w whose removal disconnects u and v in G
C. There must exist a cycle in G containing u and v
D. There must exist a cycle in G containing u and all its neighbours in G

gatecse-2006 algorithms graph-algorithms normal graph-search

Answer key

1.18.7 Graph Search: GATE CSE 2008 | Question: 19

The Breadth First Search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is:



A. MNOPQR
B. NQMPOR
C. QMNPRO
D. QMNPOR

gatecse-2008 normal algorithms graph-algorithms graph-search

Answer key

1.18.8 Graph Search: GATE CSE 2014 | Set 1 | Question: 11

Let G be a graph with n vertices and m edges. What is the tightest upper bound on the running time of Depth First Search on G, when G is represented as an adjacency matrix?

A. $\Theta(n)$ B. $\Theta(n+m)$ C. $\Theta(n^2)$ D. $\Theta(m^2)$

gatecse-2014-set1 algorithms graph-algorithms normal graph-search

Answer key

1.18.9 Graph Search: GATE CSE 2014 | Set 2 | Question: 14

Consider the tree arcs of a BFS traversal from a source node W in an unweighted, connected, undirected graph. The tree T formed by the tree arcs is a data structure for computing

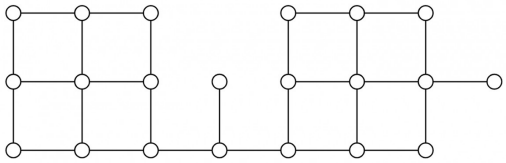
A. the shortest path between every pair of vertices.
B. the shortest path from W to every vertex in the graph.
C. the shortest paths from W to only those nodes that are leaves of T.
D. the longest path in the graph.

gatecse-2014-set2 algorithms graph-algorithms normal graph-search

Answer key

1.18.10 Graph Search: GATE CSE 2014 | Set 3 | Question: 13

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is _____.



gatecse-2014-set3 algorithms graph-algorithms numerical-answers normal graph-search

Answer key

1.18.11 Graph Search: GATE CSE 2015 | Set 1 | Question: 45

Let $G = (V, E)$ be a simple undirected graph, and s be a particular vertex in it called the source. For $x \in V$, let $d(x)$ denote the shortest distance in G from s to x. A breadth first search (BFS) is performed starting at s. Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T, then which one of the following CANNOT be the value of $d(u) - d(v)$?

A. -1 B. 0 C. 1 D. 2

gatecse-2015-set1 algorithms graph-algorithms normal graph-search

Answer key

1.18.12 Graph Search: GATE CSE 2016 | Set 2 | Question: 11

Breadth First Search (BFS) is started on a binary tree beginning from the root vertex. There is a vertex t at a distance four from the root. If t is the n^{th} vertex in this BFS traversal, then the maximum possible value of n is _____

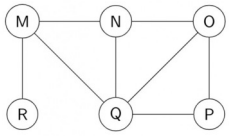
gatecse-2016-set2 algorithms graph-algorithms normal numerical-answers graph-search

Answer key

1.18.13 Graph Search: GATE CSE 2017 | Set 2 | Question: 15

The Breadth First Search (BFS) algorithm has been implemented using the queue data structure. Which one

of the following is a possible order of visiting the nodes in the graph below?



A. MNOPQR B. NQMPOR C. QMNROP D. POQNMR

gatecse-2017-set2 algorithms graph-algorithms graph-search

Answer key

1.18.14 Graph Search: GATE CSE 2018 | Question: 30

Let G be a simple undirected graph. Let T_D be a depth first search tree of G. Let T_B be a breadth first search tree of G. Consider the following statements.

I. No edge of G is a cross edge with respect to T_D. (A cross edge in G is between two nodes neither of which is an ancestor of the other in T_D).
II. For every edge (u, v) of G, if u is at depth i and v is at depth j in T_B, then $|i - j| = 1$.

Which of the statements above must necessarily be true?

A. I only B. II only C. Both I and II D. Neither I nor II

gatecse-2018 algorithms graph-algorithms graph-search normal two-marks

Answer key

1.18.15 Graph Search: GATE CSE 2023 | Question: 46

Let $U = \{1, 2, 3\}$. Let 2^U denote the powerset of U. Consider an undirected graph G whose vertex set is 2^U. For any $A, B \in 2^U$, (A, B) is an edge in G if and only if (i) $A \neq B$, and (ii) either $A \subsetneq B$ or $B \subsetneq A$. For any vertex A in G, the set of all possible orderings in which the vertices of G can be visited in a Breadth First Search (BFS) starting from A is denoted by $\mathcal{B}(A)$.

If $\emptyset$ denotes the empty set, then the cardinality of $\mathcal{B}(\emptyset)$ is _____.

gatecse-2023 algorithms breadth-first-search numerical-answers two-marks graph-search

Answer key

1.18.16 Graph Search: GATE CSE 2024 | Set 1 | Question: 35

Let G be a directed graph and T a depth first search (DFS) spanning tree in G that is rooted at a vertex v. Suppose T is also a breadth first search (BFS) tree in G, rooted at v. Which of the following statements is/are TRUE for *every* such graph G and tree T?

A. There are no back-edges in G with respect to the tree T
B. There are no cross-edges in G with respect to the tree T
C. There are no forward-edge in G with respect to the tree T
D. The only edges in G are the edges in T

gatecse-2024-set1 algorithms multiple-selects graph-search two-marks

Answer key

1.18.17 Graph Search: GATE CSE 2024 | Set 1 | Question: 50

The number of edges present in the forest generated by the DFS traversal of an undirected graph G with 100 vertices is 40. The number of connected components in G is _____.

gatecse-2024-set1 numerical-answers graph-search graph-algorithms two-marks

Answer key

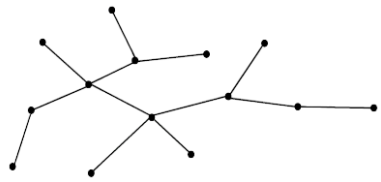
1.18.18 Graph Search: GATE CSE 2025 | Set 2 | Question: 49

Consider the following algorithm **someAlgo** that takes an undirected graph G as input.

someAlgo (G)

1. Let v be any vertex in G. Run BFS on G starting at v. Let u be a vertex in G at maximum distance from v as given by the BFS.
2. Run BFS on G again with u as the starting vertex. Let z be the vertex at maximum distance from u as given by the BFS.
3. Output the distance between u and z in G.

The output of **someAlgo** (T) for the tree shown in the given figure is _____. (Answer in integer)



gatecse2025-set2 algorithms breadth-first-search graph-search numerical-answers two-marks

Answer key

1.18.19 Graph Search: GATE DS&AI 2024 | Question: 4

Consider performing depth-first search (DFS) on an undirected and unweighted graph G starting at vertex s. For any vertex u in $G, d[u]$ is the length of the shortest path from s to u. Let (u, v) be an edge in G such that $d[u] < d[v]$. If the edge (u, v) is explored first in the direction from u to v during the above DFS, then (u, v) becomes a _____ edge.

A. tree B. cross C. back D. gray

gate-ds-ai-2024 graph-search depth-first-search algorithms one-mark

Answer key

1.18.20 Graph Search: GATE IT 2005 | Question: 14

In a depth-first traversal of a graph G with n vertices, k edges are marked as tree edges. The number of connected components in G is

A. k B. $k+1$ C. $n-k-1$ D. $n-k$

gateit-2005 algorithms graph-algorithms normal graph-search

Answer key

1.18.21 Graph Search: GATE IT 2006 | Question: 47

Consider the depth-first-search of an undirected graph with 3 vertices P, Q, and R. Let discovery time $d(u)$ represent the time instant when the vertex u is first visited, and finish time $f(u)$ represent the time instant when the vertex u is last visited. Given that

$d(P) = 5$ units	$f(P) = 12$ units
$d(Q) = 6$ units	$f(Q) = 10$ units
$d(R) = 14$ unit	$f(R) = 18$ units

Which one of the following statements is TRUE about the graph?

A. There is only one connected component
B. There are two connected components, and P and R are connected
C. There are two connected components, and Q and R are connected
D. There are two connected components, and P and Q are connected

gateit-2006 algorithms graph-algorithms normal graph-search depth-first-search

Answer key

1.18.22 Graph Search: GATE IT 2007 | Question: 24

A depth-first search is performed on a directed acyclic graph. Let $d[u]$ denote the time at which vertex u is visited for the first time and $f[u]$ the time at which the DFS call to the vertex u terminates. Which of the following statements is always TRUE for all edges (u, v) in the graph ?

A. $d[u] < d[v]$
B. $d[u] < f[v]$
C. $f[u] < f[v]$
D. $f[u] > f[v]$

gateit-2007 algorithms graph-algorithms normal graph-search depth-first-search

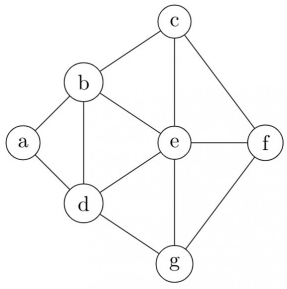
Answer key

1.18.23 Graph Search: GATE IT 2008 | Question: 47

Consider the following sequence of nodes for the undirected graph given below:

1. $a\,b\,e\,f\,d\,g\,c$
2. $a\,b\,e\,f\,c\,g\,d$
3. $a\,d\,g\,e\,b\,c\,f$
4. $a\,d\,b\,c\,g\,e\,f$

A Depth First Search (DFS) is started at node a. The nodes are listed in the order they are first visited. Which of the above is/are possible output(s)?



A. 1 and 3 only
B. 2 and 3 only
C. $2,3$ and 4 only
D. $1,2$ and 3 only

gateit-2008 algorithms graph-algorithms normal graph-search depth-first-search

Answer key

1.19 Greedy Algorithms (5)

✍ **Practice Test:** Test 1 (7Q)

1.19.1 Greedy Algorithms: GATE CSE 1999 | Question: 2.20

The minimum number of record movements required to merge five files A (with 10 records), B (with 20 records), C (with 15 records), D (with 5 records) and E (with 25 records) is:

A. 165
B. 90
C. 75
D. 65

gate1999 algorithms normal greedy-algorithms

Answer key

1.19.2 Greedy Algorithms: GATE CSE 2003 | Question: 69

The following are the starting and ending times of activities A, B, C, D, E, F, G and H respectively in chronological order: "$a_s\ b_s\ c_s\ a_e\ d_s\ c_e\ e_s\ f_s\ b_e\ d_e\ g_s\ e_e\ f_e\ h_s\ g_e\ h_e$". Here, x_s denotes the starting time and x_e denotes the ending time of activity X. We need to schedule the activities in a set of rooms available to us. An activity can be scheduled in a room only if the room is reserved for the activity for its entire duration. What is the minimum number of rooms required?

A. 3 B. 4 C. 5 D. 6

gatecse-2003 algorithms normal greedy-algorithms

Answer key

1.19.3 Greedy Algorithms: GATE CSE 2005 | Question: 84a

We are given 9 tasks $T_1, T_2, \ldots, T_9$. The execution of each task requires one unit of time. We can execute one task at a time. Each task T_i has a profit P_i and a deadline d_i. Profit P_i is earned if the task is completed before the end of the d_i^{th} unit of time.

Task	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9
Profit	15	20	30	18	18	10	23	16	25
Deadline	7	2	5	3	4	5	2	7	3

Are all tasks completed in the schedule that gives maximum profit?

A. All tasks are completed
B. T_1 and T_6 are left out
C. T_1 and T_8 are left out
D. T_4 and T_6 are left out

gatecse-2005 algorithms greedy-algorithms process-scheduling normal

Answer key

1.19.4 Greedy Algorithms: GATE CSE 2005 | Question: 84b

We are given 9 tasks $T_1, T_2, \ldots, T_9$. The execution of each task requires one unit of time. We can execute one task at a time. Each task T_i has a profit P_i and a deadline d_i. Profit P_i is earned if the task is completed before the end of the d_i^{th} unit of time.

Task	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9
Profit	15	20	30	18	18	10	23	16	25
Deadline	7	2	5	3	4	5	2	7	3

What is the maximum profit earned?

A. 147 B. 165 C. 167 D. 175

gatecse-2005 algorithms greedy-algorithms process-scheduling normal

Answer key

1.19.5 Greedy Algorithms: GATE CSE 2018 | Question: 48

Consider the weights and values of items listed below. Note that there is only one unit of each item.

Item number	**Weight (in Kgs)**	**Value (in rupees)**
1	10	60
2	7	28
3	4	20
4	2	24

The task is to pick a subset of these items such that their total weight is no more than 11 Kgs and their total value is maximized. Moreover, no item may be split. The total value of items picked by an optimal algorithm is denoted by V_{opt}. A greedy algorithm sorts the items by their value-to-weight ratios in descending order and packs them greedily, starting from the first item in the ordered list. The total value of items picked by the greedy algorithm is denoted by V_{greedy}.

The value of $V_{opt} - V_{greedy}$ is ____

gatecse-2018 algorithms greedy-algorithms numerical-answers two-marks

Answer key

1.20 Hashing (6)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (3Q)

1.20.1 Hashing: GATE CSE 1989 | Question: 1-vii, ISRO2015-14

A hash table with ten buckets with one slot per bucket is shown in the following figure. The symbols $S1$ to $S7$ initially entered using a hashing function with linear probing. The maximum number of comparisons needed in searching an item that is not present is

0	S7
1	S1
2	
3	S4
4	S2
5	
6	S5
7	
8	S6
9	S3

A. 4 B. 5 C. 6 D. 3

hashing isro2015 gate1989 algorithms normal

Answer key

1.20.2 Hashing: GATE CSE 1990 | Question: 13b

Consider a hash table with chaining scheme for overflow handling:

i. What is the worst-case timing complexity of inserting n elements into such a table?
ii. For what type of instance does this hashing scheme take the worst-case time for insertion?

gate1990 hashing algorithms descriptive

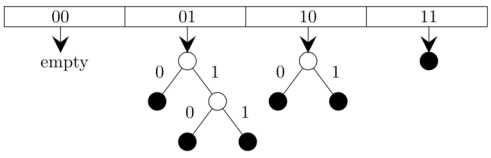
Answer key

1.20.3 Hashing: GATE CSE 2021 | Set 1 | Question: 47

Consider a $dynamic$ hashing approach for 4-bit integer keys:

1. There is a main hash table of size 4.
2. The 2 least significant bits of a key is used to index into the main hash table.
3. Initially, the main hash table entries are empty.
4. Thereafter, when more keys are hashed into it, to resolve collisions, the set of all keys corresponding to a main hash table entry is organized as a binary tree that grows on demand.
5. First, the 3^{rd} least significant bit is used to divide the keys into left and right subtrees.
6. To resolve more collisions, each node of the binary tree is further sub-divided into left and right subtrees based on the 4^{th} least significant bit.
7. A split is done only if it is needed, i.e., only when there is a collision.

Consider the following state of the hash table.



Which of the following sequences of key insertions can cause the above state of the hash table (assume the keys are in decimal notation)?

A. $5,9,4,13,10,7$ B. $9,5,10,6,7,1$

C. $10,9,6,7,5,13$ D. $9,5,13,6,10,14$

gatecse-2021-set1 multiple-selects algorithms hashing two-marks

Answer key

1.20.4 Hashing: GATE CSE 2023 | Question: 10

An algorithm has to store several keys generated by an adversary in a hash table. The adversary is malicious who tries to maximize the number of collisions. Let k be the number of keys, m be the number of slots in the hash table, and $k > m$.

Which one of the following is the best hashing strategy to counteract the adversary?

A. Division method, i.e., use the hash function $h(k) = k \bmod m$.
B. Multiplication method, i.e., use the hash function $h(k) = \lfloor m(kA - \lfloor kA \rfloor) \rfloor$, where A is a carefully chosen constant.
C. Universal hashing method.
D. If k is a prime number, use Division method. Otherwise, use Multiplication method.

gatecse-2023 algorithms hashing one-mark

Answer key

1.20.5 Hashing: GATE CSE 2026 | Set 2 | Question: 20

The keys $5, 28, 19, 15, 26, 33, 12, 17, 10$ are inserted into a hash table using the hash function $h(k) = k \bmod 9$. The collisions are resolved by chaining. After all the keys are inserted, the length of the longest chain is _____. (answer in integer)

gatecse-2026-set2 algorithms hashing numerical-answers one-mark

Answer key

1.20.6 Hashing: GATE IT 2005 | Question: 16

A hash table contains 10 buckets and uses linear probing to resolve collisions. The key values are integers and the hash function used is $\text{key}\%10$. If the values $43, 165, 62, 123, 142$ are inserted in the table, in what location would the key value 142 be inserted?

A. 2 B. 3 C. 4 D. 6

gateit-2005 algorithms hashing easy

Answer key

1.21 Heap Sort (2)

1.21.1 Heap Sort: GATE CSE 2013 | Question: 30

The number of elements that can be sorted in $\Theta(\log n)$ time using heap sort is

A. $\Theta(1)$
B. $\Theta(\sqrt{\log n})$
C. $\Theta(\frac{\log n}{\log \log n})$
D. $\Theta(\log n)$

gatecse-2013 algorithms sorting normal heap-sort

Answer key

1.21.2 Heap Sort: GATE CSE 2024 | Set 1 | Question: 31

An array $[82, 101, 90, 11, 111, 75, 33, 131, 44, 93]$ is heapified. Which one of the following options represents the first three elements in the heapified array?

A. $82, 90, 101$ B. $82, 11, 93$ C. $131, 11, 93$ D. $131, 111, 90$

gatecse-2024-set1 algorithms heap-sort sorting two-marks

Answer key

1.22 Huffman Code (6)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (2Q)

1.22.1 Huffman Code: GATE CSE 1989 | Question: 13a

A language uses an alphabet of six letters, $\{a, b, c, d, e, f\}$. The relative frequency of use of each letter of the alphabet in the language is as given below:

LETTER	RELATIVE FREQUENCY OF USE
a	0.19
b	0.05
c	0.17
d	0.08
e	0.40
f	0.11

Design a prefix binary code for the language which would minimize the average length of the encoded words of the language.

descriptive gate1989 algorithms huffman-code

Answer key

1.22.2 Huffman Code: GATE CSE 2007 | Question: 76

Suppose the letters a, b, c, d, e, f have probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{32}$, respectively.

Which of the following is the Huffman code for the letter a, b, c, d, e, f?

A. $0, 10, 110, 1110, 11110, 11111$
B. $11, 10, 011, 010, 001, 000$
C. $11, 10, 01, 001, 0001, 0000$
D. $110, 100, 010, 000, 001, 111$

gatecse-2007 algorithms greedy-algorithms normal huffman-code

Answer key

1.22.3 Huffman Code: GATE CSE 2007 | Question: 77

Suppose the letters a, b, c, d, e, f have probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{32}$, respectively.

What is the average length of the Huffman code for the letters a, b, c, d, e, f?

A. 3
B. 2.1875
C. 2.25
D. 1.9375

gatecse-2007 algorithms greedy-algorithms normal huffman-code

Answer key

1.22.4 Huffman Code: GATE CSE 2017 | Set 2 | Question: 50

A message is made up entirely of characters from the set $X = \{P, Q, R, S, T\}$. The table of probabilities for each of the characters is shown below:

Character	**Probability**
P	0.22
Q	0.34
R	0.17
S	0.19
T	0.08
Total	1.00

If a message of 100 characters over X is encoded using Huffman coding, then the expected length of the encoded

message in bits is _____.

gatecse-2017-set2 huffman-code numerical-answers algorithms

Answer key

1.22.5 Huffman Code: GATE CSE 2021 | Set 2 | Question: 26

Consider the string $\mathrm{abbccddeee}$. Each letter in the string must be assigned a binary code satisfying the following properties:

1. For any two letters, the code assigned to one letter must not be a prefix of the code assigned to the other letter.
2. For any two letters of the same frequency, the letter which occurs earlier in the dictionary order is assigned a code whose length is at most the length of the code assigned to the other letter.

Among the set of all binary code assignments which satisfy the above two properties, what is the minimum length of the encoded string?

A. 21 B. 23 C. 25 D. 30

gatecse-2021-set2 algorithms huffman-code two-marks

Answer key

1.22.6 Huffman Code: GATE IT 2006 | Question: 48

The characters a to h have the set of frequencies based on the first 8 Fibonacci numbers as follows

$a : 1, b : 1, c : 2, d : 3, e : 5, f : 8, g : 13, h : 21$

A Huffman code is used to represent the characters. What is the sequence of characters corresponding to the following code?

110111100111010

A. $fdheg$ B. $ecgdf$ C. $dchfg$ D. $fehdg$

gateit-2006 algorithms greedy-algorithms normal huffman-code

Answer key

1.23 Identify Function (38)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (9Q)

1.23.1 Identify Function: GATE CSE 1989 | Question: 8a

What is the output produced by the following program, when the input is "HTGATE"

```
Function what (s:string): string;
var n:integer;
begin
   n = s.length
   if n <= 1
   then what := s
   else what :=contact (what (substring (s, 2, n)), s.C [1])
end;
```

Note

i. type string=record
   length:integer;
   C:array[1..100] of char
   end

ii. Substring (s, i, j): this yields the string made up of the i^{th} through j^{th} characters in s; for appropriately defined in i and j.

iii. Contact (s_1, s_2): this function yields a string of length s_1 length + s_2 - length obtained by concatenating s_1 with s_2 such that s_1 precedes s_2.

gate1989 descriptive algorithms identify-function

Answer key

1.23.2 Identify Function: GATE CSE 1990 | Question: 11b

The following program computes values of a mathematical function $f(x)$. Determine the form of $f(x)$.

```
main ()
{
   int m, n; float x, y, t;
   scanf ("%f%d", &x, &n);
   t = 1; y = 0; m = 1;
   do
   {
      t *= (-x/m);
      y += t;
   } while (m++ < n);
   printf ("The value of y is %f", y);
}
```

gate1990 descriptive algorithms identify-function

Answer key

1.23.3 Identify Function: GATE CSE 1991 | Question: 03-viii

Consider the following Pascal function:

```
Function X(M:integer):integer;
Var i:integer;
Begin
   i := 0;
   while i*i < M
   do i:= i+1
   X := i
end
```

The function call $X(N)$, if N is positive, will return

A. $\lfloor \sqrt{N} \rfloor$
B. $\lfloor \sqrt{N} \rfloor + 1$
C. $\lceil \sqrt{N} \rceil$
D. $\lceil \sqrt{N} \rceil + 1$
E. None of the above

gate1991 algorithms easy identify-function multiple-selects

Answer key

1.23.4 Identify Function: GATE CSE 1993 | Question: 7.4

What does the following code do?

```
var a, b: integer;
begin
   a:=a+b;
   b:=a-b;
   a:=a-b;
end;
```

A. exchanges a and b
B. doubles a and stores in b
C. doubles b and stores in a
D. leaves a and b unchanged
E. none of the above

gate1993 algorithms identify-function easy

Answer key

1.23.5 Identify Function: GATE CSE 1994 | Question: 6

What function of x, n is computed by this program?

```
Function what(x, n:integer): integer:
Var
   value : integer
begin
   value := 1
   if n > 0 then
```

```
  begin
    if n mod 2 =1 then
       value := value * x;
    value := value * what(x*x, n div 2);
  end;
  what := value;
end;
```

gate1994 algorithms identify-function normal descriptive

Answer key

1.23.6 Identify Function: GATE CSE 1995 | Question: 1.4

In the following Pascal program segment, what is the value of X after the execution of the program segment?

```
X := -10; Y := 20;
If X > Y then if X < 0 then X := abs(X) else X := 2*X;
```

A. 10 B. -20 C. -10 D. None

gate1995 algorithms identify-function easy

Answer key

1.23.7 Identify Function: GATE CSE 1995 | Question: 2.3

Assume that X and Y are non-zero positive integers. What does the following Pascal program segment do?

```
while X <> Y do
if  X > Y then
   X := X - Y
else
   Y := Y - X;
write(X);
```

A. Computes the LCM of two numbers
B. Divides the larger number by the smaller number
C. Computes the GCD of two numbers
D. None of the above

gate1995 algorithms identify-function normal

Answer key

1.23.8 Identify Function: GATE CSE 1995 | Question: 4

A. Consider the following Pascal function where A and B are non-zero positive integers. What is the value of $\mathrm{GET}(3, 2)$?

```
function GET(A,B:integer): integer;
begin
   if B=0 then
      GET:= 1
   else if A < B then
      GET:= 0
   else
      GET:= GET(A-1, B) + GET(A-1, B-1)
end;
```

B. The Pascal procedure given for computing the transpose of an $N \times N, (N > 1)$ matrix A of integers has an error. Find the error and correct it. Assume that the following declaration are made in the main program

```
const
   MAXSIZE=20;
type
   INTARR=array [1..MAXSIZE,1..MAXSIZE] of integer;
Procedure TRANSPOSE (var A: INTARR; N : integer);
var
   I, J, TMP: integer;
begin
   for I:=1 to N – 1 do
   for J:=1 to N do
   begin
      TMP:= A[I, J];
      A[I, J]:= A[J, I];
```

```
        A[J, I]:= TMP
    end
end;
```

gate1995 algorithms identify-function normal descriptive

Answer key

1.23.9 Identify Function: GATE CSE 1998 | Question: 2.12

What value would the following function return for the input $x = 95$?

```
Function fun (x:integer):integer;
Begin
   If x > 100 then fun = x – 10
   Else fun = fun(fun (x+11))
End;
```

A. 89 B. 90 C. 91 D. 92

gate1998 algorithms recursion identify-function normal

Answer key

1.23.10 Identify Function: GATE CSE 1999 | Question: 2.24

Consider the following C function definition

```
int Trial (int a, int b, int c)
{
   if ((a>=b) && (c<b)) return b;
   else if (a>=b) return Trial(a, c, b);
   else return Trial(b, a, c);
}
```

The functional Trial:

A. Finds the maximum of a, b, and c
B. Finds the minimum of a, b, and c
C. Finds the middle number of a, b, c
D. None of the above

gate1999 algorithms identify-function normal

Answer key

1.23.11 Identify Function: GATE CSE 2000 | Question: 2.15

Suppose you are given an array $s[1 \ldots n]$ and a procedure reverse (s, i, j) which reverses the order of elements in s between positions i and j (both inclusive). What does the following sequence do, where $1 \leqslant k \leqslant n$:

```
        reverse (s, 1, k);
        reverse (s, k+1, n);
        reverse (s, 1, n);
```

A. Rotates s left by k positions
B. Leaves s unchanged
C. Reverses all elements of s
D. None of the above

gatecse-2000 algorithms normal identify-function

Answer key

1.23.12 Identify Function: GATE CSE 2003 | Question: 1

Consider the following C function.

For large values of y, the return value of the function f best approximates

```
float f,(float x, int y) {
   float p, s; int i;
   for (s=1,p=1,i=1; i<y; i++) {
      p *= x/i;
      s += p;
   }
   return s;
}
```

A. x^y B. e^x C. $\ln(1+x)$ D. x^x

gatecse-2003 algorithms identify-function normal

Answer key

1.23.13 Identify Function: GATE CSE 2003 | Question: 88

In the following C program fragment, j, k, n and TwoLog_n are integer variables, and A is an array of integers. The variable n is initialized to an integer $\geqslant 3$, and TwoLog_n is initialized to the value of $2^*\lceil \log_2(n) \rceil$

```
for (k = 3;  k <= n; k++)
      A[k] = 0;
for (k = 2; k <= TwoLog_n; k++)
   for (j = k+1; j <= n; j++)
      A[j] = A[j] || (j%k);
for (j = 3; j <= n; j++)
   if (!A[j]) printf("%d", j);
```

The set of numbers printed by this program fragment is

A. $\{m \mid m \leq n, (\exists i)\, [m = i!]\}$ B. $\{m \mid m \leq n, (\exists i)\, [m = i^2]\}$

C. $\{m \mid m \leq n, \text{m is prime}\}$ D. $\{\,\}$

gatecse-2003 algorithms identify-function normal

Answer key

1.23.14 Identify Function: GATE CSE 2004 | Question: 41

Consider the following C program

```
main()
{
   int x, y, m, n;
   scanf("%d %d", &x, &y);
   /* Assume x>0 and y>0*/
   m = x; n = y;
   while(m != n)
      {
         if (m > n)
            m = m-n;
         else
            n = n-m;
      }
   printf("%d", n);
}
```

The program computes

A. $x + y$ using repeated subtraction B. $x \mod y$ using repeated subtraction

C. the greatest common divisor of x and y D. the least common multiple of x and y

gatecse-2004 algorithms normal identify-function

Answer key

1.23.15 Identify Function: GATE CSE 2004 | Question: 42

What does the following algorithm approximate? (Assume $m > 1, \epsilon > 0$).

```
x = m;
y = 1;
While (x-y > ϵ)
{
   x = (x+y)/2;
   y = m/x;
}
print(x);
```

A. $\log m$ B. m^2 C. $m^{\frac{1}{2}}$ D. $m^{\frac{1}{3}}$

gatecse-2004 algorithms identify-function normal

Answer key

1.23.16 Identify Function: GATE CSE 2005 | Question: 31

Consider the following C-program:

```
void foo (int n, int sum) {
   int k = 0, j = 0;
   if (n == 0) return;
   k = n % 10; j = n/10;
   sum = sum + k;
   foo (j, sum);
   printf ("%d,",k);
}

int main() {
   int a = 2048, sum = 0;
   foo(a, sum);
   printf("%d\n", sum);
}
```

What does the above program print?

A. $8, 4, 0, 2, 14$
B. $8, 4, 0, 2, 0$
C. $2, 0, 4, 8, 14$
D. $2, 0, 4, 8, 0$

gatecse-2005 algorithms identify-function recursion normal

Answer key

1.23.17 Identify Function: GATE CSE 2006 | Question: 50

A set X can be represented by an array $x[n]$ as follows:

$$x[i] = \begin{cases} 1 & \text{if } i \in X \\ 0 & \text{otherwise} \end{cases}$$

Consider the following algorithm in which x, y, and z are Boolean arrays of size n:

```
algorithm zzz(x[], y[], z[]) {
int i;

for(i=0; i<n; ++i)
z[i] = (x[i] ∧ ~y[i]) ∨ (~x[i] ∧ y[i]);
}
```

The set Z computed by the algorithm is:

A. $(X \cup Y)$
B. $(X \cap Y)$
C. $(X - Y) \cap (Y - X)$
D. $(X - Y) \cup (Y - X)$

gatecse-2006 algorithms identify-function normal

Answer key

1.23.18 Identify Function: GATE CSE 2006 | Question: 53

Consider the following C-function in which $a[n]$ and $b[m]$ are two sorted integer arrays and $c[n+m]$ be another integer array,

```
void xyz(int a[], int b [], int c []){
   int i,j,k;
   i=j=k=0;
   while ((i<n) && (j<m))
      if (a[i] < b[j]) c[k++] = a[i++];
      else c[k++] = b[j++];
}
```

Which of the following condition(s) hold(s) after the termination of the while loop?

i. $j < m, k = n + j - 1$ and $a[n-1] < b[j]$ if $i = n$
ii. $i < n, k = m + i - 1$ and $b[m-1] \leq a[i]$ if $j = m$

A. only (i)
B. only (ii)
C. either (i) or (ii) but not both
D. neither (i) nor (ii)

gatecse-2006 algorithms identify-function normal

Answer key

1.23.19 Identify Function: GATE CSE 2009 | Question: 18

Consider the program below:

```
#include <stdio.h>
int fun(int n, int *f_p) {
   int t, f;
   if (n <= 1) {
      *f_p = 1;
      return 1;
   }
   t = fun(n-1, f_p);
   f = t + *f_p;
   *f_p = t;
   return f;
}

int main() {
   int x = 15;
   printf("%d/n", fun(5, &x));
   return 0;
}
```

The value printed is:

A. 6 B. 8 C. 14 D. 15

gatecse-2009 algorithms recursion identify-function normal

Answer key

1.23.20 Identify Function: GATE CSE 2010 | Question: 35

What is the value printed by the following C program?

```
#include<stdio.h>

int f(int *a, int n)
{
   if (n <= 0) return 0;
   else if (*a % 2 == 0) return *a+f(a+1, n-1);
   else return *a - f(a+1, n-1);
}

int main()
{
   int a[] = {12, 7, 13, 4, 11, 6};
   printf("%d", f(a, 6));
   return 0;
}
```

A. -9 B. 5 C. 15 D. 19

gatecse-2010 algorithms recursion identify-function normal

Answer key

1.23.21 Identify Function: GATE CSE 2011 | Question: 48

Consider the following recursive C function that takes two arguments.

```
unsigned int foo(unsigned int n, unsigned int r) {
   if (n>0) return ((n%r) + foo(n/r, r));
   else return 0;
}
```

What is the return value of the function foo when it is called as $\text{foo}(345, 10)$?

A. 345 B. 12 C. 5 D. 3

gatecse-2011 algorithms recursion identify-function normal

Answer key

1.23.22 Identify Function: GATE CSE 2011 | Question: 49

Consider the following recursive C function that takes two arguments.

```
unsigned int foo(unsigned int n, unsigned int r) {
  if (n>0) return ((n%r) + foo(n/r, r));
  else return 0;
}
```

What is the return value of the function foo when it is called as $\text{foo}(513, 2)$?

A. 9 B. 8 C. 5 D. 2

gatecse-2011 algorithms recursion identify-function normal

Answer key

1.23.23 Identify Function: GATE CSE 2013 | Question: 31

Consider the following function:

```
int unknown(int n){

int i, j, k=0;
for (i=n/2; i<=n; i++)
   for (j=2; j<=n; j=j*2)
      k = k + n/2;
 return (k);

}
```

The return value of the function is

A. $\Theta(n^2)$
B. $\Theta(n^2 \log n)$
C. $\Theta(n^3)$
D. $\Theta(n^3 \log n)$

gatecse-2013 algorithms identify-function normal

Answer key

1.23.24 Identify Function: GATE CSE 2014 | Set 1 | Question: 41

Consider the following C function in which **size** is the number of elements in the array **E**:

```
int MyX(int *E, unsigned int size)
{
  int Y = 0;
  int Z;
  int i, j, k;

  for(i = 0; i< size; i++)
       Y = Y + E[i];

   for(i=0; i < size; i++)
      for(j = i; j < size; j++)
      {
         Z = 0;
         for(k = i; k <= j; k++)
           Z = Z + E[k];
         if(Z > Y)
           Y = Z;
      }
  return Y;
}
```

The value returned by the function **MyX** is the

A. maximum possible sum of elements in any sub-array of array **E**.
B. maximum element in any sub-array of array **E.**
C. sum of the maximum elements in all possible sub-arrays of array **E**.
D. the sum of all the elements in the array **E.**

gatecse-2014-set1 algorithms identify-function normal

Answer key

1.23.25 Identify Function: GATE CSE 2014 | Set 2 | Question: 10

Consider the function func shown below:

```
int func(int num) {
  int count = 0;
  while (num) {
    count++;
    num>>= 1;
  }
  return (count);
}
```

The value returned by func(435) is _____

gatecse-2014-set2 algorithms identify-function numerical-answers easy

Answer key

1.23.26 Identify Function: GATE CSE 2014 | Set 3 | Question: 10

Let A be the square matrix of size $n \times n$. Consider the following pseudocode. What is the expected output?

```
C=100;
for i=1 to n do
   for j=1 to n do
   {
      Temp = A[i][j]+C;
      A[i][j] = A[j][i];
      A[j][i] = Temp -C;
   }
for i=1 to n do
   for j=1 to n do
      output (A[i][j]);
```

A. The matrix A itself
B. Transpose of the matrix A
C. Adding 100 to the upper diagonal elements and subtracting 100 from lower diagonal elements of A
D. None of the above

gatecse-2014-set3 algorithms identify-function easy

Answer key

1.23.27 Identify Function: GATE CSE 2015 | Set 1 | Question: 31

Consider the following C function.

```
int fun1 (int n) {
    int i, j, k, p, q = 0;
    for (i = 1; i < n; ++i)
    {
      p = 0;
     for (j = n; j > 1; j = j/2)
        ++p;
     for (k = 1; k < p; k = k * 2)
        ++q;
    }
    return q;
}
```

Which one of the following most closely approximates the return value of the function fun1?

A. n^3 B. $n(\log n)^2$ C. $n \log n$ D. $n \log(\log n)$

gatecse-2015-set1 algorithms normal identify-function

Answer key

1.23.28 Identify Function: GATE CSE 2015 | Set 2 | Question: 11

Consider the following C function.

```
int fun(int n) {
```

```
int x=1, k;
if (n==1) return x;
for (k=1; k<n; ++k)
    x = x + fun(k) * fun (n-k);
return x;
}
```

The return value of $fun(5)$ is _____.

gatecse-2015-set2 algorithms identify-function recurrence-relation normal numerical-answers

Answer key

1.23.29 Identify Function: GATE CSE 2015 | Set 3 | Question: 49

Suppose $c = \langle c[0], \ldots, c[k-1] \rangle$ is an array of length k, where all the entries are from the set $\{0, 1\}$. For any positive integers a and n, consider the following pseudocode.

```
DOSOMETHING (c, a, n)

z ← 1
for i ← 0 to k-1
    do z ← z² mod n
    if c[i]=1
        then z ← (z × a) mod n
return z
```

If $k = 4, c = \langle 1, 0, 1, 1 \rangle, a = 2,$ and $n = 8$, then the output of DOSOMETHING(*c, a, n)* is _____.

gatecse-2015-set3 algorithms identify-function normal numerical-answers

Answer key

1.23.30 Identify Function: GATE CSE 2019 | Question: 26

Consider the following C function.

```
void convert (int n ) {
    if (n<0)
        printf{"%d", n);
    else {
        convert(n/2);
        printf("%d", n%2);
    }
}
```

Which one of the following will happen when the function *convert* is called with any positive integer n as argument?

A. It will print the binary representation of n and terminate
B. It will print the binary representation of n in the reverse order and terminate
C. It will print the binary representation of n but will not terminate
D. It will not print anything and will not terminate

gatecse-2019 algorithms identify-function two-marks

Answer key

1.23.31 Identify Function: GATE CSE 2020 | Question: 48

Consider the following C functions.

```
int tob (int b, int* arr) {
  int i;
  for (i = 0; b>0; i++)  {
    if (b%2)  arr [i] = 1;
    else      arr[i] = 0;
    b = b/2;
  }
  return (i);
}
```

```
int pp(int a, int b)  {
  int  arr[20];
  int i, tot = 1, ex, len;
  ex = a;
  len = tob(b, arr);
  for (i=0; i<len ; i++) {
      if (arr[i] ==1)
          tot = tot * ex;
      ex= ex*ex;
  }
return (tot) ;
}
```

The value returned by $pp(3, 4)$ is _____.

gatecse-2020 numerical-answers identify-function two-marks

Answer key

1.23.32 Identify Function: GATE CSE 2021 | Set 1 | Question: 48

Consider the following ANSI C function:

```
int SimpleFunction(int Y[], int n, int x)
{
  int total = Y[0], loopIndex;
  for (loopIndex=1; loopIndex<=n-1; loopIndex++)
      total=x*total +Y[loopIndex];
  return total;
}
```

Let Z be an array of 10 elements with $\mathsf{Z}[i] = 1$, for all i such that $0 \leq i \leq 9$. The value returned by $\mathsf{SimpleFunction}(\mathsf{Z}, 10, 2)$ is _____

gatecse-2021-set1 algorithms numerical-answers identify-function two-marks

Answer key

1.23.33 Identify Function: GATE CSE 2021 | Set 2 | Question: 23

Consider the following ANSI C function:

```
int SomeFunction (int x, int y)
{
  if ((x==1) || (y==1)) return 1;
  if (x==y) return x;
  if (x > y) return SomeFunction(x-y, y);
  if (y > x) return SomeFunction (x, y-x);

}
```

The value returned by $\text{SomeFunction}(15, 255)$ is _____

gatecse-2021-set2 numerical-answers algorithms identify-function output one-mark

Answer key

1.23.34 Identify Function: GATE IT 2005 | Question: 53

The following$C$ function takes two ASCII strings and determines whether one is an anagram of the other. An anagram of a string s is a string obtained by permuting the letters in s.

```
int anagram (char *a, char *b) {
  int count [128], j;
  for (j = 0;  j < 128; j++) count[j] = 0;
  j = 0;
  while (a[j] && b[j]) {
    A;
    B;
  }
  for (j = 0; j < 128; j++) if (count [j]) return 0;
  return 1;
}
```

Choose the correct alternative for statements A and B.

A. A: count [a[j]]++ and B: count[b[j]]--
B. A: count [a[j]]++ and B: count[b[j]]++
C. A: count [a[j++]]++ and B: count[b[j]]--
D. A: count [a[j]]++ and B: count[b[j++]]--

gateit-2005 normal identify-function

Answer key

1.23.35 Identify Function: GATE IT 2005 | Question: 57

What is the output printed by the following program?

```
#include <stdio.h>

int f(int n, int k) {
    if (n == 0) return 0;
    else if (n % 2) return f(n/2, 2*k) + k;
    else return f(n/2, 2*k) - k;
}

int main () {
    printf("%d", f(20, 1));
    return 0;
}
```

A. 5 B. 8 C. 9 D. 20

gateit-2005 algorithms identify-function normal

Answer key

1.23.36 Identify Function: GATE IT 2006 | Question: 52

The following function computes the value of $\binom{m}{n}$ correctly for all legal values m and n ($m \geq 1, n \geq 0$ and $m > n$)

```
int func(int m, int n)
{
    if (E) return 1;
    else return(func(m -1, n) + func(m - 1, n - 1));
}
```

In the above function, which of the following is the correct expression for E?

A. $(n == 0)||(m == 1)$
B. $(n == 0)$ && $(m == 1)$
C. $(n == 0)||(m == n)$
D. $(n == 0)$ && $(m == n)$

gateit-2006 algorithms identify-function normal

Answer key

1.23.37 Identify Function: GATE IT 2008 | Question: 82

Consider the code fragment written in C below :

```
void f (int n)
{
  if (n <=1)  {
    printf ("%d", n);
  }
  else {
    f (n/2);
    printf ("%d", n%2);
  }
}
```

What does f(173) print?

A. 010110101 B. 010101101 C. 10110101 D. 10101101

gateit-2008 algorithms recursion identify-function normal

Answer key

1.23.38 Identify Function: GATE IT 2008 | Question: 83

Consider the code fragment written in C below :

```
void f (int n)
{
   if (n <= 1)  {
      printf ("%d", n);
   }
   else {
      f (n/2);
      printf ("%d", n%2);
   }
}
```

Which of the following implementations will produce the same output for $f(173)$ as the above code?

P1

```
void f (int n)
{
   if (n/2)  {
      f(n/2);
   }
   printf ("%d", n%2);
}
```

P2

```
void f (int n)
{
   if (n <=1)  {
      printf ("%d", n);
   }
   else {
      printf ("%d", n%2);
      f (n/2);
   }
}
```

A. Both $P1$ and $P2$ B. $P2$ only C. $P1$ only D. Neither $P1$ nor $P2$

gateit-2008 algorithms recursion identify-function normal

Answer key

1.24 Insertion Sort (2)

1.24.1 Insertion Sort: GATE CSE 2003 | Question: 22

The usual $\Theta(n^2)$ implementation of Insertion Sort to sort an array uses linear search to identify the position where an element is to be inserted into the already sorted part of the array. If, instead, we use binary search to identify the position, the worst case running time will

A. remain $\Theta(n^2)$
B. become $\Theta(n(\log n)^2)$
C. become $\Theta(n \log n)$
D. become $\Theta(n)$

gatecse-2003 algorithms sorting time-complexity normal insertion-sort

Answer key

1.24.2 Insertion Sort: GATE CSE 2003 | Question: 62

In a permutation $a_1 \ldots a_n$, of n distinct integers, an inversion is a pair (a_i, a_j) such that $i < j$ and $a_i > a_j$.

What would be the worst case time complexity of the Insertion Sort algorithm, if the inputs are restricted to permutations of $1 \ldots n$ with at most n inversions?

A. $\Theta(n^2)$ B. $\Theta(n \log n)$ C. $\Theta(n^{1.5})$ D. $\Theta(n)$

gatecse-2003 algorithms sorting normal insertion-sort

Answer key

1.25 Inversion (2)

1.25.1 Inversion: GATE CSE 2003 | Question: 61

In a permutation $a_1 \ldots a_n$, of n distinct integers, an inversion is a pair (a_i, a_j) such that $i < j$ and $a_i > a_j$.

If all permutations are equally likely, what is the expected number of inversions in a randomly chosen permutation of $1 \ldots n$?

A. $\frac{n(n-1)}{2}$ B. $\frac{n(n-1)}{4}$ C. $\frac{n(n+1)}{4}$ D. $2n[\log_2 n]$

gatecse-2003 algorithms sorting inversion normal

Answer key

1.25.2 Inversion: GATE DA 2025 | Question: 19

Suppose that insertion sort is applied to the array $[1, 3, 5, 7, 9, 11, x, 15, 13]$ and it takes exactly two swaps to sort the array. Select all possible values of x.

A. 10 B. 12 C. 14 D. 16

gateda-2025 algorithms insertion-sort sorting multiple-selects easy one-mark inversion

Answer key

1.26 Linear Probing (2)

✍ **Practice Test:** Test 1 (5Q)

1.26.1 Linear Probing: GATE CSE 2026 | Set 1 | Question: 14

Consider a hash table $P[0, 1, \ldots, 10]$ that is initially empty. The hash table is maintained using open addressing with linear probing. The hash function used is $h(x) = (x + 7) \bmod 11$.

Consider the following sequence of insertions performed on P :

$$1,13, 22, 15, 11, 24$$

Which of the following positions in the hash table is/are empty after these insertions are performed?

A. 0 B. 10 C. 2 D. 1

gatecse-2026-set1 algorithms hashing linear-probing multiple-selects one-mark

Answer key

1.26.2 Linear Probing: GATE DA 2025 | Question: 8

Consider a hash table of size 10 with indices $\{0, 1, \ldots, 9\}$, with the hash function

$$h(x) = 3x(\bmod 10)$$

where linear probing is used to handle collisions. The hash table is initially empty and then the following sequence of keys is inserted into the hash table: $1, 4, 5, 6, 14, 15$. The indices where the keys 14 and 15 are stored are, respectively

A. 2 and 5 B. 2 and 6 C. 4 and 5 D. 4 and 6

gateda-2025 algorithms hashing linear-probing easy one-mark

Answer key

1.27 Matrix Chain Ordering (3)

✍ **Practice Test:** Test 1 (5Q)

1.27.1 Matrix Chain Ordering: GATE CSE 2011 | Question: 38

Four Matrices M_1, M_2, M_3 and M_4 of dimensions $p \times q$, $q \times r$, $r \times s$ and $s \times t$ respectively can be multiplied in several ways with different number of total scalar multiplications. For example when multiplied as $((M_1 \times M_2) \times (M_3 \times M_4))$, the total number of scalar multiplications is $pqr + rst + prt$. When multiplied as $(((M_1 \times M_2) \times M_3) \times M_4)$, the total number of scalar multiplications is $pqr + prs + pst$.

If $p = 10, q = 100, r = 20, s = 5$ and $t = 80$, then the minimum number of scalar multiplications needed is

A. 248000 B. 44000 C. 19000 D. 25000

gatecse-2011 algorithms dynamic-programming normal matrix-chain-ordering

Answer key

1.27.2 Matrix Chain Ordering: GATE CSE 2016 | Set 2 | Question: 38

Let A_1, A_2, A_3 and A_4 be four matrices of dimensions $10 \times 5, 5 \times 20, 20 \times 10$ and 10×5, respectively. The minimum number of scalar multiplications required to find the product $A_1A_2A_3A_4$ using the basic matrix multiplication method is _____.

gatecse-2016-set2 dynamic-programming algorithms matrix-chain-ordering normal numerical-answers

Answer key

1.27.3 Matrix Chain Ordering: GATE CSE 2018 | Question: 31

Assume that multiplying a matrix G_1 of dimension $p \times q$ with another matrix G_2 of dimension $q \times r$ requires pqr scalar multiplications. Computing the product of n matrices $G_1G_2G_3 \ldots G_n$ can be done by parenthesizing in different ways. Define G_iG_{i+1} as an **explicitly computed pair** for a given paranthesization if they are directly multiplied. For example, in the matrix multiplication chain $G_1G_2G_3G_4G_5G_6$ using parenthesization $(G_1(G_2G_3))(G_4(G_5G_6)), G_2G_3$ and G_5G_6 are only explicitly computed pairs.

Consider a matrix multiplication chain $F_1F_2F_3F_4F_5$, where matrices F_1, F_2, F_3, F_4 and F_5 are of dimensions $2 \times 25, 25 \times 3, 3 \times 16, 16 \times 1$ and 1×1000, respectively. In the parenthesization of $F_1F_2F_3F_4F_5$ that minimizes the total number of scalar multiplications, the explicitly computed pairs is/are

A. F_1F_2 and F_3F_4 only
B. F_2F_3 only
C. F_3F_4 only
D. F_1F_2 and F_4F_5 only

gatecse-2018 algorithms dynamic-programming two-marks matrix-chain-ordering

Answer key

1.28 Maximum Minimum (1)

1.28.1 Maximum Minimum: GATE CSE 2014 | Set 1 | Question: 39

The minimum number of comparisons required to find the minimum and the maximum of 100 numbers is _____

gatecse-2014-set1 algorithms numerical-answers normal maximum-minimum sorting

Answer key

1.29 Merge Sort (4)

✍ **Practice Test:** Test 1 (14Q)

1.29.1 Merge Sort: GATE CSE 1999 | Question: 1.14, ISRO2015-42

If one uses straight two-way merge sort algorithm to sort the following elements in ascending order:

$20, 47, 15, 8, 9, 4, 40, 30, 12, 17$

then the order of these elements after second pass of the algorithm is:

A. $8, 9, 15, 20, 47, 4, 12, 17, 30, 40$
B. $8, 15, 20, 47, 4, 9, 30, 40, 12, 17$
C. $15, 20, 47, 4, 8, 9, 12, 30, 40, 17$

D. $4, 8, 9, 15, 20, 47, 12, 17, 30, 40$

gate1999 algorithms merge-sort normal isro2015 sorting

Answer key

1.29.2 Merge Sort: GATE CSE 2012 | Question: 39

A list of n strings, each of length n, is sorted into lexicographic order using the merge-sort algorithm. The worst case running time of this computation is

A. $O(n \log n)$ B. $O(n^2 \log n)$ C. $O(n^2 + \log n)$ D. $O(n^2)$

gatecse-2012 algorithms sorting normal merge-sort

Answer key

1.29.3 Merge Sort: GATE CSE 2015 | Set 3 | Question: 27

Assume that a mergesort algorithm in the worst case takes 30 seconds for an input of size 64. Which of the following most closely approximates the maximum input size of a problem that can be solved in 6 minutes?

A. 256 B. 512 C. 1024 D. 2018

gatecse-2015-set3 algorithms sorting merge-sort

Answer key

1.29.4 Merge Sort: GATE CSE 2026 | Set 2 | Question: 22

Consider an array $A = [10, 7, 8, 19, 41, 35, 25, 31]$. Suppose the merge sort algorithm is executed on array A to sort it in increasing order. The merge sort algorithm will carry out a total of 7 merge operations.

A merge operation on sorted left array L and sorted right array R is said to be void if the output of the merge operation is the elements of array L followed by the elements of array R.

The number of void merge operations among these 7 merge operations is ____. (answer in integer)

gatecse-2026-set2 algorithms merge-sort numerical-answers one-mark

Answer key

1.30 Merging (2)

✍ **Practice Test:** Test 1 (6Q)

1.30.1 Merging: GATE CSE 1995 | Question: 1.16

For merging two sorted lists of sizes m and n into a sorted list of size $m + n$, we require comparisons of

A. $O(m)$ B. $O(n)$ C. $O(m + n)$ D. $O(\log m + \log n)$

gate1995 algorithms sorting normal merging

Answer key

1.30.2 Merging: GATE CSE 2014 | Set 2 | Question: 38

Suppose P, Q, R, S, T are sorted sequences having lengths $20, 24, 30, 35, 50$ respectively. They are to be merged into a single sequence by merging together two sequences at a time. The number of comparisons that will be needed in the worst case by the optimal algorithm for doing this is ____.

gatecse-2014-set2 algorithms sorting normal numerical-answers merging

Answer key

1.31 Minimum Spanning Tree (35)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (9Q)

1.31.1 Minimum Spanning Tree: GATE CSE 1991 | Question: 03,vi

Kruskal's algorithm for finding a minimum spanning tree of a weighted graph G with n vertices and m edges has the time complexity of:

A. $O(n^2)$ B. $O(mn)$ C. $O(m+n)$ D. $O(m \log n)$

E. $O(m^2)$

gate1991 algorithms graph-algorithms minimum-spanning-tree time-complexity multiple-selects

Answer key

1.31.2 Minimum Spanning Tree: GATE CSE 1992 | Question: 01,ix

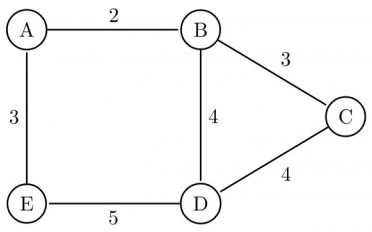
Complexity of Kruskal's algorithm for finding the minimum spanning tree of an undirected graph containing n vertices and m edges if the edges are sorted is _____

gate1992 minimum-spanning-tree algorithms time-complexity easy fill-in-the-blanks

Answer key

1.31.3 Minimum Spanning Tree: GATE CSE 1995 | Question: 22

How many minimum spanning trees does the following graph have? Draw them. (Weights are assigned to edges).



gate1995 algorithms graph-algorithms minimum-spanning-tree easy descriptive

Answer key

1.31.4 Minimum Spanning Tree: GATE CSE 1996 | Question: 16

A complete, undirected, weighted graph G is given on the vertex $\{0, 1, \ldots, n-1\}$ for any fixed 'n'. Draw the minimum spanning tree of G if

A. the weight of the edge (u, v) is $|u - v|$

B. the weight of the edge (u, v) is $u + v$

gate1996 algorithms graph-algorithms minimum-spanning-tree normal descriptive

Answer key

1.31.5 Minimum Spanning Tree: GATE CSE 1997 | Question: 9

Consider a graph whose vertices are points in the plane with integer co-ordinates (x, y) such that $1 \leq x \leq n$ and $1 \leq y \leq n$, where $n \geq 2$ is an integer. Two vertices (x_1, y_1) and (x_2, y_2) are adjacent iff $|x_1 - x_2| \leq 1$ and $|y_1 - y_2| \leq 1$. The weight of an edge $\{(x_1, y_1), (x_2, y_2)\}$ is $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

A. What is the weight of a minimum weight-spanning tree in this graph? Write only the answer without any explanations.

B. What is the weight of a maximum weight-spanning tree in this graph? Write only the answer without any explanations.

gate1997 algorithms minimum-spanning-tree normal descriptive

Answer key

1.31.6 Minimum Spanning Tree: GATE CSE 2000 | Question: 2.18

Let G be an undirected connected graph with distinct edge weights. Let e_{max} be the edge with maximum weight and e_{min} the edge with minimum weight. Which of the following statements is false?

A. Every minimum spanning tree of G must contain e_{min}
B. If e_{max} is in a minimum spanning tree, then its removal must disconnect G
C. No minimum spanning tree contains e_{max}
D. G has a unique minimum spanning tree

gatecse-2000 algorithms minimum-spanning-tree normal

Answer key

1.31.7 Minimum Spanning Tree: GATE CSE 2001 | Question: 15

Consider a weighted undirected graph with vertex set $V = \{n1, n2, n3, n4, n5, n6\}$ and edge set $E = \{(n1, n2, 2), (n1, n3, 8), (n1, n6, 3), (n2, n4, 4), (n2, n5, 12), (n3, n4, 7), (n4, n5, 9), (n4, n6, 4)\}$.

The third value in each tuple represents the weight of the edge specified in the tuple.

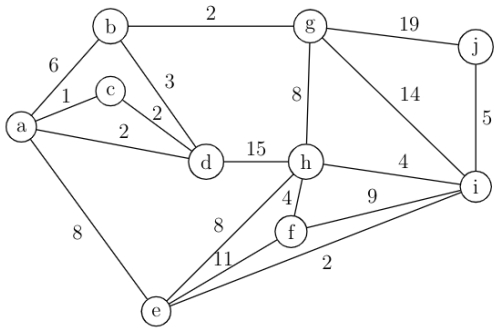
A. List the edges of a minimum spanning tree of the graph.
B. How many distinct minimum spanning trees does this graph have?
C. Is the minimum among the edge weights of a minimum spanning tree unique over all possible minimum spanning trees of a graph?
D. Is the maximum among the edge weights of a minimum spanning tree unique over all possible minimum spanning tree of a graph?

gatecse-2001 algorithms minimum-spanning-tree normal descriptive

Answer key

1.31.8 Minimum Spanning Tree: GATE CSE 2003 | Question: 68

What is the weight of a minimum spanning tree of the following graph?



A. 29 B. 31 C. 38 D. 41

gatecse-2003 algorithms minimum-spanning-tree normal

Answer key

1.31.9 Minimum Spanning Tree: GATE CSE 2005 | Question: 6

An undirected graph G has n nodes. its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. Which one of the following is TRUE?

A. Graph G has no minimum spanning tree (MST)
B. Graph G has unique MST of cost $n-1$
C. Graph G has multiple distinct MSTs, each of cost $n-1$
D. Graph G has multiple spanning trees of different costs

gatecse-2005 algorithms minimum-spanning-tree normal

Answer key

1.31.10 Minimum Spanning Tree: GATE CSE 2006 | Question: 11

Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \ldots . v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is:

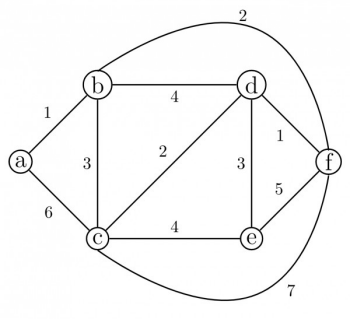
A. $n-1$ B. $2n-2$ C. $\binom{n}{2}$ D. n^2

gatecse-2006 algorithms minimum-spanning-tree normal

Answer key

1.31.11 Minimum Spanning Tree: GATE CSE 2006 | Question: 47

Consider the following graph:



Which one of the following cannot be the sequence of edges added, **in that order**, to a minimum spanning tree using Kruskal's algorithm?

A. $(a-b),(d-f),(b-f),(d-c),(d-e)$
B. $(a-b),(d-f),(d-c),(b-f),(d-e)$
C. $(d-f),(a-b),(d-c),(b-f),(d-e)$
D. $(d-f),(a-b),(b-f),(d-e),(d-c)$

gatecse-2006 algorithms graph-algorithms minimum-spanning-tree normal

Answer key

1.31.12 Minimum Spanning Tree: GATE CSE 2007 | Question: 49

Let w be the minimum weight among all edge weights in an undirected connected graph. Let e be a specific edge of weight w. Which of the following is FALSE?

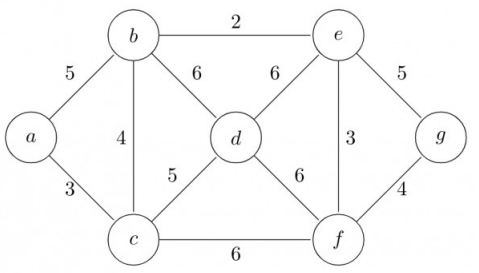
A. There is a minimum spanning tree containing e
B. If e is not in a minimum spanning tree T, then in the cycle formed by adding e to T, all edges have the same weight.
C. Every minimum spanning tree has an edge of weight w
D. e is present in every minimum spanning tree

gatecse-2007 algorithms minimum-spanning-tree normal

Answer key

1.31.13 Minimum Spanning Tree: GATE CSE 2009 | Question: 38

Consider the following graph:



Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?

A. $(\mathrm{b}, \mathrm{e})\ (\mathrm{e}, \mathrm{f})\ (\mathrm{a}, \mathrm{c})\ (\mathrm{b}, \mathrm{c})\ (\mathrm{f}, \mathrm{g})\ (\mathrm{c}, \mathrm{d})$
B. $(\mathrm{b}, \mathrm{e})\ (\mathrm{e}, \mathrm{f})\ (\mathrm{a}, \mathrm{c})\ (\mathrm{f}, \mathrm{g})\ (\mathrm{b}, \mathrm{c})\ (\mathrm{c}, \mathrm{d})$
C. $(\mathrm{b}, \mathrm{e})\ (\mathrm{a}, \mathrm{c})\ (\mathrm{e}, \mathrm{f})\ (\mathrm{b}, \mathrm{c})\ (\mathrm{f}, \mathrm{g})\ (\mathrm{c}, \mathrm{d})$
D. $(\mathrm{b}, \mathrm{e})\ (\mathrm{e}, \mathrm{f})\ (\mathrm{b}, \mathrm{c})\ (\mathrm{a}, \mathrm{c})\ (\mathrm{f}, \mathrm{g})\ (\mathrm{c}, \mathrm{d})$

gatecse-2009 algorithms minimum-spanning-tree normal

Answer key

1.31.14 Minimum Spanning Tree: GATE CSE 2010 | Question: 50

Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry W_{ij} in the matrix W below is the weight of the edge $\{i, j\}$

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

What is the minimum possible weight of a spanning tree T in this graph such that vertex 0 is a leaf node in the tree T?

A. 7 B. 8 C. 9 D. 10

gatecse-2010 algorithms minimum-spanning-tree normal

Answer key

1.31.15 Minimum Spanning Tree: GATE CSE 2010 | Question: 51

Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry W_{ij} in the matrix W below is the weight of the edge $\{i, j\}$

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

What is the minimum possible weight of a path P from vertex 1 to vertex 2 in this graph such that P contains at most 3 edges?

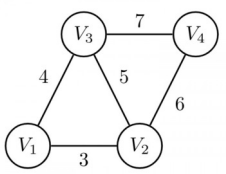
A. 7 B. 8 C. 9 D. 10

gatecse-2010 normal algorithms minimum-spanning-tree

Answer key

1.31.16 Minimum Spanning Tree: GATE CSE 2011 | Question: 54

An undirected graph $G(V, E)$ contains $n\,(n > 2)$ nodes named $v_1, v_2, \ldots, v_n$. Two nodes v_i, v_j are connected if and only if $0 <\mid i - j \mid \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below.



What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

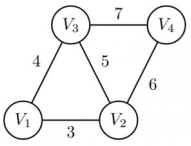
A. $\frac{1}{12}(11n^2 - 5n)$ B. $n^2 - n + 1$ C. $6n - 11$ D. $2n + 1$

gatecse-2011 algorithms graph-algorithms minimum-spanning-tree normal

Answer key

1.31.17 Minimum Spanning Tree: GATE CSE 2011 | Question: 55

An undirected graph $G(V, E)$ contains $n\,(n > 2)$ nodes named $v_1, v_2, \ldots, v_n$. Two nodes v_i, v_j are connected if and only if $0 <\mid i - j \mid \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below.



The length of the path from v_5 to v_6 in the MST of previous question with $n = 10$ is

A. 11 B. 25 C. 31 D. 41

gatecse-2011 algorithms graph-algorithms minimum-spanning-tree normal

Answer key

1.31.18 Minimum Spanning Tree: GATE CSE 2012 | Question: 29

Let G be a weighted graph with edge weights greater than one and G' be the graph constructed by squaring the weights of edges in G. Let T and T' be the minimum spanning trees of G and G', respectively, with total weights t and t'. Which of the following statements is **TRUE**?

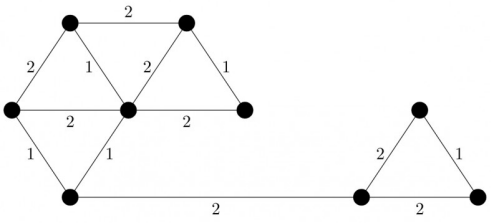
A. $T' = T$ with total weight $t' = t^2$
B. $T' = T$ with total weight $t' < t^2$
C. $T' \neq T$ but total weight $t' = t^2$
D. None of the above

gatecse-2012 algorithms minimum-spanning-tree normal marks-to-all

Answer key

1.31.19 Minimum Spanning Tree: GATE CSE 2014 | Set 2 | Question: 52

The number of distinct minimum spanning trees for the weighted graph below is _____

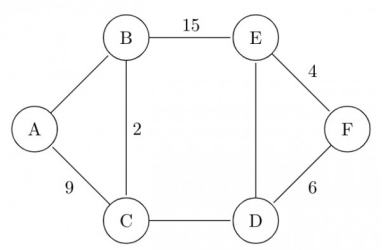


gatecse-2014-set2 algorithms minimum-spanning-tree numerical-answers normal

Answer key

1.31.20 Minimum Spanning Tree: GATE CSE 2015 | Set 1 | Question: 43

The graph shown below has 8 edges with distinct integer edge weights. The minimum spanning tree (**MST**) is of weight 36 and contains the edges: $\{(A, C), (B, C), (B, E), (E, F), (D, F)\}$. The edge weights of only those edges which are in the **MST** are given in the figure shown below. The minimum possible sum of weights of all 8 edges of this graph is_____.



gatecse-2015-set1 algorithms minimum-spanning-tree normal numerical-answers

Answer key

1.31.21 Minimum Spanning Tree: GATE CSE 2015 | Set 3 | Question: 40

Let G be a connected undirected graph of 100 vertices and 300 edges. The weight of a minimum spanning tree of G is 500. When the weight of each edge of G is increased by five, the weight of a minimum spanning tree becomes _____.

gatecse-2015-set3 algorithms minimum-spanning-tree easy numerical-answers

Answer key

1.31.22 Minimum Spanning Tree: GATE CSE 2016 | Set 1 | Question: 14

Let G be a weighted connected undirected graph with distinct positive edge weights. If every edge weight is increased by the same value, then which of the following statements is/are TRUE?

- P: Minimum spanning tree of G does not change.
- Q: Shortest path between any pair of vertices does not change.

A. P only B. Q only C. Neither P nor Q D. Both P and Q

gatecse-2016-set1 algorithms minimum-spanning-tree normal

Answer key

1.31.23 Minimum Spanning Tree: GATE CSE 2016 | Set 1 | Question: 39

Let G be a complete undirected graph on 4 vertices, having 6 edges with weights being $1, 2, 3, 4, 5,$ and 6. The maximum possible weight that a minimum weight spanning tree of G can have is _____

gatecse-2016-set1 algorithms minimum-spanning-tree normal numerical-answers

Answer key

1.31.24 Minimum Spanning Tree: GATE CSE 2016 | Set 1 | Question: 40

$G = (V, E)$ is an undirected simple graph in which each edge has a distinct weight, and e is a particular edge of G. Which of the following statements about the minimum spanning trees $(MSTs)$ of G is/are TRUE?

I. If e is the lightest edge of some cycle in G, then every MST of G includes e.
II. If e is the heaviest edge of some cycle in G, then every MST of G excludes e.

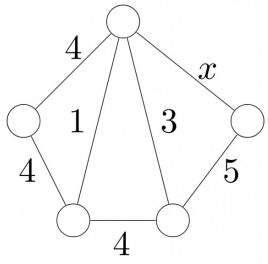
A. I only. B. II only. C. Both I and II. D. Neither I nor II.

gatecse-2016-set1 algorithms minimum-spanning-tree normal

Answer key

1.31.25 Minimum Spanning Tree: GATE CSE 2018 | Question: 47

Consider the following undirected graph G:



Choose a value for x that will maximize the number of minimum weight spanning trees (MWSTs) of G. The number of MWSTs of G for this value of x is ____.

gatecse-2018 algorithms graph-algorithms minimum-spanning-tree numerical-answers two-marks

Answer key

1.31.26 Minimum Spanning Tree: GATE CSE 2020 | Question: 31

Let $G = (V, E)$ be a weighted undirected graph and let T be a Minimum Spanning Tree (MST) of G maintained using adjacency lists. Suppose a new weighed edge $(u, v) \in V \times V$ is added to G. The worst case time complexity of determining if T is still an MST of the resultant graph is

A. $\Theta(\mid E \mid + \mid V \mid)$
B. $\Theta(\mid E \mid \mid V \mid)$
C. $\Theta(E \mid \log \mid V \mid)$
D. $\Theta(\mid V \mid)$

gatecse-2020 algorithms minimum-spanning-tree graph-algorithms two-marks

Answer key

1.31.27 Minimum Spanning Tree: GATE CSE 2020 | Question: 49

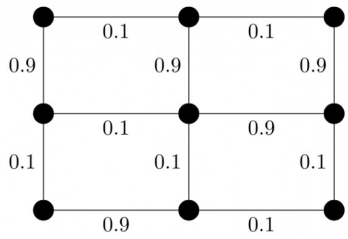
Consider a graph $G = (V, E)$, where $V = \{v_1, v_2, \ldots, v_{100}\}$, $E = \{(v_i, v_j) \mid 1 \leq i < j \leq 100\}$, and weight of the edge (v_i, v_j) is $|i-j|$. The weight of minimum spanning tree of G is _____

gatecse-2020 numerical-answers algorithms graph-algorithms two-marks minimum-spanning-tree

Answer key

1.31.28 Minimum Spanning Tree: GATE CSE 2021 | Set 1 | Question: 17

Consider the following undirected graph with edge weights as shown:



The number of minimum-weight spanning trees of the graph is _____.

gatecse-2021-set1 algorithms graph-algorithms minimum-spanning-tree numerical-answers one-mark

Answer key

1.31.29 Minimum Spanning Tree: GATE CSE 2021 | Set 2 | Question: 1

Let G be a connected undirected weighted graph. Consider the following two statements.

- S_1: There exists a minimum weight edge in G which is present in every minimum spanning tree of G.
- S_2: If every edge in G has distinct weight, then G has a unique minimum spanning tree.

Which one of the following options is correct?

A. Both S_1 and S_2 are true
B. S_1 is true and S_2 is false
C. S_1 is false and S_2 is true
D. Both S_1 and S_2 are false

gatecse-2021-set2 algorithms graph-algorithms minimum-spanning-tree one-mark

Answer key

1.31.30 Minimum Spanning Tree: GATE CSE 2022 | Question: 39

Consider a simple undirected weighted graph G, all of whose edge weights are distinct. Which of the following statements about the minimum spanning trees of G is/are TRUE?

A. The edge with the second smallest weight is always part of any minimum spanning tree of G.
B. One or both of the edges with the third smallest and the fourth smallest weights are part of any minimum spanning tree of G.
C. Suppose $S \subseteq V$ be such that $S \neq \phi$ and $S \neq V$. Consider the edge with the minimum weight such that one of its vertices is in S and the other in $V \setminus S$. Such an edge will always be part of any minimum spanning tree of G.
D. G can have multiple minimum spanning trees.

gatecse-2022 algorithms minimum-spanning-tree multiple-selects two-marks

Answer key

1.31.31 Minimum Spanning Tree: GATE CSE 2022 | Question: 48

Let $G(V,E)$ be a directed graph, where $V = \{1,2,3,4,5\}$ is the set of vertices and E is the set of directed edges, as defined by the following adjacency matrix A.

$$A[i][j] = \begin{cases} 1, & 1 \leq j \leq i \leq 5 \\ 0, & \text{otherwise} \end{cases}$$

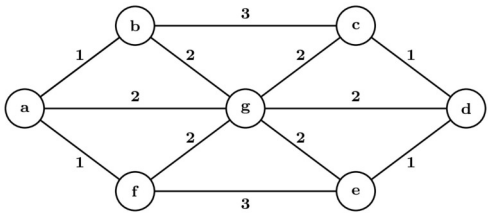
$A[i][j] = 1$ indicates a directed edge from node i to node j. A *directed spanning tree* of G, rooted at $r \in V$, is defined as a subgraph T of G such that the undirected version of T is a tree, and T contains a directed path from r to every other vertex in V. The number of such directed spanning trees rooted at vertex 5 is _____.

gatecse-2022 numerical-answers algorithms minimum-spanning-tree two-marks

Answer key

1.31.32 Minimum Spanning Tree: GATE CSE 2024 | Set 2 | Question: 49

The number of distinct minimum-weight spanning trees of the following graph is

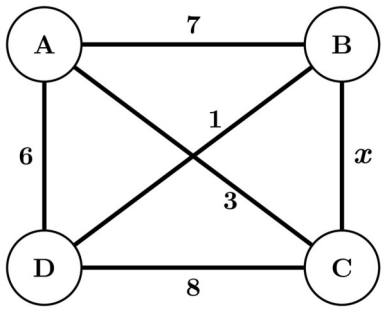


gatecse-2024-set2 numerical-answers algorithms minimum-spanning-tree two-marks

Answer key

1.31.33 Minimum Spanning Tree: GATE CSE 2025 | Set 1 | Question: 54

The maximum value of x such that the edge between the nodes B and C is included in every minimum spanning tree of the given graph is _____. (answer in integer)



gatecse2025-set1 algorithms minimum-spanning-tree numerical-answers easy two-marks

Answer key

1.31.34 Minimum Spanning Tree: GATE CSE 2026 | Set 1 | Question: 39

Let $G(V, E)$ be a simple, undirected, edge-weighted graph with unique edge weights.
Which of the following statements about the minimum spanning trees (MST) of G is/are true?

A. In every cycle C of G, the edge with the largest weight in C is not in any MST
B. In every cycle C of G, the edge with the smallest weight in C is in every MST
C. For every vertex $v \in V$, the edge with the largest weight incident on v is not in any MST
D. For every vertex $v \in V$, the edge with the smallest weight incident on v is in every MST

gatecse-2026-set1 two-marks algorithms minimum-spanning-tree multiple-selects

Answer key

1.31.35 Minimum Spanning Tree: GATE IT 2005 | Question: 52

Let G be a weighted undirected graph and e be an edge with maximum weight in G. Suppose there is a minimum weight spanning tree in G containing the edge e. Which of the following statements is always TRUE?

A. There exists a cutset in G having all edges of maximum weight.
B. There exists a cycle in G having all edges of maximum weight.
C. Edge e cannot be contained in a cycle.
D. All edges in G have the same weight.

gateit-2005 algorithms minimum-spanning-tree normal

Answer key

1.32 Number of Swap (1)

1.32.1 Number of Swap: GATE CSE 2025 | Set 1 | Question: 23

The pseudocode of a function $\text{fun}\ ()$ is given below:

```
fun(int A[0,....,n-1]) {
for i=0 to n-2
for j=0 to n-i-2
if (A[j]>A[j+1])
then swap A[j] and A[j+1]
}
```

Let $A[0, \ldots, 29]$ be an array storing 30 distinct integers in descending order. The number of swap operations that will be performed, if the function $\text{fun}\ ()$ is called with $A[0, \ldots, 29]$ as argument, is _____. (Answer in integer)

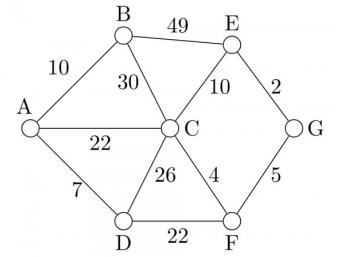
gatecse2025-set1 data-structures array number-of-swap numerical-answers one-mark

Answer key

1.33 Prims Algorithm (2)

1.33.1 Prims Algorithm: GATE IT 2004 | Question: 56

Consider the undirected graph below:



Using Prim's algorithm to construct a minimum spanning tree starting with node A, which one of the following sequences of edges represents a possible order in which the edges would be added to construct the minimum spanning tree?

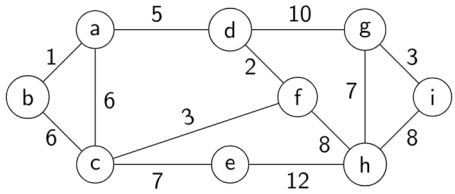
A. $(E, G), (C, F), (F, G), (A, D), (A, B), (A, C)$
B. $(A, D), (A, B), (A, C), (C, F), (G, E), (F, G)$
C. $(A, B), (A, D), (D, F), (F, G), (G, E), (F, C)$
D. $(A, D), (A, B), (D, F), (F, C), (F, G), (G, E)$

gateit-2004 algorithms graph-algorithms normal prims-algorithm

Answer key

1.33.2 Prims Algorithm: GATE IT 2008 | Question: 45

For the undirected, weighted graph given below, which of the following sequences of edges represents a correct execution of Prim's algorithm to construct a Minimum Spanning Tree?



A. $(a, b), (d, f), (f, c), (g, i), (d, a), (g, h), (c, e), (f, h)$
B. $(c, e), (c, f), (f, d), (d, a), (a, b), (g, h), (h, f), (g, i)$
C. $(d, f), (f, c), (d, a), (a, b), (c, e), (f, h), (g, h), (g, i)$
D. $(h, g), (g, i), (h, f), (f, c), (f, d), (d, a), (a, b), (c, e)$

gateit-2008 algorithms graph-algorithms minimum-spanning-tree normal prims-algorithm

Answer key

1.34 Quick Sort (15)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (2Q)

1.34.1 Quick Sort: GATE CSE 1987 | Question: 1-xviii

Let P be a quicksort program to sort numbers in ascending order. Let t_1 and t_2 be the time taken by the program for the inputs $[1\ 2\ 3\ 4]$ and $[5\ 4\ 3\ 2\ 1]$, respectively. Which of the following holds?

A. $t_1 = t_2$
B. $t_1 > t_2$
C. $t_1 < t_2$
D. $t_1 = t_2 + 5 \log 5$

gate1987 algorithms sorting quick-sort

Answer key

1.34.2 Quick Sort: GATE CSE 1989 | Question: 9

An input files has 10 records with keys as given below:

$25 \quad 7 \quad 34 \quad 2 \quad 70 \quad 9 \quad 61 \quad 16 \quad 49 \quad 19$

This is to be sorted in non-decreasing order.

i. Sort the input file using QUICKSORT by correctly positioning the first element of the file/subfile. Show the subfiles obtained at all intermediate steps. Use square brackets to demarcate subfiles.
ii. Sort the input file using 2-way- MERGESORT showing all major intermediate steps. Use square brackets to demarcate subfiles.

gate1989 descriptive algorithms sorting quick-sort

Answer key

1.34.3 Quick Sort: GATE CSE 1992 | Question: 03,iv

Assume that the last element of the set is used as partition element in Quicksort. If n distinct elements from the set $[1 \ldots n]$ are to be sorted, give an input for which Quicksort takes maximum time.

gate1992 algorithms sorting easy quick-sort descriptive

Answer key

1.34.4 Quick Sort: GATE CSE 1996 | Question: 2.15

Quick-sort is run on two inputs shown below to sort in ascending order taking first element as pivot

i. $1, 2, 3, \ldots n$
ii. $n, n-1, n-2, \ldots, 2, 1$

Let C_1 and C_2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

A. $C_1 < C_2$
B. $C_1 > C_2$
C. $C_1 = C_2$
D. we cannot say anything for arbitrary n

gate1996 algorithms sorting normal quick-sort

Answer key

1.34.5 Quick Sort: GATE CSE 2001 | Question: 1.14

Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using Randomized quicksort?

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n!)$

gatecse-2001 algorithms sorting time-complexity easy quick-sort

Answer key

1.34.6 Quick Sort: GATE CSE 2006 | Question: 52

The median of n elements can be found in $O(n)$ time. Which one of the following is correct about the complexity of quick sort, in which median is selected as pivot?

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(n^3)$

gatecse-2006 algorithms sorting easy quick-sort

Answer key

1.34.7 Quick Sort: GATE CSE 2008 | Question: 43

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let $T(n)$ be the number of comparisons required to sort n elements. Then

A. $T(n) \leq 2T(n/5) + n$
B. $T(n) \leq T(n/5) + T(4n/5) + n$
C. $T(n) \leq 2T(4n/5) + n$
D. $T(n) \leq 2T(n/2) + n$

gatecse-2008 algorithms sorting easy quick-sort

Answer key

1.34.8 Quick Sort: GATE CSE 2009 | Question: 39

In quick-sort, for sorting n elements, the $(n/4)^{th}$ smallest element is selected as pivot using an $O(n)$ time algorithm. What is the worst case time complexity of the quick sort?

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(n^2 \log n)$

gatecse-2009 algorithms sorting normal quick-sort

Answer key

1.34.9 Quick Sort: GATE CSE 2014 | Set 1 | Question: 14

Let P be quicksort program to sort numbers in ascending order using the first element as the pivot. Let t_1 and t_2 be the number of comparisons made by P for the inputs $[1\,2\,3\,4\,5]$ and $[4\,1\,5\,3\,2]$ respectively. Which one of the following holds?

A. $t_1 = 5$
B. $t_1 < t_2$
C. $t_1 > t_2$
D. $t_1 = t_2$

gatecse-2014-set1 algorithms sorting easy quick-sort

Answer key

1.34.10 Quick Sort: GATE CSE 2014 | Set 3 | Question: 14

You have an array of n elements. Suppose you implement quicksort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

A. $O(n^2)$
B. $O(n \log n)$
C. $\Theta(n \log n)$
D. $O(n^3)$

gatecse-2014-set3 algorithms sorting easy quick-sort

Answer key

1.34.11 Quick Sort: GATE CSE 2015 | Set 1 | Question: 2

Which one of the following is the recurrence equation for the worst case time complexity of the quick sort algorithm for sorting n (≥ 2) numbers? In the recurrence equations given in the options below, c is a constant.

A. $T(n) = 2T(n/2) + cn$
B. $T(n) = T(n-1) + T(1) + cn$
C. $T(n) = 2T(n-1) + cn$
D. $T(n) = T(n/2) + cn$

gatecse-2015-set1 algorithms recurrence-relation sorting easy quick-sort

Answer key

1.34.12 Quick Sort: GATE CSE 2015 | Set 2 | Question: 45

Suppose you are provided with the following function declaration in the C programming language.

```
int partition(int a[], int n);
```

The function treats the first element of $a[\,]$ as a pivot and rearranges the array so that all elements less than or equal to the pivot is in the left part of the array, and all elements greater than the pivot is in the right part. In addition, it moves the pivot so that the pivot is the last element of the left part. The return value is the number of elements in the left part.

The following partially given function in the C programming language is used to find the k^{th} smallest element in an array $a[\,]$ of size n using the partition function. We assume $k \leq n$.

```
int kth_smallest (int a[], int n, int k)
{
   int left_end = partition (a, n);
```

```
    if (left_end+1==k) {
        return a[left_end];
    }
    if (left_end+1 > k) {
        return kth_smallest (_____);
    } else {
        return kth_smallest (_____);
    }
}
```

The missing arguments lists are respectively

A. $(a,$ left_end$,k)$ and $(a+$left_end $+1, n-$left_end$-1, k-$left_end $-1)$

B. $(a,$ left_end$,k)$ and $(a, n-$left_end $-1, k-$left_end$-1)$

C. $(a+$ left_end$+1, n-$left_end $-1, k-$left_end$-1)$ and $(a,$ left_end$,k)$

D. $(a, n-$left_end$-1, k-$left_end $-1)$ and $(a,$left_end$,k)$

gatecse-2015-set2 algorithms normal sorting quick-sort

Answer key

1.34.13 Quick Sort: GATE CSE 2019 | Question: 20

An array of 25 distinct elements is to be sorted using quicksort. Assume that the pivot element is chosen uniformly at random. The probability that the pivot element gets placed in the worst possible location in the first round of partitioning (rounded off to 2 decimal places) is _____

gatecse-2019 numerical-answers algorithms quick-sort probability one-mark

Answer key

1.34.14 Quick Sort: GATE DA 2026 | Question: 5

Consider that the quick sort algorithm is used to sort an array of n distinct randomly ordered elements. In every call, the pivot is chosen as the first element of the current subarray.

Let $T(n)$ denote the expected time to sort the array. Assume that the time to partition is linear in the size of the current subarray.

Which of the following recurrence relations correctly represents $T(n)$ in this scenario?

A. $T(n) = T(1) + T(n-1) + O(n)$

B. $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + O(n)$

C. $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

D. $T(n) = \frac{1}{n}\sum_{k=0}^{n-1}[T(k) + T(n-k-1)] + O(n)$

gateda-2026 algorithms quick-sort recurrence-relation one-mark

Answer key

1.34.15 Quick Sort: GATE DS&AI 2024 | Question: 20

Consider sorting the following array of integers in ascending order using an inplace Quicksort algorithm that uses the last element as the pivot.

60	70	80	90	100

The minimum number of swaps performed during this Quicksort is _____.

gate-ds-ai-2024 numerical-answers algorithms quick-sort one-mark

Answer key

1.35 Recurrence Relation (36)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (14Q)

1.35.1 Recurrence Relation: GATE CSE 1987 | Question: 10a

Solve the recurrence equations:

- $T(n) = T(n-1) + n$
- $T(1) = 1$

gate1987 algorithms recurrence-relation descriptive

Answer key

1.35.2 Recurrence Relation: GATE CSE 1988 | Question: 13iv

Solve the recurrence equations:

- $T(n) = T(\frac{n}{2}) + 1$
- $T(1) = 1$

gate1988 descriptive algorithms recurrence-relation

Answer key

1.35.3 Recurrence Relation: GATE CSE 1989 | Question: 13b

Find a solution to the following recurrence equation:

- $T(n) = \sqrt{n} + T\left(\frac{n}{2}\right)$
- $T(1) = 1$

gate1989 descriptive algorithms recurrence-relation

Answer key

1.35.4 Recurrence Relation: GATE CSE 1990 | Question: 17a

Express $T(n)$ in terms of the harmonic number $H_n = \sum_{i=1}^{n} \frac{1}{i}, \quad n \geq 1,$ where $T(n)$ satisfies the recurrence relation,

$T(n) = \frac{n+1}{n} T(n-1) + 1$, for $n \geq \sum$ and $T(1) = 1$

What is the asymptotic behaviour of $T(n)$ as a function of n ?

gate1990 descriptive algorithms recurrence-relation

Answer key

1.35.5 Recurrence Relation: GATE CSE 1992 | Question: 07a

Consider the function $F(n)$ for which the pseudocode is given below :

```
Function F(n)
begin
F1 ← 1
if(n=1) then F ← 3
 else
   For i = 1 to n do
      begin
         C ← 0
        For j = 1 to n – 1 do
        begin C ← C + 1 end
        F1 = F1 * C
     end
 F = F1
end
```

[n is a positive integer greater than zero]

A. Derive a recurrence relation for $F(n)$.

gate1992 algorithms recurrence-relation descriptive

Answer key

1.35.6 Recurrence Relation: GATE CSE 1992 | Question: 07b

Consider the function $F(n)$ for which the pseudocode is given below :

```
Function F(n)
begin
F1 ← 1
if(n=1) then F ← 3
 else
   For i = 1 to n do
      begin
         C ← 0
        For j = 1 to n – 1 do
        begin C ← C + 1 end
        F1 = F1 * C
      end
 F = F1
end
```

[n is a positive integer greater than zero]

B. Solve the recurrence relation for a closed form solution of $F(n)$.

gate1992 algorithms recurrence-relation descriptive

Answer key

1.35.7 Recurrence Relation: GATE CSE 1993 | Question: 15

Consider the recursive algorithm given below:

```
procedure bubblesort (n);
var i,j: index; temp : item;
begin
   for i:=1 to n-1 do
      if A[i] > A[i+1] then
      begin
         temp := A[i];
         A[i] := A[i+1];
         A[i+1] := temp;
      end;
   bubblesort (n-1)
end
```

Let a_n be the number of times the 'if…then…' statement gets executed when the algorithm is run with value n. Set up the recurrence relation by defining a_n in terms of a_{n-1}. Solve for a_n.

gate1993 algorithms recurrence-relation normal descriptive

Answer key

1.35.8 Recurrence Relation: GATE CSE 1994 | Question: 1.7, ISRO2017-14

The recurrence relation that arises in relation with the complexity of binary search is:

A. $T(n) = 2T\left(\frac{n}{2}\right) + k$, k is a constant

B. $T(n) = T\left(\frac{n}{2}\right) + k$, k is a constant

C. $T(n) = T\left(\frac{n}{2}\right) + \log n$

D. $T(n) = T\left(\frac{n}{2}\right) + n$

gate1994 algorithms recurrence-relation easy isro2017

Answer key

1.35.9 Recurrence Relation: GATE CSE 1996 | Question: 2.12

The recurrence relation

- $T(1) = 2$
- $T(n) = 3T(\frac{n}{4}) + n$

has the solution $T(n)$ equal to

A. $O(n)$ B. $O(\log n)$ C. $O\left(n^{\frac{3}{4}}\right)$ D. None of the above

gate1996 algorithms recurrence-relation normal

Answer key

1.35.10 Recurrence Relation: GATE CSE 1997 | Question: 15

Consider the following function.

```
Function F(n, m:integer):integer;
begin
  if (n<=0) or (m<=0) then F:=1
  else
    F:F(n-1, m) + F(n-1, m-1);
end;
```

Use the recurrence relation $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$ to answer the following questions. Assume that n, m are positive integers. Write only the answers without any explanation.

a. What is the value of $F(n, 2)$?
b. What is the value of $F(n, m)$?
c. How many recursive calls are made to the function F, including the original call, when evaluating $F(n, m)$.

gate1997 algorithms recurrence-relation descriptive

Answer key

1.35.11 Recurrence Relation: GATE CSE 1997 | Question: 4.6

Let $T(n)$ be the function defined by $T(1) = 1$, $T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + \sqrt{n}$ for $n \geq 2$.
Which of the following statements is true?

A. $T(n) = O\sqrt{n}$ B. $T(n) = O(n)$
C. $T(n) = O(\log n)$ D. None of the above

gate1997 algorithms recurrence-relation normal

Answer key

1.35.12 Recurrence Relation: GATE CSE 1998 | Question: 6a

Solve the following recurrence relation

$x_n = 2x_{n-1} - 1, n > 1$

$x_1 = 2$

gate1998 algorithms recurrence-relation descriptive

Answer key

1.35.13 Recurrence Relation: GATE CSE 2002 | Question: 1.3

The solution to the recurrence equation $T(2^k) = 3T(2^{k-1}) + 1, T(1) = 1$ is

A. 2^k B. $\frac{(3^{k+1}-1)}{2}$ C. $3^{\log_2 k}$ D. $2^{\log_3 k}$

gatecse-2002 algorithms recurrence-relation normal

Answer key

1.35.14 Recurrence Relation: GATE CSE 2002 | Question: 2.11

The running time of the following algorithm

Procedure $A(n)$

If $n \leqslant 2$ return (1) else return $(A(\lceil \sqrt{n} \rceil))$;

is best described by

A. $O(n)$ B. $O(\log n)$ C. $O(\log \log n)$ D. $O(1)$

gatecse-2002 algorithms recurrence-relation normal

Answer key

1.35.15 Recurrence Relation: GATE CSE 2003 | Question: 35

Consider the following recurrence relation

$T(1) = 1$

$T(n+1) = T(n) + \lfloor \sqrt{n+1} \rfloor$ for all $n \geq 1$

The value of $T(m^2)$ for $m \geq 1$ is

A. $\frac{m}{6}(21m - 39) + 4$
B. $\frac{m}{6}(4m^2 - 3m + 5)$
C. $\frac{m}{2}(3m^{2.5} - 11m + 20) - 5$
D. $\frac{m}{6}(5m^3 - 34m^2 + 137m - 104) + \frac{5}{6}$

gatecse-2003 algorithms time-complexity recurrence-relation difficult

Answer key

1.35.16 Recurrence Relation: GATE CSE 2004 | Question: 83, ISRO2015-40

The time complexity of the following C function is (assume $n > 0$)

```
int recursive (int n) {
   if(n == 1)
      return (1);
   else
      return (recursive (n-1) + recursive (n-1));
}
```

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(2^n)$

gatecse-2004 algorithms recurrence-relation time-complexity normal isro2015

Answer key

1.35.17 Recurrence Relation: GATE CSE 2004 | Question: 84

The recurrence equation

$T(1) = 1$

$T(n) = 2T(n-1) + n, n \geq 2$

evaluates to

A. $2^{n+1} - n - 2$ B. $2^n - n$ C. $2^{n+1} - 2n - 2$ D. $2^n + n$

gatecse-2004 algorithms recurrence-relation normal

Answer key

1.35.18 Recurrence Relation: GATE CSE 2006 | Question: 51, ISRO2016-34

Consider the following recurrence:

$T(n) = 2T(\sqrt{n}) + 1, T(1) = 1$

Which one of the following is true?

A. $T(n) = \Theta(\log \log n)$
B. $T(n) = \Theta(\log n)$
C. $T(n) = \Theta(\sqrt{n})$
D. $T(n) = \Theta(n)$

algorithms recurrence-relation isro2016 gatecse-2006

Answer key

1.35.19 Recurrence Relation: GATE CSE 2008 | Question: 78

Let x_n denote the number of binary strings of length n that contain no consecutive 0s.
Which of the following recurrences does x_n satisfy?

A. $x_n = 2x_{n-1}$
B. $x_n = x_{\lfloor n/2 \rfloor} + 1$
C. $x_n = x_{\lfloor n/2 \rfloor} + n$
D. $x_n = x_{n-1} + x_{n-2}$

gatecse-2008 algorithms recurrence-relation normal

Answer key

1.35.20 Recurrence Relation: GATE CSE 2008 | Question: 79

Let x_n denote the number of binary strings of length n that contain no consecutive 0s.
The value of x_5 is

A. 5
B. 7
C. 8
D. 16

gatecse-2008 algorithms recurrence-relation normal

Answer key

1.35.21 Recurrence Relation: GATE CSE 2009 | Question: 35

The running time of an algorithm is represented by the following recurrence relation:

$$T(n) = \begin{cases} n & n \leq 3 \\ T(\frac{n}{3}) + cn & \text{otherwise} \end{cases}$$

Which one of the following represents the time complexity of the algorithm?

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(n^2 \log n)$

gatecse-2009 algorithms recurrence-relation time-complexity normal

Answer key

1.35.22 Recurrence Relation: GATE CSE 2012 | Question: 16

The recurrence relation capturing the optimal execution time of the $Towers\ of\ Hanoi$ problem with n discs is

A. $T(n) = 2T(n-2) + 2$
B. $T(n) = 2T(n-1) + n$
C. $T(n) = 2T(n/2) + 1$
D. $T(n) = 2T(n-1) + 1$

gatecse-2012 algorithms easy recurrence-relation

Answer key

1.35.23 Recurrence Relation: GATE CSE 2014 | Set 2 | Question: 13

Which one of the following correctly determines the solution of the recurrence relation with $T(1) = 1$?

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n$$

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(\log n)$

gatecse-2014-set2 algorithms recurrence-relation normal

Answer key

1.35.24 Recurrence Relation: GATE CSE 2015 | Set 1 | Question: 49

Let a_n represent the number of bit strings of length n containing two consecutive 1s. What is the recurrence relation for a_n?

A. $a_{n-2} + a_{n-1} + 2^{n-2}$
B. $a_{n-2} + 2a_{n-1} + 2^{n-2}$
C. $2a_{n-2} + a_{n-1} + 2^{n-2}$
D. $2a_{n-2} + 2a_{n-1} + 2^{n-2}$

gatecse-2015-set1 algorithms recurrence-relation normal

Answer key

1.35.25 Recurrence Relation: GATE CSE 2015 | Set 3 | Question: 39

Consider the following recursive C function.

```
void get(int n)
{
   if (n<1) return;
   get (n-1);
   get (n-3);
   printf("%d", n);
}
```

If $get(6)$ function is being called in $main()$ then how many times will the $get()$ function be invoked before returning to the $main()$?

A. 15 B. 25 C. 35 D. 45

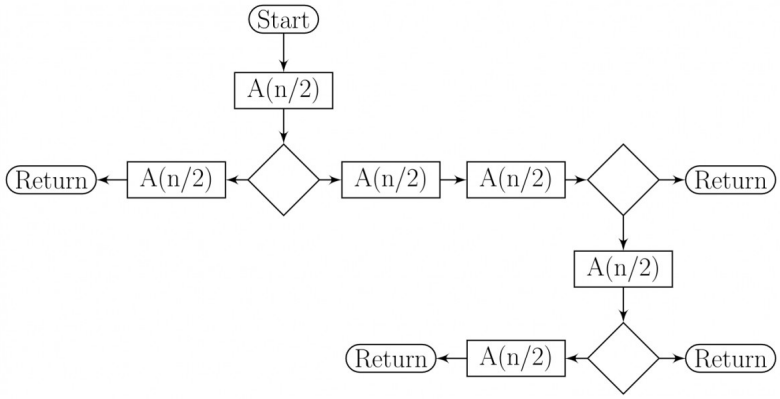
gatecse-2015-set3 algorithms recurrence-relation normal

Answer key

1.35.26 Recurrence Relation: GATE CSE 2016 | Set 2 | Question: 39

The given diagram shows the flowchart for a recursive function $A(n)$. Assume that all statements, except for the recursive calls, have $O(1)$ time complexity. If the worst case time complexity of this function is $O(n^{\alpha})$, then the least possible value (accurate up to two decimal positions) of α is _____.

Flow chart for Recursive Function $A(n)$.



gatecse-2016-set2 algorithms time-complexity recurrence-relation normal numerical-answers

Answer key

1.35.27 Recurrence Relation: GATE CSE 2017 | Set 2 | Question: 30

Consider the recurrence function

$$T(n) = \begin{cases} 2T(\sqrt{n}) + 1, & n > 2 \\ 2, & 0 < n \leq 2 \end{cases}$$

Then $T(n)$ in terms of Θ notation is

A. $\Theta(\log \log n)$ B. $\Theta(\log n)$

C. $\Theta(\sqrt{n})$ D. $\Theta(n)$

gatecse-2017-set2 algorithms recurrence-relation

Answer key

1.35.28 Recurrence Relation: GATE CSE 2020 | Question: 2

For parameters a and b, both of which are $\omega(1)$, $T(n) = T(n^{1/a}) + 1$, and $T(b) = 1$. Then $T(n)$ is

A. $\Theta(\log_a \log_b n)$ B. $\Theta(\log_{ab} n)$
C. $\Theta(\log_b \log_a n)$ D. $\Theta(\log_2 \log_2 n)$

gatecse-2020 algorithms recurrence-relation one-mark

Answer key

1.35.29 Recurrence Relation: GATE CSE 2021 | Set 1 | Question: 30

Consider the following recurrence relation.

$$T(n) = \begin{cases} T(n/2) + T(2n/5) + 7n & \text{if } n > 0 \\ 1 & \text{if } n = 0 \end{cases}$$

Which one of the following options is correct?

A. $T(n) = \Theta(n^{5/2})$ B. $T(n) = \Theta(n \log n)$
C. $T(n) = \Theta(n)$ D. $T(n) = \Theta((\log n)^{5/2})$

gatecse-2021-set1 algorithms recurrence-relation time-complexity two-marks

Answer key

1.35.30 Recurrence Relation: GATE CSE 2021 | Set 2 | Question: 39

For constants $a \geq 1$ and $b > 1$, consider the following recurrence defined on the non-negative integers:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Which one of the following options is correct about the recurrence $T(n)$?

A. If $f(n)$ is $n \log_2(n)$, then $T(n)$ is $\Theta(n \log_2(n))$
B. If $f(n)$ is $\dfrac{n}{\log_2(n)}$, then $T(n)$ is $\Theta(\log_2(n))$
C. If $f(n)$ is $O(n^{\log_b(a)-\epsilon})$ for some $\epsilon > 0$, then $T(n)$ is $\Theta(n^{\log_b(a)})$
D. If $f(n)$ is $\Theta(n^{\log_b(a)})$, then $T(n)$ is $\Theta(n^{\log_b(a)})$

gatecse-2021-set2 algorithms recurrence-relation two-marks

Answer key

1.35.31 Recurrence Relation: GATE CSE 2024 | Set 1 | Question: 32

Consider the following recurrence relation:

$$T(n) = \begin{cases} \sqrt{n}T(\sqrt{n}) + n \text{ for } n \geq 1, \\ 1 \quad \text{for } n = 1 \end{cases}$$

Which one of the following options is CORRECT?

A. $T(n) = \Theta(n \log \log n)$ B. $T(n) = \Theta(n \log n)$
C. $T(n) = \Theta(n^2 \log n)$ D. $T(n) = \Theta(n^2 \log \log n)$

gatecse-2024-set1 algorithms recurrence-relation two-marks

Answer key

1.35.32 Recurrence Relation: GATE CSE 2025 | Set 1 | Question: 10

Consider the following recurrence relation:

$$T(n) = 2T(n-1) + n2^n \text{ for } n > 0, \quad T(0) = 1$$

Which ONE of the following options is CORRECT?

A. $T(n) = \Theta\left(n^2 2^n\right)$
B. $T(n) = \Theta\left(n2^n\right)$
C. $T(n) = \Theta\left((\log n)^2 2^n\right)$
D. $T(n) = \Theta\left(4^n\right)$

gatecse2025-set1 algorithms time-complexity recurrence-relation one-mark

Answer key

1.35.33 Recurrence Relation: GATE CSE 2026 | Set 2 | Question: 15

Which of the following can be recurrence relation(s) corresponding to an algorithm with time complexity $\Theta(n)$?

A. $T(n) = T(n-1) + 1, \quad T(1) = 1$
B. $T(n) = 2T\left(\frac{n}{2}\right) + 1, \quad T(1) = 1$
C. $T(n) = 2T\left(\frac{n}{2}\right) + n, \quad T(1) = 1$
D. $T(n) = T(n-1) + n, \quad T(1) = 1$

gatecse-2026-set2 algorithms recurrence-relation multiple-selects one-mark

Answer key

1.35.34 Recurrence Relation: GATE IT 2004 | Question: 57

Consider a list of recursive algorithms and a list of recurrence relations as shown below. Each recurrence relation corresponds to exactly one algorithm and is used to derive the time complexity of the algorithm.

	Recursive Algorithm		**Recurrence Relation**
P	Binary search	I.	$T(n) = T(n-k) + T(k) + cn$
Q.	Merge sort	II.	$T(n) = 2T(n-1) + 1$
R.	Quick sort	III.	$T(n) = 2T(n/2) + cn$
S.	Tower of Hanoi	IV.	$T(n) = T(n/2) + 1$

Which of the following is the correct match between the algorithms and their recurrence relations?

A. P-II, Q-III, R-IV, S-I
B. P-IV, Q-III, R-I, S-II
C. P-III, Q-II, R-IV, S-I
D. P-IV, Q-II, R-I, S-III

gateit-2004 algorithms recurrence-relation normal match-the-following

Answer key

1.35.35 Recurrence Relation: GATE IT 2005 | Question: 51

Let $T(n)$ be a function defined by the recurrence

$T(n) = 2T(n/2) + \sqrt{n}$ for $n \geq 2$ and
$T(1) = 1$

Which of the following statements is **TRUE**?

A. $T(n) = \Theta(\log n)$
B. $T(n) = \Theta(\sqrt{n})$
C. $T(n) = \Theta(n)$
D. $T(n) = \Theta(n \log n)$

gateit-2005 algorithms recurrence-relation easy

Answer key

1.35.36 Recurrence Relation: GATE IT 2008 | Question: 44

When $n = 2^{2k}$ for some $k \geqslant 0$, the recurrence relation

$T(n) = \sqrt{(2)}T(n/2) + \sqrt{n}, T(1) = 1$

evaluates to :

A. $\sqrt{(n)}(\log n + 1)$
B. $\sqrt{(n)}\log n$
C. $\sqrt{(n)}\log\sqrt{(n)}$
D. $n\log\sqrt{n}$

gateit-2008 algorithms recurrence-relation normal

Answer key

1.36 Recursion (5)

Practice Test: Test 1 (10Q)

1.36.1 Recursion: GATE CSE 1995 | Question: 2.9

A language with string manipulation facilities uses the following operations

head(s): first character of a string
tail(s): all but exclude the first character of a string

concat(s1, s2): s1s2

For the string "$acbc$" what will be the output of

```
concat(head(s), head(tail(tail(s))))
```

A. ac
B. bc
C. ab
D. cc

gate1995 algorithms normal recursion

Answer key

1.36.2 Recursion: GATE CSE 2007 | Question: 44

In the following C function, let $n \geq m$.

```
int gcd(n,m)  {
   if (n%m == 0) return m;
   n = n%m;
   return gcd(m,n);
}
```

How many recursive calls are made by this function?

A. $\Theta(\log_2 n)$
B. $\Omega(n)$
C. $\Theta(\log_2 \log_2 n)$
D. $\Theta(\sqrt{n})$

gatecse-2007 algorithms recursion time-complexity normal

Answer key

1.36.3 Recursion: GATE CSE 2018 | Question: 45

Consider the following program written in pseudo-code. Assume that x and y are integers.

```
Count (x, y) {
   if (y !=1 ) {
      if (x !=1) {
         print("*");
         Count (x/2, y);
      }
      else {
         y=y-1;
         Count (1024, y);
      }
   }
}
```

The number of times that the $print$ statement is executed by the call $Count(1024, 1024)$ is _____

gatecse-2018 numerical-answers algorithms recursion two-marks

Answer key

1.36.4 Recursion: GATE CSE 2021 | Set 2 | Question: 49

Consider the following ANSI C program

```
#include <stdio.h>
int foo(int x, int y, int q)
  {
    if ((x<=0) && (y<=0))
    return q;
    if (x<=0)
    return foo(x, y-q, q);
    if (y<=0)
    return foo(x-q, y, q);
    return foo(x, y-q, q) + foo(x-q, y, q);
  }
int main( )
{
  int r = foo(15, 15, 10);
  printf("%d", r);
  return 0;
}
```

The output of the program upon execution is _____

gatecse-2021-set2 algorithms recursion output numerical-answers two-marks

Answer key

1.36.5 Recursion: GATE CSE 2026 | Set 1 | Question: 51

Consider the recursive functions represented by the following code segment:

```
int bar(int n) {
  if (n == 1) return 0;
  else return 1 + bar(n/2);
}
int foo(int n) {
  if (n == 1) return 1;
  else return 1 + foo(bar(n));
}
```

The smallest positive integer n for which $f \circ \circ(n)$ returns 5 is ____. (answer in integer)

Note: Ignore syntax errors (if any) in the function.

gatecse-2026-set1 algorithms recursion numerical-answers two-marks

Answer key

1.37 Searching (8)

Practice Test: Test 1 (9Q)

1.37.1 Searching: GATE CSE 1996 | Question: 18

Consider the following program that attempts to locate an element x in an array $a[]$ using binary search. Assume $N > 1$. The program is erroneous. Under what conditions does the program fail?

```
var i,j,k: integer; x: integer;
   a: array; [1..N] of integer;
begin i:= 1; j:= n;
repeat
   k:(i+j) div 2;
   if a[k] < x then i:= k
   else j:= k
until (a[k] = x) or (i >= j);

if (a[k] = x) then
   writeln ('x is in the array')
else
   writeln ('x is not in the array')
end;
```

gate1996 algorithms searching normal descriptive

Answer key

1.37.2 Searching: GATE CSE 1996 | Question: 2.13, ISRO2016-28

The average number of key comparisons required for a successful search for sequential search on n items is

A. $\frac{n}{2}$ B. $\frac{n-1}{2}$ C. $\frac{n+1}{2}$ D. None of the above

gate1996 algorithms easy isro2016 searching

Answer key

1.37.3 Searching: GATE CSE 2002 | Question: 2.10

Consider the following algorithm for searching for a given number x in an unsorted array $A[1..n]$ having n distinct values:

1. Choose an i at random from $1..n$
2. If $A[i] = x$, then Stop else Goto 1;

Assuming that x is present in A, what is the expected number of comparisons made by the algorithm before it terminates?

A. n B. $n-1$ C. $2n$ D. $\frac{n}{2}$

gatecse-2002 searching normal

Answer key

1.37.4 Searching: GATE CSE 2008 | Question: 84

Consider the following C program that attempts to locate an element x in an array $Y[\,]$ using binary search. The program is erroneous.

```
f (int Y[10] , int x) {
  int i, j, k;
  i= 0; j = 9;
  do {
     k = (i+ j) / 2;
     if( Y[k] < x) i = k;else j = k;
     } while (Y[k] != x) && (i < j)) ;
  if(Y[k] == x) printf(" x is in the array ") ;
  else printf(" x is not in the array ") ;
}
```

On which of the following contents of Y and x does the program fail?

A. Y is $[1\,2\,3\,4\,5\,6\,7\,8\,9\,10]$ and $x < 10$
B. Y is $[1\,3\,5\,7\,9\,11\,13\,15\,17\,19]$ and $x < 1$
C. Y is $[2\,2\,2\,2\,2\,2\,2\,2\,2\,2]$ and $x > 2$
D. Y is $[2\,4\,6\,8\,10\,12\,14\,16\,18\,20]$ and $2 < x < 20$ and x is even

gatecse-2008 algorithms searching normal

Answer key

1.37.5 Searching: GATE CSE 2008 | Question: 85

Consider the following C program that attempts to locate an element x in an array $Y[\,]$ using binary search. The program is erroneous.

```
f (int Y[10] , int x) {
  int i, j, k;
  i= 0; j = 9;
  do {
     k = (i + j) / 2;
     if( Y[k] < x) i = k;else j = k;
     } while (Y[k] != x) && (i < j)) ;
  if(Y[k] == x) printf(" x is in the array ") ;
  else printf(" x is not in the array ") ;
```

```
}
```

The correction needed in the program to make it work properly is

A. Change line 6 to: if $(Y[k] < x) i = k + 1$; else $j = k - 1$;
B. Change line 6 to: if $(Y[k] < x) i = k - 1$; else $j = k + 1$;
C. Change line 6 to: if $(Y[k] < x) i = k$; else $j = k$;
D. Change line 7 to: } while $((Y[k] == x) \&\& (i < j))$;

gatecse-2008 algorithms searching normal

Answer key

1.37.6 Searching: GATE CSE 2017 | Set 1 | Question: 48

Let A be an array of 31 numbers consisting of a sequence of 0's followed by a sequence of 1's. The problem is to find the smallest index i such that $A[i]$ is 1 by probing the minimum number of locations in A. The *worst case* number of probes performed by an *optimal* algorithm is _____.

gatecse-2017-set1 algorithms normal numerical-answers searching

Answer key

1.37.7 Searching: GATE CSE 2025 | Set 2 | Question: 19

Which of the following statements regarding Breadth First Search (BFS) and Depth First Search (DFS) on an undirected simple graph G is/are TRUE?

A. A DFS tree of G is a Shortest Path tree of G.
B. Every non-tree edge of G with respect to a DFS tree is a forward/back edge.
C. If (u, v) is a non-tree edge of G with respect to a BFS tree, then the distances from the source vertex s to u and v in the BFS tree are within ± 1 of each other.
D. Both BFS and DFS can be used to find the connected components of G.

gatecse2025-set2 algorithms searching breadth-first-search depth-first-search multiple-selects one-mark

Answer key

1.37.8 Searching: GATE CSE 2025 | Set 2 | Question: 31

An array A of length n with distinct elements is said to be bitonic if there is an index $1 \leq i \leq n$ such that $A[1..i]$ is sorted in the non-decreasing order and $A[i+1 \ldots n]$ is sorted in the non-increasing order.

Which ONE of the following represents the best possible asymptotic bound for the worst-case number of comparisons by an algorithm that searches for an element in a bitonic array A ?

A. $\Theta(n)$
B. $\Theta(1)$
C. $\Theta\left(\log^2 n\right)$
D. $\Theta(\log n)$

gatecse2025-set2 algorithms searching bitonic-array time-complexity two-marks

Answer key

1.38 Selection Sort (2)

1.38.1 Selection Sort: GATE CSE 2009 | Question: 11

What is the number of swaps required to sort n elements using selection sort, in the worst case?

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(n^2 \log n)$

gatecse-2009 algorithms sorting easy selection-sort

Answer key

1.38.2 Selection Sort: GATE CSE 2013 | Question: 6

Which one of the following is the tightest upper bound that represents the number of swaps required to sort n numbers using selection sort?

A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

gatecse-2013 algorithms sorting easy selection-sort

Answer key

1.39 Shortest Path (8)

✍ **Practice Test:** Test 1 (15Q)

1.39.1 Shortest Path: GATE CSE 2002 | Question: 12

Fill in the blanks in the following template of an algorithm to compute all pairs shortest path lengths in a directed graph G with $n * n$ adjacency matrix A. $A[i, j]$ equals 1 if there is an edge in G from i to j, and 0 otherwise. Your aim in filling in the blanks is to ensure that the algorithm is correct.

```
INITIALIZATION: For i = 1 ... n
  {For j = 1 ... n
    { if a[i,j] = 0 then P[i,j] =_______ else P[i,j] =_______;}
  }

ALGORITHM: For i = 1 ... n
  {For j = 1 ... n
    {For k = 1 ... n
      {P[__,__] = min{_______,_______}; }
    }
  }
```

a. Copy the complete line containing the blanks in the Initialization step and fill in the blanks.
b. Copy the complete line containing the blanks in the Algorithm step and fill in the blanks.
c. Fill in the blank: The running time of the Algorithm is O(___).

gatecse-2002 algorithms graph-algorithms time-complexity normal descriptive shortest-path

Answer key

1.39.2 Shortest Path: GATE CSE 2003 | Question: 67

Let $G = (V, E)$ be an undirected graph with a subgraph $G_1 = (V_1, E_1)$. Weights are assigned to edges of G as follows.

$$w(e) = \begin{cases} 0, & \text{if } e \in E_1 \\ 1, & \text{otherwise} \end{cases}$$

A single-source shortest path algorithm is executed on the weighted graph (V, E, w) with an arbitrary vertex v_1 of V_1 as the source. Which of the following can always be inferred from the path costs computed?

A. The number of edges in the shortest paths from v_1 to all vertices of G
B. G_1 is connected
C. V_1 forms a clique in G
D. G_1 is a tree

gatecse-2003 algorithms graph-algorithms normal shortest-path

Answer key

1.39.3 Shortest Path: GATE CSE 2007 | Question: 41

In an unweighted, undirected connected graph, the shortest path from a node S to every other node is computed most efficiently, in terms of *time complexity*, by

A. Dijkstra's algorithm starting from S. B. Warshall's algorithm.

C. Performing a DFS starting from S.

D. Performing a BFS starting from S.

gatecse-2007 algorithms graph-algorithms easy shortest-path

Answer key

1.39.4 Shortest Path: GATE CSE 2020 | Question: 40

Let $G = (V, E)$ be a directed, weighted graph with weight function $w : E \to \mathbb{R}$. For some function $f : V \to \mathbb{R}$, for each edge $(u, v) \in E$, define $w'(u, v)$ as $w(u, v) + f(u) - f(v)$.

Which one of the options completes the following sentence so that it is TRUE?

"The shortest paths in G under w are shortest paths under w' too,_____".

A. for every $f : V \to \mathbb{R}$
B. if and only if $\forall u \in V,\ f(u)$ is positive
C. if and only if $\forall u \in V,\ f(u)$ is negative
D. if and only if $f(u)$ is the distance from s to u in the graph obtained by adding a new vertex s to G and edges of zero weight from s to every vertex of G

gatecse-2020 algorithms graph-algorithms two-marks shortest-path

Answer key

1.39.5 Shortest Path: GATE CSE 2025 | Set 1 | Question: 8

Let G be any undirected graph with positive edge weights, and T be a minimum spanning tree of G. For any two vertices, u and v, let $d_1(u, v)$ and $d_2(u, v)$ be the shortest distances between u and v in G and T, respectively. Which ONE of the options is CORRECT for all possible G, T, u and v ?

A. $d_1(u, v) = d_2(u, v)$
B. $d_1(u, v) \leq d_2(u, v)$
C. $d_1(u, v) \geq d_2(u, v)$
D. $d_1(u, v) \neq d_2(u, v)$

gatecse2025-set1 algorithms minimum-spanning-tree shortest-path one-mark

Answer key

1.39.6 Shortest Path: GATE CSE 2025 | Set 2 | Question: 27

Let G be an edge-weighted undirected graph with positive edge weights. Suppose a positive constant α is added to the weight of every edge.

Which ONE of the following statements is TRUE about the minimum spanning trees (MSTs) and shortest paths (SPs) in G before and after the edge weight update?

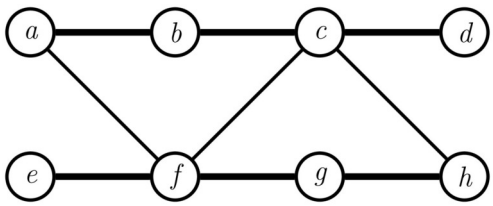
A. Every MST remains an MST, and every SP remains an SP.
B. MSTs need not remain MSTs, and every SP remains an SP.
C. Every MST remains an MST, and SPs need not remain SPs.
D. MSTs need not remain MSTs, and SPs need not remain SPs.

gatecse2025-set2 algorithms minimum-spanning-tree shortest-path two-marks

Answer key

1.39.7 Shortest Path: GATE DA 2025 | Question: 48

Let G be a simple, unweighted, and undirected graph. A subset of the vertices and edges of G are shown below.



It is given that $a-b-c-d$ is a shortest path between a and d; $e-f-g-h$ is a shortest path between e and h; $a-f-c-h$ is a shortest path between a and h. Which of the following is/are NOT the edges of G ?

A. (b,d) B. (b,g) C. (b,h) D. (e,g)

gateda-2025 algorithms shortest-path multiple-selects two-marks

Answer key

1.39.8 Shortest Path: GATE IT 2007 | Question: 3, UGCNET-June2012-III: 34

Consider a weighted, undirected graph with positive edge weights and let uv be an edge in the graph. It is known that the shortest path from the source vertex s to u has weight 53 and the shortest path from s to v has weight 65. Which one of the following statements is always TRUE?

A. Weight $(u,v) \leq 12$
B. Weight $(u,v) = 12$
C. Weight $(u,v) \geq 12$
D. Weight $(u,v) > 12$

gateit-2007 algorithms graph-algorithms normal ugcnetcse-june2012-paper3 shortest-path

Answer key

1.40 Sorting (22)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (7Q)

1.40.1 Sorting: GATE CSE 1988 | Question: 1iii

Quicksort is _____ efficient than heapsort in the worst case.

gate1988 algorithms sorting fill-in-the-blanks easy

Answer key

1.40.2 Sorting: GATE CSE 1990 | Question: 3-v

The complexity of comparison based sorting algorithms is:

A. $\Theta(n \log n)$
B. $\Theta(n)$
C. $\Theta\left(n^2\right)$
D. $\Theta(n\sqrt{n})$

gate1990 normal algorithms sorting easy time-complexity multiple-selects

Answer key

1.40.3 Sorting: GATE CSE 1991 | Question: 01,vii

The minimum number of comparisons required to sort 5 elements is _____

gate1991 normal algorithms sorting numerical-answers

Answer key

1.40.4 Sorting: GATE CSE 1991 | Question: 13

Give an optimal algorithm in pseudo-code for sorting a sequence of n numbers which has only k distinct numbers (k is not known a Priori). Give a brief analysis for the time-complexity of your algorithm.

gate1991 sorting time-complexity algorithms difficult descriptive

Answer key

1.40.5 Sorting: GATE CSE 1992 | Question: 02,ix

Following algorithm(s) can be used to sort n in the range $[1 \ldots n^3]$ in $O(n)$ time

a. Heap sort b. Quick sort c. Merge sort d. Radix sort

gate1992 easy algorithms sorting multiple-selects

Answer key

1.40.6 Sorting: GATE CSE 1996 | Question: 14

A two dimensional array $A[1..n][1..n]$ of integers is partially sorted if $\forall i, j \in [1..n-1], A[i][j] < A[i][j+1]$ and $A[i][j] < A[i+1][j]$

a. The smallest item in the array is at $A[i][j]$ where $i =$ __ and $j =$ __.

b. The smallest item is deleted. Complete the following $O(n)$ procedure to insert item x (which is guaranteed to be smaller than any item in the last row or column) still keeping A partially sorted.

```
procedure insert (x: integer);
var i,j: integer;
begin
   i:=1; j:=1, A[i][j]:=x;
   while (x > __ or x > __) do
      if A[i+1][j] < A[i][j] ___ then begin
         A[i][j]:=A[i+1][j]; i:=i+1;
      end
      else begin
         _____
      end
   A[i][j]:= ____
end
```

gate1996 algorithms sorting normal descriptive

Answer key

1.40.7 Sorting: GATE CSE 1998 | Question: 1.22

Give the correct matching for the following pairs:

(A)	$O(\log n)$	(P)	Selection
(B)	$O(n)$	(Q)	Insertion sort
(C)	$O(n \log n)$	(R)	Binary search
(D)	$O(n^2)$	(S)	Merge sort

A. A-R B-P C-Q D-S
B. A-R B-P C-S D-Q
C. A-P B-R C-S D-Q
D. A-P B-S C-R D-Q

gate1998 algorithms sorting easy match-the-following

Answer key

1.40.8 Sorting: GATE CSE 1999 | Question: 1.12

A sorting technique is called stable if

A. it takes $O(n \log n)$ time
B. it maintains the relative order of occurrence of non-distinct elements
C. it uses divide and conquer paradigm
D. it takes $O(n)$ space

gate1999 algorithms sorting easy

Answer key

1.40.9 Sorting: GATE CSE 1999 | Question: 8

Let A be an $n \times n$ matrix such that the elements in each row and each column are arranged in ascending order. Draw a decision tree, which finds 1^{st}, 2^{nd} and 3^{rd} smallest elements in minimum number of comparisons.

gate1999 algorithms sorting normal descriptive

Answer key

1.40.10 Sorting: GATE CSE 2000 | Question: 17

An array contains four occurrences of 0, five occurrences of 1, and three occurrences of 2 in any order. The array is to be sorted using swap operations (elements that are swapped need to be adjacent).

a. What is the minimum number of swaps needed to sort such an array in the worst case?
b. Give an ordering of elements in the above array so that the minimum number of swaps needed to sort the array is maximum.

gatecse-2000 algorithms sorting normal descriptive

Answer key

1.40.11 Sorting: GATE CSE 2005 | Question: 39

Suppose there are $\lceil \log n \rceil$ sorted lists of $\lfloor n/\log n \rfloor$ elements each. The time complexity of producing a sorted list of all these elements is: (Hint:Use a heap data structure)

A. $O(n \log \log n)$
B. $\Theta(n \log n)$
C. $\Omega(n \log n)$
D. $\Omega\left(n^{3/2}\right)$

gatecse-2005 algorithms sorting normal

Answer key

1.40.12 Sorting: GATE CSE 2006 | Question: 14, ISRO2011-14

Which one of the following in place sorting algorithms needs the minimum number of swaps?

A. Quick sort
B. Insertion sort
C. Selection sort
D. Heap sort

gatecse-2006 algorithms sorting easy isro2011

Answer key

1.40.13 Sorting: GATE CSE 2007 | Question: 14

Which of the following sorting algorithms has the lowest worse-case complexity?

A. Merge sort
B. Bubble sort
C. Quick sort
D. Selection sort

gatecse-2007 algorithms sorting time-complexity easy

Answer key

1.40.14 Sorting: GATE CSE 2016 | Set 1 | Question: 13

The worst case running times of *Insertion sort* , *Merge sort* and *Quick sort*, respectively are:

A. $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$
B. $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \log n)$
C. $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$
D. $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

gatecse-2016-set1 algorithms sorting easy

Answer key

1.40.15 Sorting: GATE CSE 2016 | Set 2 | Question: 13

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in the ascending order, which of the following are **TRUE**?

I. Quicksort runs in $\Theta(n^2)$ time
II. Bubblesort runs in $\Theta(n^2)$ time
III. Mergesort runs in $\Theta(n)$ time
IV. Insertion sort runs in $\Theta(n)$ time

A. I and II only B. I and III only C. II and IV only D. I and IV only

gatecse-2016-set2 algorithms sorting time-complexity normal ambiguous

Answer key

1.40.16 Sorting: GATE CSE 2021 | Set 1 | Question: 9

Consider the following array.

23	32	45	69	72	73	89	97

Which algorithm out of the following options uses the least number of comparisons (among the array elements) to sort the above array in ascending order?

A. Selection sort
B. Mergesort
C. Insertion sort
D. Quicksort using the last element as pivot

gatecse-2021-set1 algorithms sorting one-mark

Answer key

1.40.17 Sorting: GATE CSE 2024 | Set 2 | Question: 25

Let A be an array containing integer values. The distance of A is defined as the minimum number of elements in A that must be replaced with another integer so that the resulting array is sorted in non-decreasing order. The distance of the array $[2, 5, 3, 1, 4, 2, 6]$ is _____.

gatecse-2024-set2 numerical-answers algorithms sorting one-mark

Answer key

1.40.18 Sorting: GATE CSE 2025 | Set 2 | Question: 10

Consider an unordered list of N distinct integers.

What is the minimum number of element comparisons required to find an integer in the list that is NOT the largest in the list?

A. 1 B. $N-1$ C. N D. $2N-1$

gatecse2025-set2 algorithms sorting one-mark

Answer key

1.40.19 Sorting: GATE DA 2026 | Question: 39

Consider the problem of sorting the given array in ascending order:

$$P = [1, 2, 3, 5, 4]$$

Consider two sorting algorithms Bubble Sort (BS) and Insertion Sort (IS).
Let $\mathrm{N1}$ be the total number of comparisons done by BS on the elements of P and $\mathrm{N2}$ be the total number of comparisons done by IS on the elements of P.

Which of the following options is/are correct?

A. $\mathrm{N1} = 10,\ \mathrm{N2} = 4$
B. $\mathrm{N1} > \mathrm{N2}$
C. IS on P will perform only one swap
D. Both BS and IS on P will make at least one unnecessary comparison (i.e., comparing elements that are already in correct order)

gateda-2026 algorithms sorting multiple-selects two-marks

Answer key

1.40.20 Sorting: GATE DS&AI 2024 | Question: 35

Consider the following sorting algorithms:

i. Bubble sort
ii. Insertion sort
iii. Selection sort

Which ONE among the following choices of sorting algorithms sorts the numbers in the array $[4,3,2,1,5]$ in increasing order after exactly two passes over the array?

A. (i) only B. (iii) only C. (i) and (iii) only D. (ii) and (iii) only

gate-ds-ai-2024 algorithms sorting two-marks

Answer key

1.40.21 Sorting: GATE IT 2005 | Question: 59

Let a and b be two sorted arrays containing n integers each, in non-decreasing order. Let c be a sorted array containing $2n$ integers obtained by merging the two arrays a and b. Assuming the arrays are indexed starting from 0, consider the following four statements

I. $a[i] \geq b[i] \Rightarrow c[2i] \geq a[i]$
II. $a[i] \geq b[i] \Rightarrow c[2i] \geq b[i]$
III. $a[i] \geq b[i] \Rightarrow c[2i] \leq a[i]$
IV. $a[i] \geq b[i] \Rightarrow c[2i] \leq b[i]$

Which of the following is TRUE?

A. only I and II B. only I and IV C. only II and III D. only III and IV

gateit-2005 algorithms sorting normal

Answer key

1.40.22 Sorting: GATE IT 2008 | Question: 43

If we use Radix Sort to sort n integers in the range $\left(n^{k/2}, n^k\right]$, for some $k > 0$ which is independent of n, the time taken would be?

A. $\Theta(n)$ B. $\Theta(kn)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

gateit-2008 algorithms sorting normal

Answer key

1.41 Space Complexity (1)

1.41.1 Space Complexity: GATE CSE 2005 | Question: 81a

```
double foo(int n)
{
   int i;
   double sum;
   if(n == 0)
   {
      return 1.0;
   }
   else
   {
      sum = 0.0;
      for(i = 0; i < n; i++)
      {
         sum += foo(i);
      }
      return sum;
   }
}
```

The space complexity of the above code is?

A. $O(1)$ B. $O(n)$ C. $O(n!)$ D. n^n

gatecse-2005 algorithms recursion normal space-complexity

Answer key

1.42 Strongly Connected Components (3)

1.42.1 Strongly Connected Components: GATE CSE 2008 | Question: 7

The most efficient algorithm for finding the number of connected components in an undirected graph on n vertices and m edges has time complexity

A. $\Theta(n)$ B. $\Theta(m)$ C. $\Theta(m+n)$ D. $\Theta(mn)$

gatecse-2008 algorithms graph-algorithms time-complexity normal strongly-connected-components

Answer key

1.42.2 Strongly Connected Components: GATE CSE 2018 | Question: 43

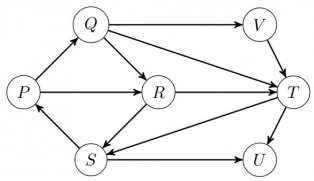
Let G be a graph with $100!$ vertices, with each vertex labelled by a distinct permutation of the numbers $1, 2, \ldots, 100$. There is an edge between vertices u and v if and only if the label of u can be obtained by swapping two adjacent numbers in the label of v. Let y denote the degree of a vertex in G, and z denote the number of connected components in G. Then, $y + 10z =$ _____.

gatecse-2018 algorithms graph-algorithms numerical-answers two-marks strongly-connected-components

Answer key

1.42.3 Strongly Connected Components: GATE IT 2006 | Question: 46

Which of the following is the correct decomposition of the directed graph given below into its strongly connected components?



A. $\{P, Q, R, S\}, \{T\}, \{U\}, \{V\}$
B. $\{P, Q, R, S, T, V\}, \{U\}$
C. $\{P, Q, S, T, V\}, \{R\}, \{U\}$
D. $\{P, Q, R, S, T, U, V\}$

gateit-2006 algorithms graph-algorithms normal strongly-connected-components

Answer key

1.43 Time Complexity (31)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (9Q)

1.43.1 Time Complexity: GATE CSE 1988 | Question: 6i

Given below is the sketch of a program that represents the path in a two-person game tree by the sequence of active procedure calls at any time. The program assumes that the payoffs are real number in a limited range; that the constant INF is larger than any positive payoff and its negation is smaller than any negative payoff and that there is a function “payoff” and that computes the payoff for any board that is a leaf. The type “boardtype” has been suitably declared to represent board positions. It is player-1’s move if mode = MAX and player-2’s move if mode=MIN. The type modetype = (MAX, MIN). The functions “min” and “max” find the minimum and maximum of two real numbers.

```
function search(B: boardtype; mode: modetype): real;
```

```
var
   C:boardtype; {a child of board B}
   value:real;
begin
   if B is a leaf then
      return (payoff(B))
   else
      begin
      if mode = MAX then value :=-INF
      else
         value:INF;
      for each child C of board B do
      if mode = MAX then
         value:=max (value, search (C, MIN))
      else
         value:=min(value, search(C, MAX))
      return(value)
      end
end; (search)
```

Comment on the working principle of the above program. Suggest a possible mechanism for reducing the amount of search.

gate1988 normal descriptive algorithms time-complexity

Answer key

1.43.2 Time Complexity: GATE CSE 1989 | Question: 2-iii

Match the pairs in the following:

(A)	$O(\log n)$	(p)	Heapsort
(B)	$O(n)$	(q)	Depth-first search
(C)	$O(n \log n)$	(r)	Binary search
(D)	$O(n^2)$	(s)	Selection of the k^{th} smallest element in a set of n elements

gate1989 match-the-following algorithms time-complexity

Answer key

1.43.3 Time Complexity: GATE CSE 1993 | Question: 8.7

$\sum_{1 \leq k \leq n} O(n)$, where $O(n)$ stands for order n is:

A. $O(n)$ B. $O(n^2)$ C. $O(n^3)$ D. $O(3n^2)$

E. $O(1.5n^2)$

gate1993 algorithms time-complexity easy

Answer key

1.43.4 Time Complexity: GATE CSE 1999 | Question: 1.13

Suppose we want to arrange the n numbers stored in any array such that all negative values occur before all positive ones. Minimum number of exchanges required in the worst case is

A. $n-1$ B. n C. $n+1$ D. None of the above

gate1999 algorithms time-complexity normal

Answer key

1.43.5 Time Complexity: GATE CSE 1999 | Question: 1.16

If n is a power of 2, then the minimum number of multiplications needed to compute a^n is

A. $\log_2 n$ B. $\sqrt{n}$ C. $n-1$ D. n

gate1999 algorithms time-complexity normal

Answer key

1.43.6 Time Complexity: GATE CSE 1999 | Question: 11a

Consider the following algorithms. Assume, procedure A and procedure B take $O(1)$ and $O(1/n)$ unit of time respectively. Derive the time complexity of the algorithm in O-notation.

```
algorithm what (n)
begin
   if n = 1 then call A
   else
      begin
         what (n-1);
         call B(n)
      end
end.
```

gate1999 algorithms time-complexity normal descriptive

Answer key

1.43.7 Time Complexity: GATE CSE 2000 | Question: 1.15

Let S be a sorted array of n integers. Let $T(n)$ denote the time taken for the most efficient algorithm to determined if there are two elements with sum less than 1000 in S. Which of the following statement is true?

A. $T(n)$ is $O(1)$
B. $n \leq T(n) \leq n \log_2 n$
C. $n \log_2 n \leq T(n) < \frac{n}{2}$
D. $T(n) = \left(\frac{n}{2}\right)$

gatecse-2000 easy algorithms time-complexity

Answer key

1.43.8 Time Complexity: GATE CSE 2003 | Question: 66

The cube root of a natural number n is defined as the largest natural number m such that $(m^3 \leq n)$. The complexity of computing the cube root of n (n is represented by binary notation) is

A. $O(n)$ but not $O(n^{0.5})$
B. $O(n^{0.5})$ but not $O((\log n)^k)$ for any constant $k > 0$
C. $O((\log n)^k)$ for some constant $k > 0$, but not $O((\log \log n)^m)$ for any constant $m > 0$
D. $O((\log \log n)^k)$ for some constant $k > 0.5$, but not $O((\log \log n)^{0.5})$

gatecse-2003 algorithms time-complexity normal

Answer key

1.43.9 Time Complexity: GATE CSE 2004 | Question: 39

Two matrices M_1 and M_2 are to be stored in arrays A and B respectively. Each array can be stored either in row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute $M_1 \times M_2$ will be

A. best if A is in row-major, and B is in column-major order
B. best if both are in row-major order
C. best if both are in column-major order
D. independent of the storage scheme

gatecse-2004 algorithms time-complexity easy

Answer key

1.43.10 Time Complexity: GATE CSE 2004 | Question: 82

Let $A[1, \ldots, n]$ be an array storing a bit (1 or 0) at each location, and $f(m)$ is a function whose time complexity is $\Theta(m)$. Consider the following program fragment written in a C like language:

```
counter = 0;
for (i=1; i<=n; i++)
{
  if (a[i] == 1) counter++;
  else {f (counter); counter = 0;}
}
```

The complexity of this program fragment is

A. $\Omega(n^2)$
B. $\Omega(n \log n)$ and $O(n^2)$
C. $\Theta(n)$
D. $o(n)$

gatecse-2004 algorithms time-complexity normal

Answer key

1.43.11 Time Complexity: GATE CSE 2006 | Question: 15

Consider the following C-program fragment in which i, j and n are integer variables.

```
for( i = n, j = 0; i > 0; i /= 2, j +=i );
```

Let $val(j)$ denote the value stored in the variable j after termination of the for loop. Which one of the following is true?

A. $val(j) = \Theta(\log n)$
B. $val(j) = \Theta(\sqrt{n})$
C. $val(j) = \Theta(n)$
D. $val(j) = \Theta(n \log n)$

gatecse-2006 algorithms normal time-complexity

Answer key

1.43.12 Time Complexity: GATE CSE 2007 | Question: 15, ISRO2016-26

Consider the following segment of C-code:

```
int j, n;
j = 1;
while (j <= n)
  j = j * 2;
```

The number of comparisons made in the execution of the loop for any $n > 0$ is:

A. $\lceil \log_2 n \rceil + 1$
B. n
C. $\lceil \log_2 n \rceil$
D. $\lfloor \log_2 n \rfloor + 1$

gatecse-2007 algorithms time-complexity normal isro2016

Answer key

1.43.13 Time Complexity: GATE CSE 2007 | Question: 45

What is the time complexity of the following recursive function?

```
int  DoSomething (int n) {
  if (n <= 2)
    return 1;
  else
    return (DoSomething (floor (sqrt(n))) + n);
}
```

A. $\Theta(n^2)$
B. $\Theta(n \log_2 n)$
C. $\Theta(\log_2 n)$
D. $\Theta(\log_2 \log_2 n)$

gatecse-2007 algorithms time-complexity normal

Answer key

1.43.14 Time Complexity: GATE CSE 2007 | Question: 50

An array of n numbers is given, where n is an even number. The maximum as well as the minimum of these n numbers needs to be determined. Which of the following is **TRUE** about the number of comparisons needed?

A. At least $2n - c$ comparisons, for some constant c are needed.
B. At most $1.5n - 2$ comparisons are needed.
C. At least $n \log_2 n$ comparisons are needed
D. None of the above

gatecse-2007 algorithms time-complexity easy

Answer key

1.43.15 Time Complexity: GATE CSE 2007 | Question: 51

Consider the following C program segment:

```
int IsPrime (n)
  {
     int i, n;
     for (i=2; i<=sqrt(n);i++)
        if(n%i == 0)
           {printf("Not Prime \n"); return 0;}
     return 1;
  }
```

Let $T(n)$ denote number of times the for loop is executed by the program on input n. Which of the following is TRUE?

A. $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(\sqrt{n})$
B. $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$
C. $T(n) = O(n)$ and $T(n) = \Omega(\sqrt{n})$
D. None of the above

gatecse-2007 algorithms time-complexity normal

Answer key

1.43.16 Time Complexity: GATE CSE 2008 | Question: 40

The minimum number of comparisons required to determine if an integer appears more than $\frac{n}{2}$ times in a sorted array of n integers is

A. $\Theta(n)$ B. $\Theta(\log n)$ C. $\Theta(\log^* n)$ D. $\Theta(1)$

gatecse-2008 normal algorithms time-complexity

Answer key

1.43.17 Time Complexity: GATE CSE 2008 | Question: 47

We have a binary heap on n elements and wish to insert n more elements (not necessarily one after another) into this heap. The total time required for this is

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

gatecse-2008 algorithms time-complexity normal

Answer key

1.43.18 Time Complexity: GATE CSE 2008 | Question: 74

Consider the following C functions:

```
int f1 (int n)
{
   if(n == 0 || n == 1)
      return n;
   else
```

```
    return (2 * f1(n-1) + 3 * f1(n-2));
}
int f2(int n)
{
   int i;
   int X[N], Y[N], Z[N];
   X[0] = Y[0] = Z[0] = 0;
   X[1] = 1; Y[1] = 2; Z[1] = 3;
   for(i = 2; i <= n; i++){
      X[i] = Y[i-1] + Z[i-2];
      Y[i] = 2 * X[i];
      Z[i] = 3 * X[i];
   }
   return X[n];
}
```

The running time of $f1(n)$ and $f2(n)$ are

A. $\Theta(n)$ and $\Theta(n)$
B. $\Theta(2^n)$ and $\Theta(n)$
C. $\Theta(n)$ and $\Theta(2^n)$
D. $\Theta(2^n)$ and $\Theta(2^n)$

gatecse-2008 algorithms time-complexity normal

Answer key

1.43.19 Time Complexity: GATE CSE 2008 | Question: 75

Consider the following C functions:

```
int f1 (int n)
{
   if(n == 0 || n == 1)
      return n;
   else
      return (2 * f1(n-1) + 3 * f1(n-2));
}
int f2(int n)
{
   int i;
   int X[N], Y[N], Z[N];
   X[0] = Y[0] = Z[0] = 0;
   X[1] = 1; Y[1] = 2; Z[1] = 3;
   for(i = 2; i <= n; i++){
      X[i] = Y[i-1] + Z[i-2];
      Y[i] = 2 * X[i];
      Z[i] = 3 * X[i];
   }
   return X[n];
}
```

$f1(8)$ and $f2(8)$ return the values

A. 1661 and 1640
B. 59 and 59
C. 1640 and 1640
D. 1640 and 1661

gatecse-2008 normal algorithms time-complexity

Answer key

1.43.20 Time Complexity: GATE CSE 2010 | Question: 12

Two alternative packages A and B are available for processing a database having 10^k records. Package A requires $0.0001n^2$ time units and package B requires $10n\log_{10} n$ time units to process n records. What is the smallest value of k for which package B will be preferred over A?

A. 12
B. 10
C. 6
D. 5

gatecse-2010 algorithms time-complexity easy

Answer key

1.43.21 Time Complexity: GATE CSE 2014 | Set 1 | Question: 42

Consider the following pseudo code. What is the total number of multiplications to be performed?

```
D = 2
for i = 1 to n do
```

```
for j = i to n do
   for k = j + 1 to n do
      D = D * 3
```

A. Half of the product of the 3 consecutive integers.
B. One-third of the product of the 3 consecutive integers.
C. One-sixth of the product of the 3 consecutive integers.
D. None of the above.

gatecse-2014-set1 algorithms time-complexity normal

Answer key

1.43.22 Time Complexity: GATE CSE 2015 | Set 1 | Question: 40

An algorithm performs $(\log N)^{\frac{1}{2}}$ find operations , N insert operations, $(\log N)^{\frac{1}{2}}$ delete operations, and $(\log N)^{\frac{1}{2}}$ decrease-key operations on a set of data items with keys drawn from a linearly ordered set . For a delete operation, a pointer is provided to the record that must be deleted . For the decrease-key operation, a pointer is provided to the record that has its key decreased. Which one of the following data structures is the most suited for the algorithm to use, if the goal is to achieve the best total asymptotic complexity considering all the operations?

A. Unsorted array
B. Min - heap
C. Sorted array
D. Sorted doubly linked list

gatecse-2015-set1 algorithms data-structures normal time-complexity

Answer key

1.43.23 Time Complexity: GATE CSE 2015 | Set 2 | Question: 22

An unordered list contains n distinct elements. The number of comparisons to find an element in this list that is neither maximum nor minimum is

A. $\Theta(n \log n)$ B. $\Theta(n)$ C. $\Theta(\log n)$ D. $\Theta(1)$

gatecse-2015-set2 algorithms time-complexity easy

Answer key

1.43.24 Time Complexity: GATE CSE 2017 | Set 2 | Question: 03

Match the algorithms with their time complexities:

Algorithms	Time Complexity
P. Tower of Hanoi with n disks	i. $\Theta(n^2)$
Q. Binary Search given n sorted numbers	ii. $\Theta(n \log n)$
R. Heap sort given n numbers at the worst case	iii. $\Theta(2^n)$
S. Addition of two $n \times n$ matrices	iv. $\Theta(\log n)$

A. $P \rightarrow (iii)$ $Q \rightarrow (iv)$ $r \rightarrow (i)$ $S \rightarrow (ii)$
B. $P \rightarrow (iv)$ $Q \rightarrow (iii)$ $r \rightarrow (i)$ $S \rightarrow (ii)$
C. $P \rightarrow (iii)$ $Q \rightarrow (iv)$ $r \rightarrow (ii)$ $S \rightarrow (i)$
D. $P \rightarrow (iv)$ $Q \rightarrow (iii)$ $r \rightarrow (ii)$ $S \rightarrow (i)$

gatecse-2017-set2 algorithms time-complexity match-the-following easy

Answer key

1.43.25 Time Complexity: GATE CSE 2017 | Set 2 | Question: 38

Consider the following C function

```
int fun(int n) {
   int i, j;
   for(i=1; i<=n; i++) {
```

```
    for (j=1; j<n; j+=i) {
      printf("%d %d", i, j);
    }
  }
}
```

Time complexity of fun in terms of Θ notation is

A. $\Theta(n\sqrt{n})$
B. $\Theta(n^2)$
C. $\Theta(n \log n)$
D. $\Theta(n^2 \log n)$

gatecse-2017-set2 algorithms time-complexity

Answer key

1.43.26 Time Complexity: GATE CSE 2019 | Question: 37

There are n unsorted arrays: $A_1, A_2, \ldots, A_n$. Assume that n is odd.Each of $A_1, A_2, \ldots, A_n$ contains n distinct elements. There are no common elements between any two arrays. The worst-case time complexity of computing the median of the medians of $A_1, A_2, \ldots, A_n$ is

A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $\Omega(n^2 \log n)$

gatecse-2019 algorithms time-complexity two-marks

Answer key

1.43.27 Time Complexity: GATE CSE 2024 | Set 1 | Question: 7

Given an integer array of size N, we want to check if the array is sorted (in either ascending or descending order). An algorithm solves this problem by making a single pass through the array and comparing each element of the array only with its adjacent elements. The worst-case time complexity of this algorithm is

A. both $\mathrm{O}(N)$ and $\Omega(N)$
B. $\mathrm{O}(N)$ but not $\Omega(N)$
C. $\Omega(N)$ but not $\mathrm{O}(N)$
D. neither $\mathrm{O}(N)$ nor $\Omega(N)$

gatecse-2024-set1 algorithms time-complexity one-mark

Answer key

1.43.28 Time Complexity: GATE CSE 2026 | Set 1 | Question: 7

Consider the following recurrence relations:

For all $n > 1$,

$$T_1(n) = 4T_1\left(\frac{n}{2}\right) + T_2(n)$$
$$T_2(n) = 5T_2\left(\frac{n}{4}\right) + \Theta\left(\log_2 n\right)$$

Assume that for all $n \leq 1, T_1(n) = 1$ and $T_2(n) = 1$.

Which one of the following options is correct?

A. $T_1(n) = \Theta\left(n^2\right)$
B. $T_1(n) = \Theta\left(n^2 \log_2 n\right)$
C. $T_1(n) = \Theta\left(n^{\log_4 5}\right)$
D. $T_1(n) = \Theta\left(n^{\log_4 5} \log_2 n\right)$

gatecse-2026-set1 algorithms time-complexity one-mark

Answer key

1.43.29 Time Complexity: GATE CSE 2026 | Set 2 | Question: 28

Consider an array A of integers of size n. The indices of A run from 1 to n. An algorithm is to be designed to check whether A satisfies the condition given below.

$\forall i, j \in \{1, \ldots, n-1\}$ such that $i > j, (A[i+1] - A[i]) > (A[j+1] - A[j])$

Which one of the following gives the worst case time complexity of the fastest algorithm that can be designed for the problem?

A. $\Theta(n)$
B. $\Theta(\log(n))$
C. $\Theta(n \log(n))$
D. $\Theta\left(n^2\right)$

gatecse-2026-set2 algorithms time-complexity two-marks

Answer key

1.43.30 Time Complexity: GATE IT 2007 | Question: 17

Exponentiation is a heavily used operation in public key cryptography. Which of the following options is the tightest upper bound on the number of multiplications required to compute $b^n \bmod m, 0 \leq b, n \leq m$?

A. $O(\log n)$
B. $O(\sqrt{n})$
C. $O\left(\frac{n}{\log n}\right)$
D. $O(n)$

gateit-2007 algorithms time-complexity normal

Answer key

1.43.31 Time Complexity: GATE IT 2007 | Question: 81

Let $P_1, P_2, \ldots, P_n$ be n points in the xy-plane such that no three of them are collinear. For every pair of points P_i and P_j, let L_{ij} be the line passing through them. Let L_{ab} be the line with the steepest gradient among all $n(n-1)/2$ lines.

The time complexity of the best algorithm for finding P_a and P_b is

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n \log^2 n)$
D. $\Theta(n^2)$

gateit-2007 algorithms time-complexity normal

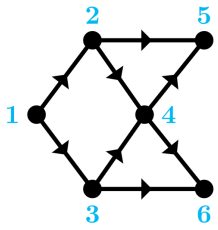
Answer key

1.44 Topological Sort (4)

✍ **Practice Test:** Test 1 (7Q)

1.44.1 Topological Sort: GATE CSE 2007 | Question: 5

Consider the DAG with $V = \{1, 2, 3, 4, 5, 6\}$ shown below.



Which of the following is not a topological ordering?

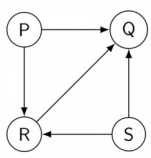
A. 1 2 3 4 5 6
B. 1 3 2 4 5 6
C. 1 3 2 4 6 5
D. 3 2 4 1 6 5

gatecse-2007 algorithms graph-algorithms topological-sort easy

Answer key

1.44.2 Topological Sort: GATE CSE 2014 | Set 1 | Question: 13

Consider the directed graph below given.



Which one of the following is **TRUE**?

A. The graph does not have any topological ordering.

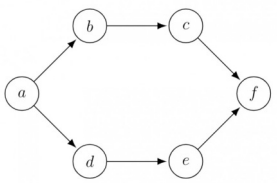
B. Both PQRS and SRQP are topological orderings.
C. Both PSRQ and SPRQ are topological orderings.
D. PSRQ is the only topological ordering.

gatecse-2014-set1 graph-algorithms easy topological-sort

Answer key

1.44.3 Topological Sort: GATE CSE 2016 | Set 1 | Question: 11

Consider the following directed graph:



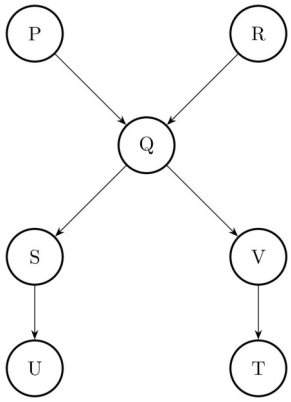
The number of different topological orderings of the vertices of the graph is _____.

gatecse-2016-set1 algorithms graph-algorithms normal numerical-answers topological-sort

Answer key

1.44.4 Topological Sort: GATE DS&AI 2024 | Question: 41

Consider the directed acyclic graph (DAG) below:



Which of the following is/are valid vertex orderings that can be obtained from a topological sort of the DAG?

A. P Q R S T U V
B. P R Q V S U T
C. P Q R S V U T
D. P R Q S V T U

gate-ds-ai-2024 algorithms topological-sort directed-acyclic-graph multiple-selects two-marks

Answer key

1.45 Tree Traversal (1)

1.45.1 Tree Traversal: GATE DA 2026 | Question: 15

You are given the following Pre-order and In-order traversals of a Binary Tree T with nodes E, F, G, P, Q, R, S.

Pre-order:	P	Q	S	E	R	F	G
In-order:	S	Q	E	P	F	R	G

Which of the following statements is/are true about the Binary Tree T?

A. Node P is the root of T
B. The Post-order traversal of T is: S
C. Node Q has only one child
D. The left subtree of node R

contains the node G

gateda-2026 algorithms tree-traversal multiple-selects one-mark

Answer key

1.46 Uniform Hashing (1)

1.46.1 Uniform Hashing: GATE CSE 2022 | Question: 6

Suppose we are given n keys, m hash table slots, and two simple uniform hash functions h_1 and h_2. Further suppose our hashing scheme uses h_1 for the odd keys and h_2 for the even keys. What is the expected number of keys in a slot?

A. $\frac{m}{n}$ B. $\frac{n}{m}$ C. $\frac{2n}{m}$ D. $\frac{n}{2m}$

gatecse-2022 algorithms hashing uniform-hashing one-mark

Answer key

Answer Keys

1.1.1	N/A	1.1.2	N/A	1.1.3	B	1.1.4	C	1.1.5	12
1.1.6	29	1.1.7	A;C	1.1.8	B	1.2.1	N/A	1.2.2	B
1.2.3	C	1.2.4	A	1.2.5	B	1.2.6	A	1.2.7	C
1.2.8	C	1.2.9	C	1.3.1	A;B	1.3.2	B	1.3.3	X
1.3.4	D	1.3.5	B	1.3.6	A	1.3.7	C	1.3.8	C
1.3.9	D	1.3.10	A	1.3.11	C	1.3.12	C	1.3.13	D
1.3.14	B	1.3.15	D	1.3.16	A	1.3.17	A;C	1.3.18	A;D
1.3.19	A	1.3.20	A	1.3.21	D	1.3.22	A	1.4.1	B
1.4.2	C	1.5.1	B;C	1.6.1	B	1.6.2	C	1.6.3	10:10
1.6.4	A	1.7.1	D	1.7.2	7 : 7	1.7.3	A;C	1.8.1	11 : 11
1.9.1	C	1.9.2	D	1.9.3	C	1.10.1	N/A	1.11.1	TBA
1.12.1	B;D	1.12.2	75:75	1.13.1	N/A	1.13.2	B	1.13.3	D
1.13.4	A	1.13.5	D	1.14.1	929 : 929	1.14.2	A	1.15.1	13
1.15.2	10:10	1.16.1	B	1.16.2	C	1.16.3	C	1.16.4	B
1.16.5	B	1.16.6	A	1.16.7	34	1.16.8	150	1.16.9	C
1.16.10	A	1.17.1	B	1.17.2	D	1.17.3	A	1.17.4	A
1.17.5	B	1.17.6	A	1.17.7	A;B	1.17.8	A	1.17.9	A
1.17.10	C	1.17.11	D	1.18.1	N/A	1.18.2	C	1.18.3	C
1.18.4	C	1.18.5	D	1.18.6	D	1.18.7	C	1.18.8	C
1.18.9	B	1.18.10	19	1.18.11	D	1.18.12	31	1.18.13	D
1.18.14	A	1.18.15	5040	1.18.16	C	1.18.17	60	1.18.18	6:6
1.18.19	A	1.18.20	D	1.18.21	D	1.18.22	D	1.18.23	B
1.19.1	A	1.19.2	B	1.19.3	D	1.19.4	A	1.19.5	16
1.20.1	B	1.20.2	N/A	1.20.3	C	1.20.4	C	1.20.5	3:3
1.20.6	D	1.21.1	C	1.21.2	D	1.22.1	2.33	1.22.2	A
1.22.3	D	1.22.4	225	1.22.5	B	1.22.6	A	1.23.1	N/A
1.23.2	N/A	1.23.3	C	1.23.4	A	1.23.5	N/A	1.23.6	C

1.23.7	C	1.23.8	N/A	1.23.9	C	1.23.10	D	1.23.11	A
1.23.12	B	1.23.13	D	1.23.14	C	1.23.15	C	1.23.16	D
1.23.17	D	1.23.18	D	1.23.19	B	1.23.20	C	1.23.21	B
1.23.22	D	1.23.23	B	1.23.24	A	1.23.25	9	1.23.26	A
1.23.27	D	1.23.28	51	1.23.29	0	1.23.30	D	1.23.31	81
1.23.32	1023 : 1023	1.23.33	15 : 15	1.23.34	D	1.23.35	C	1.23.36	C
1.23.37	D	1.23.38	C	1.24.1	A	1.24.2	D	1.25.1	B
1.25.2	A;C	1.26.1	C	1.26.2	D	1.27.1	C	1.27.2	1500
1.27.3	C	1.28.1	147.1 : 148.1	1.29.1	B	1.29.2	B	1.29.3	B
1.29.4	3:3	1.30.1	C	1.30.2	358	1.31.1	B;D;E	1.31.2	N/A
1.31.3	2	1.31.4	N/A	1.31.5	N/A	1.31.6	C	1.31.7	N/A
1.31.8	B	1.31.9	C	1.31.10	B	1.31.11	D	1.31.12	D
1.31.13	D	1.31.14	D	1.31.15	B	1.31.16	B	1.31.17	C
1.31.18	X	1.31.19	6	1.31.20	69	1.31.21	995	1.31.22	A
1.31.23	7	1.31.24	B	1.31.25	4	1.31.26	D	1.31.27	99
1.31.28	3 : 3	1.31.29	C	1.31.30	A;B;C	1.31.31	24	1.31.32	9
1.31.33	5:5	1.31.34	A;D	1.31.35	A	1.32.1	435:435	1.33.1	D
1.33.2	C	1.34.1	C	1.34.2	N/A	1.34.3	N/A	1.34.4	C
1.34.5	C	1.34.6	B	1.34.7	B	1.34.8	B	1.34.9	C
1.34.10	A	1.34.11	B	1.34.12	A	1.34.13	0.08	1.34.14	D
1.34.15	0	1.35.1	N/A	1.35.2	N/A	1.35.3	N/A	1.35.4	N/A
1.35.5	N/A	1.35.6	N/A	1.35.7	N/A	1.35.8	B	1.35.9	A
1.35.10	N/A	1.35.11	B	1.35.12	N/A	1.35.13	B	1.35.14	C
1.35.15	B	1.35.16	D	1.35.17	A	1.35.18	B	1.35.19	D
1.35.20	X	1.35.21	A	1.35.22	D	1.35.23	A	1.35.24	A
1.35.25	B	1.35.26	2.32 : 2.33	1.35.27	B	1.35.28	A	1.35.29	C
1.35.30	C	1.35.31	A	1.35.32	A	1.35.33	A;B	1.35.34	B
1.35.35	C	1.35.36	A	1.36.1	C	1.36.2	A	1.36.3	10230
1.36.4	60 : 60	1.36.5	65536 : 65536	1.37.1	N/A	1.37.2	C	1.37.3	A
1.37.4	C	1.37.5	A	1.37.6	5	1.37.7	B;C;D	1.37.8	D
1.38.1	A	1.38.2	B	1.39.1	N/A	1.39.2	B	1.39.3	D
1.39.4	A	1.39.5	B	1.39.6	C	1.39.7	A;C;D	1.39.8	C
1.40.1	N/A	1.40.2	A	1.40.3	7	1.40.4	N/A	1.40.5	D
1.40.6	N/A	1.40.7	B	1.40.8	B	1.40.9	N/A	1.40.10	N/A
1.40.11	A	1.40.12	C	1.40.13	A	1.40.14	D	1.40.15	D
1.40.16	C	1.40.17	3	1.40.18	A	1.40.19	B;C;D	1.40.20	B
1.40.21	C	1.40.22	C	1.41.1	B	1.42.1	C	1.42.2	109
1.42.3	B	1.43.1	N/A	1.43.2	N/A	1.43.3	B;C;D;E	1.43.4	D
1.43.5	A	1.43.6	N/A	1.43.7	A	1.43.8	C	1.43.9	D
1.43.10	C	1.43.11	C	1.43.12	D	1.43.13	D	1.43.14	B

1.43.15	B
1.43.20	C
1.43.25	C
1.43.30	A
1.44.4	B;D

1.43.16	B
1.43.21	C
1.43.26	C
1.43.31	B
1.45.1	A;B

1.43.17	B
1.43.22	A
1.43.27	A
1.44.1	D
1.46.1	B

1.43.18	B
1.43.23	D
1.43.28	A
1.44.2	C

1.43.19	C
1.43.24	C
1.43.29	A
1.44.3	6